Unit -1

Introduction to cloud computing: Introduction, Characteristics of cloud computing, Cloud Models, Cloud Services Examples, Cloud Based services and applications

Cloud concepts and Technologies: Virtualization, Load balancing, Scalability and Elasticity, Deployment, Replication, Monitoring, Software defined, Network function virtualization, Map Reduce, Identity and Access Management, services level Agreements, Billing.

Cloud Services and Platforms: Open-Source Private Cloud software

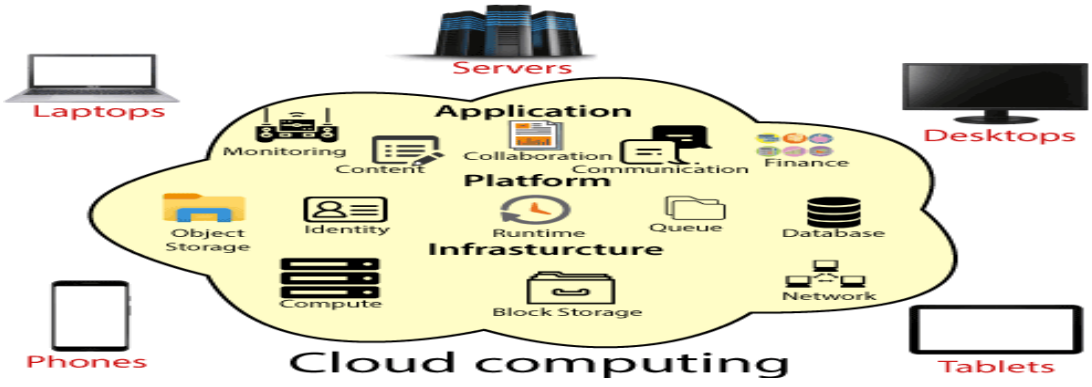
Introduction to Cloud Computing

Cloud Computing is the delivery of computing services such as servers, storage, databases, networking, software, analytics, intelligence, and more, over the Cloud (Internet).

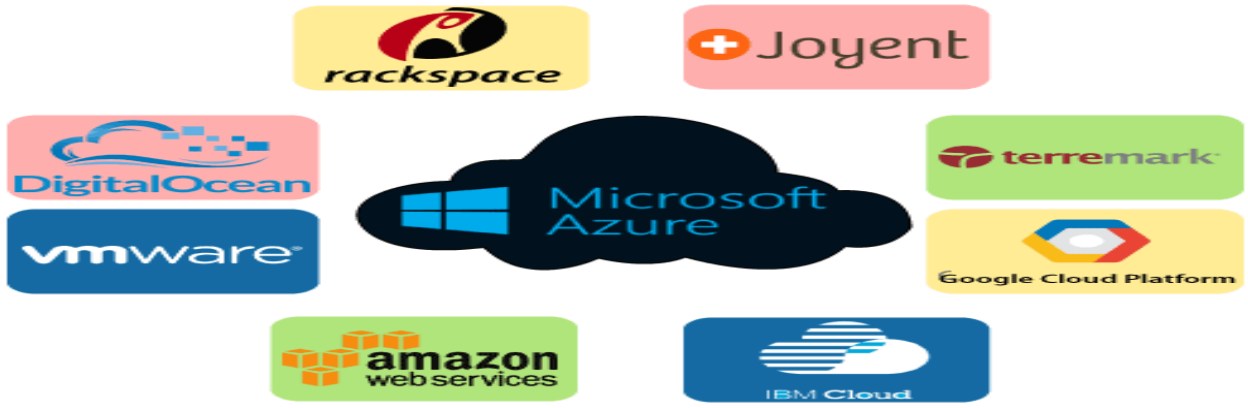


Cloud Computing provides an alternative to the on-premises data center. With an on-premises data center, we have to manage everything, such as purchasing and installing hardware, virtualization, installing the operating system, and any other required applications, setting up the network, configuring the firewall, and setting up storage for data. After doing all the set-up, we become responsible for maintaining it through its entire lifecycle.

But if we choose Cloud Computing, a cloud vendor is responsible for the hardware purchase and maintenance. They also provide a wide variety of software and platform as a service. We can take any required services on rent. The cloud computing services will be charged based on usage.



The cloud environment provides an easily accessible online portal that makes handy for the user to manage the compute, storage, network, and application resources. Some cloud service providers are in the following figure.



Characteristics of cloud computing:

The following is a list of characteristics that define cloud computing.

On-demand self-service:

Cloud computing providers offer APIs that users access to requisition new resources or scale existing resources whenever needed. Teams can automate their infrastructure provisioning in a simple way with infrastructure as code tools from the likes of Terraform and Ansible.

Broad network access

Physical hardware location is a significant concern when delivering the optimal end-user experience. Cloud computing provides a huge boon by offering globally-distributed physical hardware, which allows organizations to strategically provision location-targeted hardware.

Resource pooling

Computing resources in a cloud infrastructure platform are dynamically divided and allocated on demand. Since a cloud host's physical machines are dynamically provisioned and shared between multiple tenants, cloud hardware is thoroughly optimized for maximum usage.

Rapid elasticity

Cloud infrastructures can grow and shrink dynamically, allowing users to request that their computational resources auto-scale with traffic demands. Elasticity can happen on a per-machine basis, where an allocation of resources grows to maximize the available machine resources, or multi-machine basis, in which an application automatically scales to multi-networked machines.

Measured service

Cloud infrastructure providers give detailed usage metrics that are used to communicate usage costs. For example, Amazon Web Services (AWS) provides usage for each service category in hourly or daily line items. Cloud service providers generally take a utility style pay-as-you-go billing model that is measured and delivered, so customers are charged for the exact amount of computing resources used

Advantages of cloud computing:

- **Cost:** It reduces the huge capital costs of buying hardware and software.
- **Speed:** Resources can be accessed in minutes, typically within a few clicks.
- **Scalability:** We can increase or decrease the requirement of resources according to the business requirements.
- **Productivity:** While using cloud computing, we put less operational effort. We do not need to apply patching, as well as no need to maintain hardware and software. So, in this way, the IT team can be more productive and focus on achieving business goals.
- **Reliability:** Backup and recovery of data are less expensive and very fast for business continuity.
- **Security:** Many cloud vendors offer a broad set of policies, technologies, and controls that strengthen our data security.

Cloud models:

Generally, cloud models are divided into two categories

1. Service model
2. Deployment model

Service model: cloud computing services are offered to users in different forms. NIST defines at least three cloud services models as follows:

1. Software as a service (SaaS)

2. Platform as a service (PaaS)

3. Infrastructure as a service (IaaS)

Software as a service (SaaS):

Software-as-a-Service (SaaS) is a way of delivering services and applications over the Internet. Instead of installing and maintaining software, we simply access it via the Internet, freeing ourselves from the complex software and hardware management. It removes the need to install and run applications on our own computers or in the data centers eliminating the expenses of hardware as well as software maintenance.

SaaS provides a complete software solution that you purchase on a pay-as-you-go basis from a cloud service provider. Most SaaS applications can be run directly from a web browser without any downloads or installations required. The SaaS applications are sometimes called Web-based software, on-demand software, or hosted software.

Advantages of SaaS:

Cost-Effective: Pay only for what you use.

Reduced time: Users can run most SaaS apps directly from their web browser without needing to download and install any software. This reduces the time spent in installation and configuration and can reduce the issues that can get in the way of the software deployment.

Accessibility: We can Access app data from anywhere.

Automatic updates: Rather than purchasing new software, customers rely on a SaaS provider to automatically perform the updates.

Scalability: It allows the users to access the services and features on-demand. The various companies providing Software as a service are Cloud9 Analytics, Salesforce.com, Cloud Switch, Microsoft Office 365, Big Commerce, Eloqua, drop Box, and Cloud Tran.

Disadvantages of SaaS:

Limited customization: SaaS solutions are typically not as customizable as on-premises software, meaning that users may have to work within the constraints of the SaaS provider's platform and may not be able to tailor the software to their specific needs.

Dependence on internet connectivity: SaaS solutions are typically cloud-based, which means that they require a stable internet connection to function properly. This can be problematic for users in areas with poor connectivity or for those who need to access the software in offline environments.

Security concerns: SaaS providers are responsible for maintaining the security of the data stored on their servers, but there is still a risk of data breaches or other security incidents.

Limited control over data: SaaS providers may have access to a user's data, which can be a concern for organizations that need to maintain strict control over their data for regulatory or other reasons

Platform as a service (PaaS):

PaaS is a category of cloud computing that provides a platform and environment to allow developers to build applications and services over the internet. PaaS services are hosted in the cloud and accessed by users simply via their web browser.

A PaaS provider hosts the hardware and software on its own infrastructure. As a result, PaaS frees users from having to install in-house hardware and software to develop or run a new application. Thus, the development and deployment of the application take place independent of the hardware.

The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, or storage, but has control over the deployed applications and possibly configuration settings for the application-hosting environment. To make it simple, take the example of an annual day function, you will have two options either to create a venue or to rent a venue but the function is the same.

Advantages of PaaS:

Simple and convenient for users: It provides much of the infrastructure and other IT services, which users can access anywhere via a web browser.

Cost-Effective: It charges for the services provided on a per-use basis thus eliminating the expenses one may have for on-premises hardware and software.

Efficiently managing the lifecycle: It is designed to support the complete web application lifecycle: building, testing, deploying, managing, and updating.

Efficiency: It allows for higher-level programming with reduced complexity thus, the overall development of the application can be more effective.

The various companies providing Platform as a service are Amazon Web services Elastic Beanstalk, Salesforce, Windows Azure, Google App Engine, cloud Bees and IBM smart cloud.

Disadvantages of PaaS:

Limited control over infrastructure: PaaS providers typically manage the underlying infrastructure and take care of maintenance and updates, but this can also mean that users have less control over the environment and may not be able to make certain customizations.

Dependence on the provider: Users are dependent on the PaaS provider for the availability, scalability, and reliability of the platform, which can be a risk if the provider experiences outages or other issues.

Limited flexibility: PaaS solutions may not be able to accommodate certain types of workloads or applications, which can limit the value of the solution for certain organizations

Infrastructure as a service (IaaS)

Infrastructure as a service (IaaS) is a service model that delivers computer infrastructure on an outsourced basis to support various operations. Typically IaaS is a service where infrastructure is provided as outsourcing to enterprises such as networking equipment, devices, database, and web servers.

It is also known as **Hardware as a Service (HaaS).** IaaS customers pay on a per-user basis, typically by the hour, week, or month. Some providers also charge customers based on the amount of virtual machine space they use.

It simply provides the underlying operating systems, security, networking, and servers for developing such applications, and services, and deploying development tools, databases, etc.

Advantages of IaaS:

Cost-Effective: Eliminates capital expense and reduces ongoing cost and IaaS customers pay on a per-user basis, typically by the hour, week, or month.

Website hosting: Running websites using IaaS can be less expensive than traditional web hosting.

Security: The IaaS Cloud Provider may provide better security than your existing software.

Maintenance: There is no need to manage the underlying data center or the introduction of new releases of the development or underlying software. This is all handled by the IaaS Cloud Provider.

The various companies providing Infrastructure as a service are Amazon web services, Bluestack, IBM, Open stack, Rackspace, and VMware.

Disadvantages of laaS :

Limited control over infrastructure: IaaS providers typically manage the underlying infrastructure and take care of maintenance and updates, but this can also mean that users have less control over the environment and may not be able to make certain customizations.

Security concerns: Users are responsible for securing their own data and applications, which can be a significant undertaking.

Limited access: Cloud computing may not be accessible in certain regions and countries due to legal policies.

Deployment model:

- **Public Cloud:** The cloud resources that are owned and operated by a third-party cloud service provider are termed as public clouds. It delivers computing resources such as servers, software, and storage over the internet
- **Private Cloud:** The cloud computing resources that are exclusively used inside a single business or organization are termed as a private cloud. A private cloud may physically be located on the company's on-site data center or hosted by a third-party service provider.
- **Community Cloud**: Community Cloud is a cloud where services and infrastructure are accessible by a group of organizations.
- **Hybrid Cloud:** It is the combination of public and private clouds, which is bounded together by technology that allows data applications to be shared between them. Hybrid cloud provides flexibility and more deployment options to the business.

Cloud services with examples

IaaS-Amazon EC2, Google Compute Engine, Azure VMs Amazon EC2
Amazon Elastic Compute Cloud is an Infrastructure as a Service offering from Amazon EC2 is a web service that provides a computing capacity in the form of virtual machines.

• Amazon EC2 allows users to launch instances on demand using a simple web-based interface.

• Amazon provides pre-configured Amazon Machine Images (AMIs) which are templates of cloud instances.

• Users can also create their own AMIs with custom applications, libraries and data

Amazon EC2 also provides instances with high memory, high CPU resources, Cluster Compute instances, Cluster Graphical Processor unit (GPU)instances and high Input/Output instances.

• Instances can be launched with a variety of operating systems.

• Users can load their applications on running instances and rapidly and easily increase or decrease capacity to meet dynamic application performance requirements.

• With EC2, users can even provision hundreds or thousands of server instances simultaneously, manage network access permissions and monitor usage resources through web interface. (Create 2 VMs at 3 different places and running applications, creating network among 3 and allowing data transfer among 3 VMs and monitoring resource usage).

Amazon EC2 provides instances of various computing capacities ranging from Small Instances (Eg: 1 Virtual core with 1 EC2 compute unit, 1.7 GB memory and 160 GB instance storage) to Extra Large Instances (Eg: 4 Virtual cores with 2 EC@ compute unit each with 15GB memory and 1690 GB instance storage).

• The pricing model for EC2 instances is based on Pay-Per Use model. Users are billed based on the number on instance hours used for on demand instances.

• EC2 also provides spot instances that allow users to bid on unused Amazon EC2 capacity and run those instances for as long as their bid exceeds the current spot price.

Google Compute Engine:

Google Compute Engine (GCE) is an IaaS offering from Google.

• GCE provides virtual machines of various computing capacities ranging from small instances (Eg: Virtual core with 1.38 GCE unit and 1.7 GB memory) to high memory machine types (8 virtual cores with 22 GCE unit and 52 GB memory.

Windows Azure:

• Windows Azure Virtual Machine is an IaaS offering from Microsoft. • Azure VMs provides virtual machines of various computing capacities ranging from small instances (1 virtual core with 1.75GB memory) to memory intensive machine types (8 virtual cores with 56GB memory).

PaaS-Google App Engine:

Google App Engine (GAE) is a Platform as a Service offering from Google.

• GAE is a cloud-based web service for hosting web applications and storing data.

• GAE allows users to build scalable and reliable applications that run on the same systems that power Google's own applications.

• GAE provides a Software Development Kit (SDK) for developing web applications software that can be deployed on GAE.

Developers can develop and test their applications with GAE SDK on a local machine and then upload it to GAE with a simple click of a button

• Applications hosted in GAE are easy to build, maintain and scale. Users don't need to worry about launching additional computing instances when the applications load increases.

• GAE provides automatic scaling and load balancing capability.

• GAE supports applications written in several programming language.

• With Java runtime environment developers can build applications using Java programming language and standard Java technologies such as Java Servlets. GAE also provides runtime environment for Python programming languages

Applications hosted in GAE run in secure sandbox with limited access to the underlying operating system and hardware.

• The pricing model for GAE is based on the amount of computing resources used.

• GAE provides free computing resources for applications up to a certain limit. Beyond that limit, users are billed based on the amount of computing resources used such as amount of bandwidth consumed, number of resource instance hours, amount of data stored

SaaS-Salesforce

Salesforce Service Cloud:

• Salesforce Sales Cloud is a cloud-based Customer Relationship Management (CRM) SaaS offering.

• Users can access CRM application from anywhere through internet enabled devices such as workstations, laptops, tablets and smartphones.

• Sales Cloud allows sales representatives to manage customer profiles, track opportunities, optimize campaigns from lead to close and monitor the impact of campaigns. (A lead can be a company or an individual who has expressed interest in a company's product and/or service)

Salesforce and Marketing Cloud

Salesforce Service Cloud is a cloud-based Customer Service Management SaaS.

• Service cloud provides companies a call center like view and allows creating, tracking, routing and escalating cases.

• Service cloud includes a social networking plug-in that enables social customer service where comments from social media channels can be used to answer customer questions.

• Marketing Cloud is cloud based social marketing SaaS.

• Marketing cloud allows companies to identify sales leads from social media, discover advocates, identify most trending information on any topic.

• Marketing cloud allows companies to pro-actively engage with customers, manage social advertisement campaigns and track the performance of social campaigns.

Some of the tools included in the Salesforce Sales, Service and Marketing Clouds include

➢ Accounts and Contacts

➢ Leads

➢ Opportunities

➢ Campaigns

➢ Chatter

➢ Analytics an Forecasts

Cloud Based Services and Applications

Some examples on cloud-based services and applications are**:**

Cloud Computing for Healthcare

- Hospitals and their affiliated providers can securely access patient data stored in the cloud and share the data with the other hospitalsand physicians.
- Patients can access their own health information from all of their care
- providers and store it in a personal health record (PHR)
- The PHR can be a vehicle for e-prescribing, a technique known to
- reduce medication dispensing errors and to facilitate medication reconciliation.
- History and information stored in the cloud can be streamline the admissions, care and discharge process byeliminating redundant data collection and entry.
- Health payers can increase the effectiveness and lower the cost of their care management programs by providingvalue added services and giving access to health information to members.

Cloud Computing for Energy Systems:

- Energy systems such as Smart Grids, Power Plants, Wind Turbine etc. have thousands of sensors that gather real timemaintenance data continuously for condition monitoring and failure prediction purposes.
- The energy systems have a large number of critical components that must function correctly so that the systems can perform their operations correctly.
- For eg: a wind turbine has number of critical components like bearings, turning gears etc. that must be monitored carefully as wear and rear in such critical components or sudden change in operating conditions of the machines can result in failures. In systems such as power grids, real time information is collected using specialized electrical sensors called Phasor Measurement Units (PMU) at the sub-stations. The information received from PMUs must be monitored in real time for estimating the state of the system and for predicting failures.
- There is a generic framework "Cloud View" for storage, processing and analysis of massive machine maintenance data collected from a large number of sensors embedded in industrial machines in a cloud computing environment.
- The below figure shows a generic use case of cloud for energy systems.

Cloud Computing concepts & Technologies:

- Virtualization
- Load Balancing
- Scalability and Elasticity
- Deployment
- Replication
- Monitoring
- Software Defined Networking
- MapReduce
- Identity and Access Management
- Service Level Agreements
- Billing

Virtualization:

• Virtualization refers to the partitioning the resources of a physical system (such as computing, Storage, Network and Memory) into multiple virtual resources.

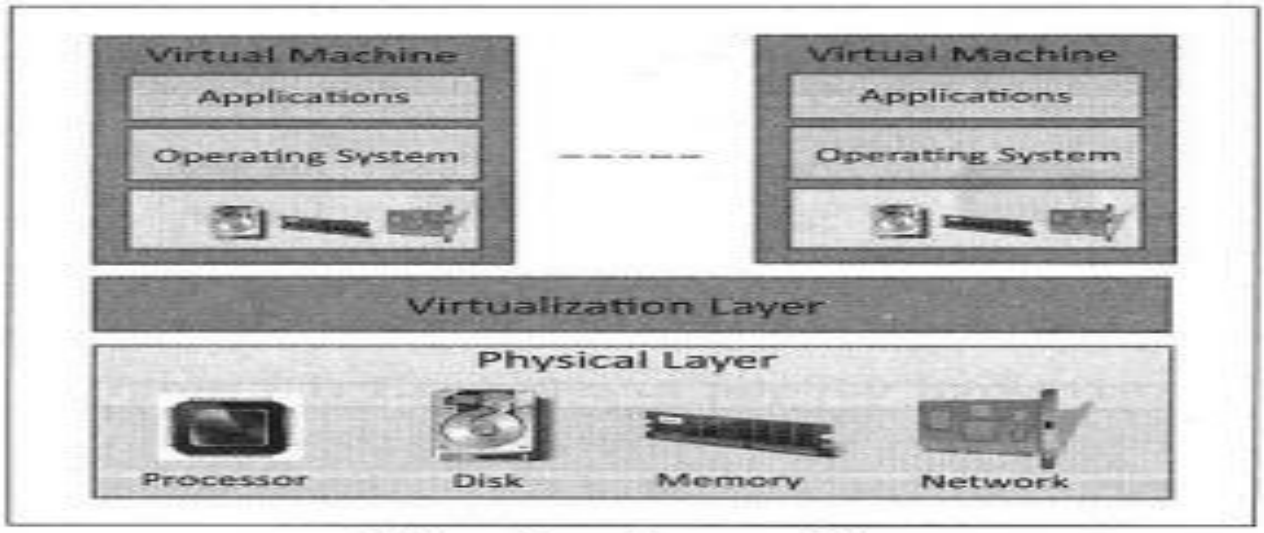
• In cloud computing, resources are pooled to serve multiple users using

Multi-Tenancy aspects of the cloud allow multiple users to be served by the same physical hardware.

• The below figure shows the architecture of a virtualization technology in cloud computing.

The physical resources such as computing, storage, memory and network resources are virtualized.

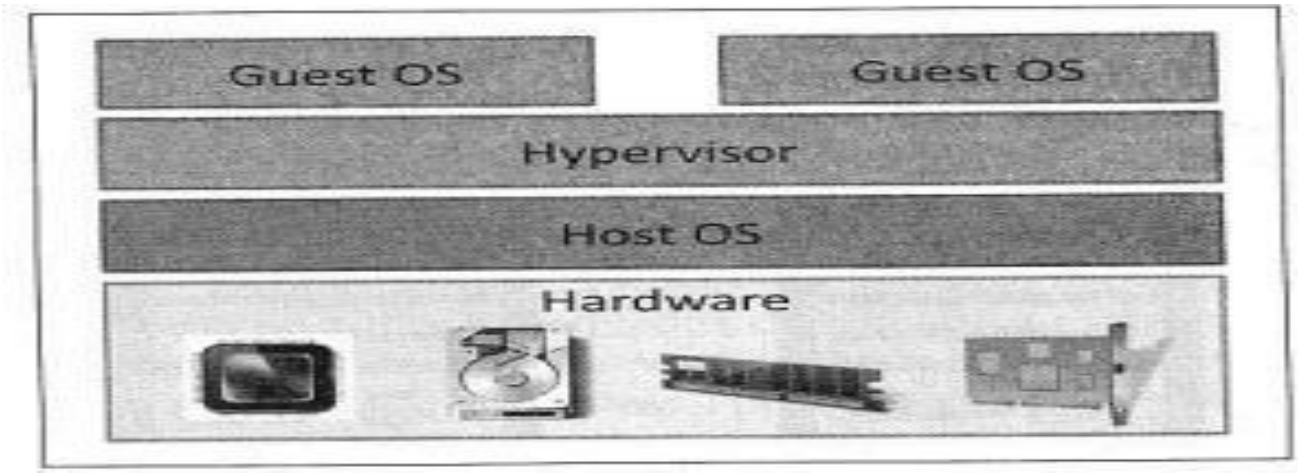
• The virtualization layer partitions the physical resources into multiple virtual machines.



Virtualization architecture

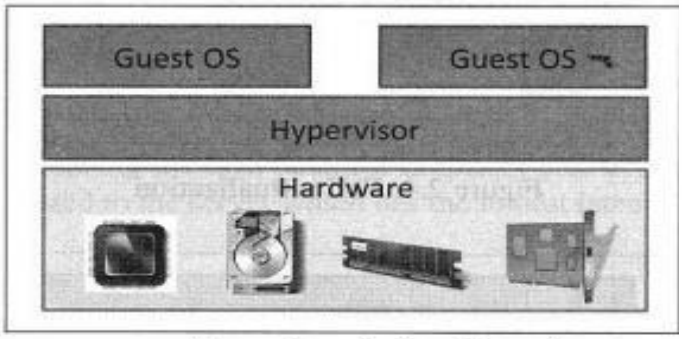
Virtualization: Guest Operating System:

A guest OS is an operating system that is installed in a virtual machine in addition to the host OS

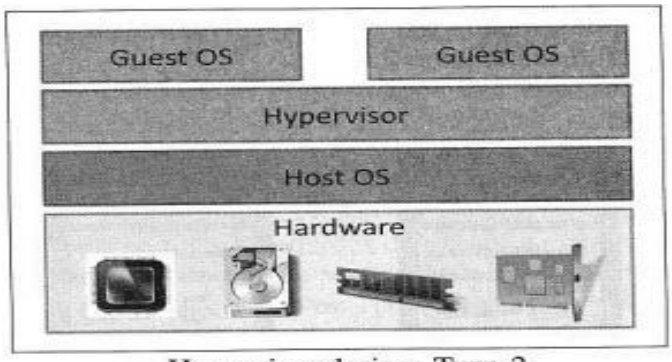


Virtualization: Hypervisor

- The virtualization layer consists of a hypervisor or a Virtual Machine Monitor (VMM).
- There are two types of hypervisors



Hypervisor design: Type-1



Hypervisor design: Type-2

Type-1 Hypervisors or Native Hypervisors

- Type-1 Hypervisors or Native Hypervisors run directly on the host hardware and control the hardware and monitor the guest operating system.

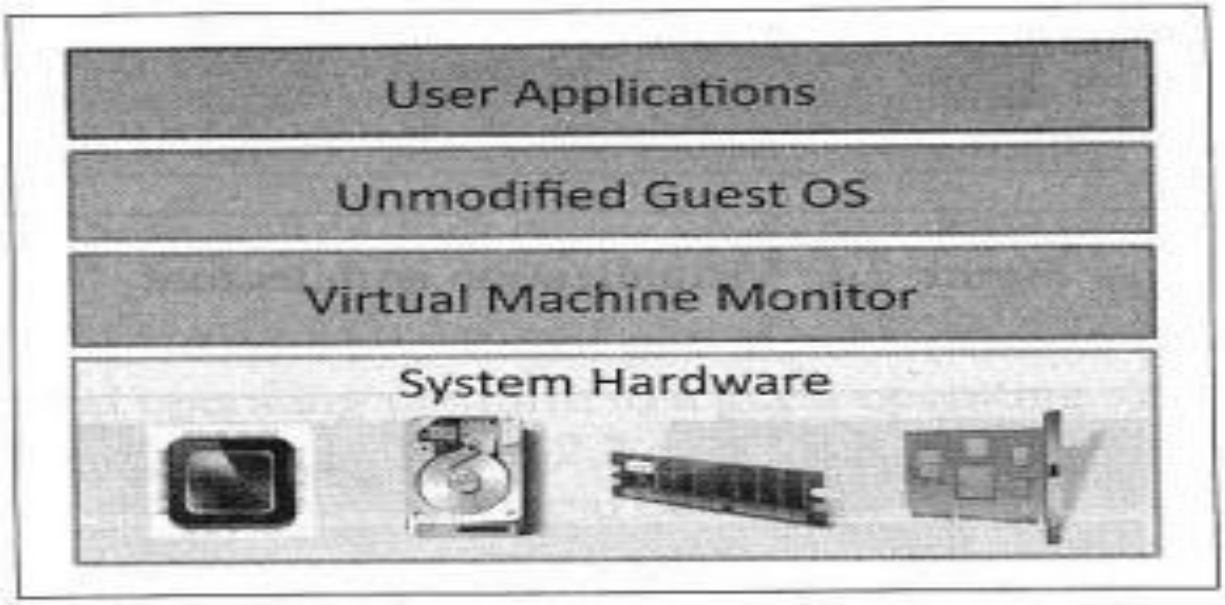
Type-2 Hypervisors or Hosted Hypervisors

- Type 2 Hypervisors or Hosted Hypervisors run on top of a conventional (main or Host) operating system and monitor the guest operation systems.

Virtualization: Guest Operating System

- Various forms of virtualization approaches exist:
  - **Full Virtualization**
  - **Para-Virtualization**
  - **Hardware Virtualization**

Full Virtualization: The guest OS requires no modification and is not aware that it is being virtualized.
Full virtualization is enabled by direct execution of user requests and binary translation of OSrequests.



Full virtualization

- In Para virtualization, the guest OS is modified to enable communication with the hypervisor to improveperformance and efficiency.
- The guest OS kernel is modified to replace non virtualizable instructions with hyper calls that communicatedirectly with the virtualization layer hypervisor.
- The below figure shows the para virtualization approach



Para-virtualization

Hardware Assisted virtualization is enabled by hardware features such as Intel's Virtualization technology (VT-x)and AMD's AMD-V. In hardware virtualization, privileged and sensitive calls are set to automatically trap to the hypervisor.

Thus, there is no needed for either binary translation or Para virtualization. Hardware-assisted full virtualization eliminates the binary translation and it directly interrupts with hardware using the virtualization technology which has been integrated on X86 processors since 2005 (Intel VT-x and AMD-V).
Guest OS's instructions might allow a virtual context execute privileged instructions directly on the processor,even though it is virtualized

Load Balancing Technique: -

- One of the important features of cloud computing is scalability.
- Cloud resources can be scaled up on demand to meet the performance requirements of applications.
- Load balancing distributes workload across multiple servers to meet the application workloads.
- The goal of load balancing techniques are to achieve maximum utilization of resources, minimize theresponse times, maximizing throughput.
- Since multiple resources under a load balancer are used to serve the user requests, in the event of failure of one or more of the resources, the load balancer can automatically reroute the user traffic to the healthy resources.

Round Robin

In round robin load balancing, the servers are selected one by one in a circular fashion to server the incoming requests from the user

Weighted Round Robin

In Weighted round robin load balancing, servers are assigned some weights. The incoming requests are proportionally routed to a server based on its weight

Low Latency

In low latency load balancing, the load balancer monitors thelatency of each server and request is routed to a server whichhas lowest latency

Least Connections

In least connection load balancing, the incoming requests are

routed to the server with least number of connections

Priority

In priority load balancing, each server is assigned a priority. The incoming traffic is routed to the highest priority servers as long as the server is available. When the highest priority server fails, the incoming traffic is routed to a server with a lower priority.

Overflow

Overflow load balancing is similar to priority load balancing. When the incoming request to high priority servers overflow, the requests are routed to a lower priority server.

A session is defined as a series of related browser requests that come from the same client during a certain time period. Session tracking ties together a series of browser requests—think of these requests as pages—that may have some meaning as a whole, such as a shopping cart application.

• A web session is a series of contiguous actions by a visitor on an individual website within a given time frame. This could include your search engine searches, filling out a form to receive content, scrolling on a website page, adding items to a shopping cart, researching airfare, or which pages you viewed on a single website. Any interaction that you have with a single website is recorded as a web session to that website property.

• To track sessions, a web session ID is stored in a visitor's browser. This session ID is passed along with any HTTP requests that the visitor makes while on the site (e.g., clicking a link).

• ("Session" is the term used to refer to a visitor's time browsing a web site. It's meant to represent the time between a visitor's first arrival at a page on the site and the time they stop using the site.

• A cookie is a small piece of data from a web site that is stored on a visitor's browser to help the website track the visitor's activity on the web site.)

• For the session-based applications, an important issue to handle during load balancing is the persistence of

multiple requests from a particular user session. (Because you may go from current page to next page and back to previous page)

• Since load balancing can route successive requests from a user session to different servers, maintain the state or the information of the session is important.

Four persistence approaches are:

Sticky Sessions

Session Database

Browser Cookies

URL Re-Writing

Sticky Sessions:

In this approach, all the requests belonging to a user session are routed to the same server. Theses sessions are called Sticky Sessions.

• The benefit of this approach is that it makes session management simple.

• However, a drawback of this approach is that if a server fails all the sessions belonging to that server are lost, since there is no automatic failover possible.

Session Database:

• In this approach, the session information is stored externally in a separate session database, which is oftenreplicated to avoid a single point of failure.
• Though, this approach involves additional overhead of storing the session information, however unlike the sticky session approach, this approach allows automatic failover.

Browser Cookies

• In this approach, the session information is stored on the client side in the form of browser cookies.

• The benefit of this approach is that it makes the session management easy and has the least amount ofoverhead for the load balancer.

URL Re-Writing

• In this approach, a URL re-write engine stores the session information by modifying the URL's on the clientside.

• Though this approach avoids overhead on the load balancer, a drawback is that the amount of session information that can be stored is limited. (Modifying each URLà Shortening URLà storing in engineà degrades performance)

• For applications that require larger amounts of session information, this approach does not work.

(Changing a URL to the required format. URL rewriting allows URLs to be more easily remembered by the user. When the URL is entered into the Web server, the URL rewrite engine modifies the syntax behind the scenes to enable the appropriate Web page or database item to be retrieved. For example, to look up the definition for "path," a user friendly URL might look like computerlanguage.com/path. The rewrite engine could turn it into the following syntax: computerlanguage.com/results .php? definition =path. So, the URL rewrite function simply puts a layer on top of the original address and turns it into something easy to find and that makes sense. Thus, turning https://wiredelta.com/?page_id=16825 into wiredelta.com/url-rewrite, for example. From a user perspective, when a URL rewrite occurs the URL of the website remains the same in the browser and they are none the wiser. But behind the scenes, the browser rewrites the URL back into that complicated mess and sends a query to the servers.)

• Load balancing can be implemented in software or hardware.

• Software based load balancers run on standard operating systems and like other cloud resources.

• Hardware load balancers implement load balancing algorithms in Application Specific Integrated Circuits (ASICs).

Scalability and Elasticity

- Muti-tier applications such as e-Commerce, Social networking, business-to-business etc. can experience rapid changes in their traffic.
- Each website has a different traffic pattern which is determined by a number of factors that are generally hardto predict beforehand.
- Capacity planning involves determining the right sizing of each tier of the deployment of an application interms of number of resources and capacity of each resource.
- Capacity planning may be for computing, storage, memory or network resources.
- Traditional approaches for capacity planning are based on predicted demands for applications and account forworst case peak load as of applications.
- When the workloads of applications increase, the traditional approaches have been either to scale up or scale out.
- Scaling up involves upgrading the hardware resources (adding computing, memory,

storage or networkresources. Scaling out involves addition of more resources of the same type.

- Traditional scaling up and scaling out approaches are based on demand forecasts at regular intervals of time.
- When variations in workload are rapid, traditional approaches are unable to keep track with the demand and either overprovisioning or under provisioning of resources.

Over provisioning of resources of resources leads to higher capital expenditure and under provisioning ofresources leads to traffic overloads, slow response time, low throughput

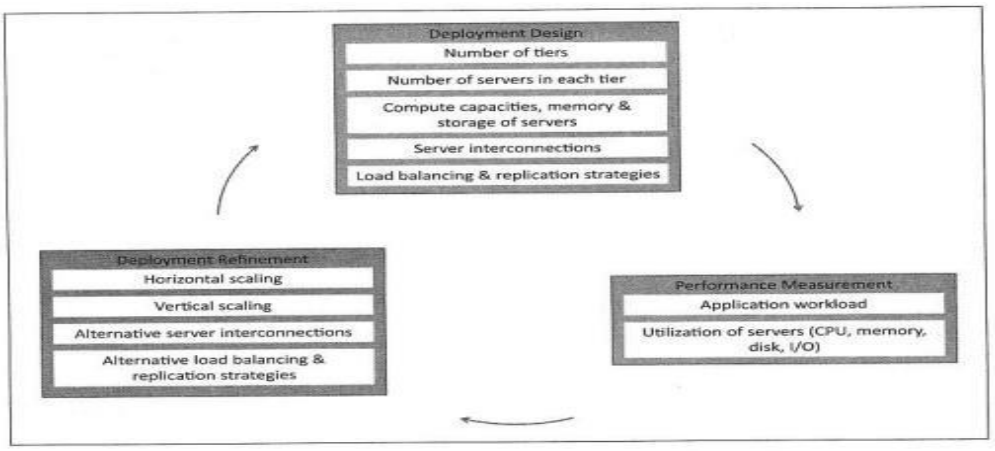
Deployment

- Figure shows the cloud application deployment lifecycle.
- Deployment prototyping can help in making architecture design choices.
- By comparing performance of alternative deployment architectures, deployment prototyping can help in choosing the best and most application performance requirements.
- Deployment design is an iterative process that involves the following steps.

  Deployment design

  Performance evolution

  Deployment refinement



Cloud application deployment lifecycle

Deployment Design

• In this step, the application deployment is created with various tiers as specified in the deployment configuration.

• The variables in this step include the number of servers in each tier, computing, memory and storage capacities of servers, server interconnection, load balancing and replication strategies.

• Deployment is created by provisioning the cloud resources as specified in the deployment configuration.

Performance Evaluation

• Once the application is deployed in the cloud, the next step in the deployment lifecycle is to verify whether the application meets the performance requirements with the deployment.

• This step involves monitoring the workload on the application and measuring various workload parameters such as response time and throughput. In addition, the utilization of servers (CPU, memory, disk I/O etc.) is also monitored.

Deployment Refinement

• After evaluating the performance of the application, deployments are refined so that the application can meet the performance requirements.

• Various alternatives can exits for deployment refinement such as vertical scaling, horizontal scaling, alternative server interconnections, alternative load balancing and replication strategies

The below table lists some popular cloud deployment management tools.

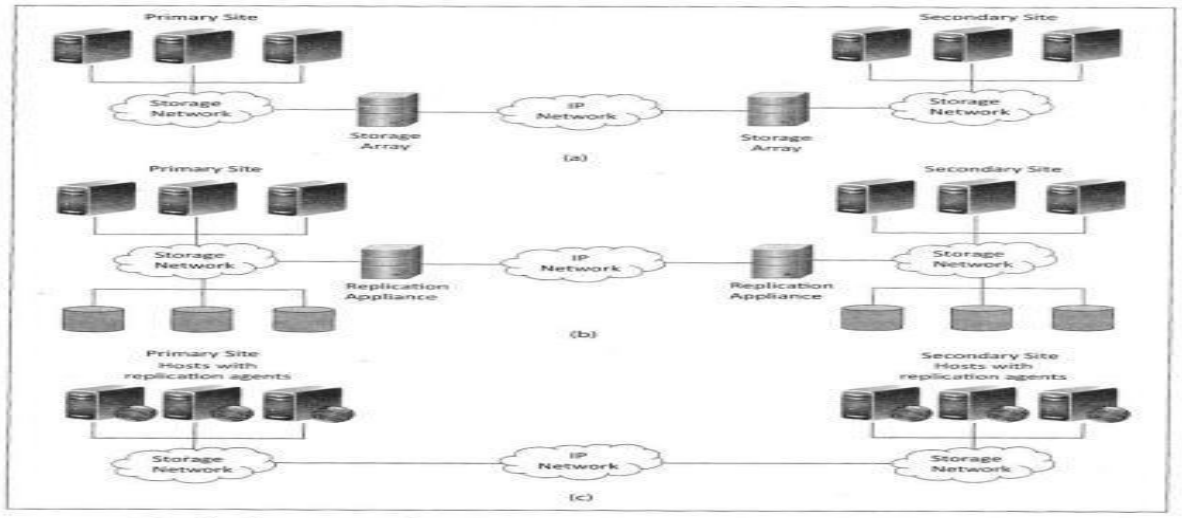
Cloud Deployment Management Tool	Features
RightScale	Design, deploy and manage cloud deployments across multiple public or private clouds.
Scalr	Provides tools to automate the management of servers, monitors servers, replaces servers that fail, provides auto scaling and backups.
Kaavo	Allows deploying applications easily across multiple clouds, managing distributed applications and automating high availability.
CloudStack	Allows simple and cost effective deployment management and configuration of cloud computing environments.

Examples of popular cloud deployment management tools

Replication:

- Replication is used to create and maintain multiple copies of the data in the cloud.
- Replication of data is important for practical reasons such as business continuity and disaster recovery.
- In the event of data loss at the primary location, organizations can continue to operate their applications fromsecondary data sources.
- Traditional business time objective (RTO).
- continuity and disaster recovery approaches don't provide efficient, cost effective and automated recovery ofdata
- Cloud based data replication approaches provide replication of data in multiple locations, automated recovery,low recovery point objective (RPO)and low recovery time.
- With cloud-based data replication, organizations can plan for disaster recovery without making any capital expenditures on purchasing, configuring and managing secondary site locations.
- There are three types of replication approaches as shown in below figure

1. Array Based Replication
2. Network Based Replication
3. Host Based Replication



Replication approaches: (a) Array-based replication, (b) Network-based replication, (c) Host-based replication

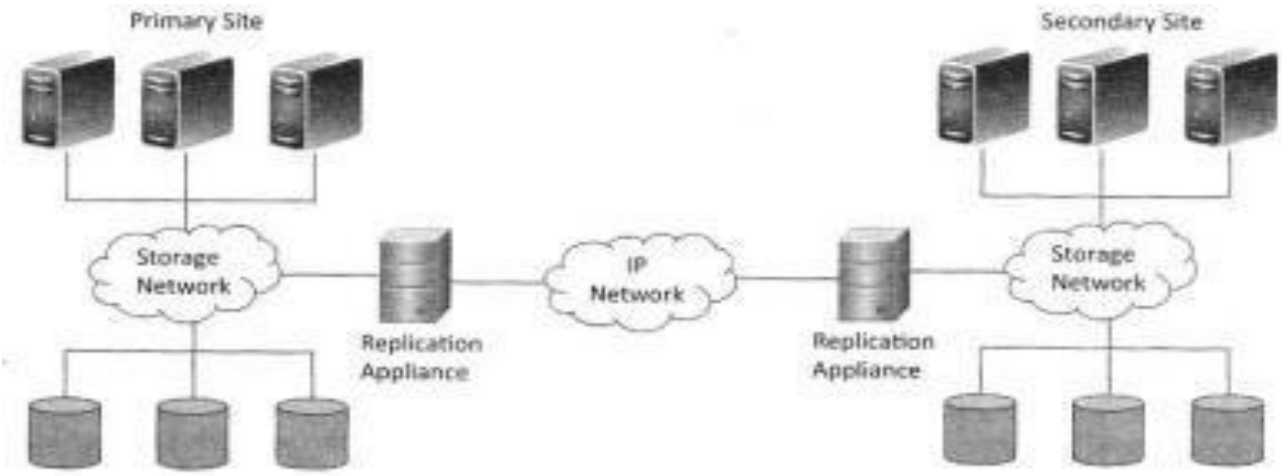
Array Based Replication

Array based replication uses compatible storage arrays to automatically copy data from a local storage array to a remote storage array.

- Arrays replicate data at the disk sub system level (in terms of blocks), therefore the type of hosts accessing the data and the type of data is not important. Thus, array based replications can work in heterogeneous environments with different operating systems.
- Array based replication uses Network Attached Storage (NAS) or Storage Area Network (SAN) to replicate.
- A drawback of this array-based replication is that it requires similar arrays at local and remote locations. (Storage methods, access methods, block size, block number, arrays etc must be of same). Thus the costs for setting up array based replications are higher than the other approaches.
- (Array-based replication is an approach to data backup in which compatible storage arrays use built-in software to automatically copy data from one storage array to another.)

Network based replication:

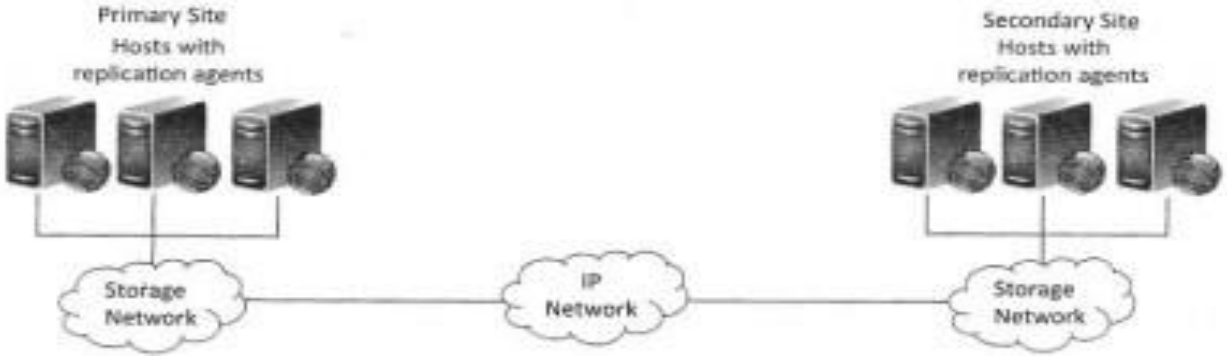
Network based replication uses an applicable or device that sits on the network and intercepts packets that are sent from hosts and storage arrays. the intercepted packets are replicated to a secondary location.



- By offloading replication from server and storage systems, network-based replication can work across a large number of server platforms and storage systems, making it ideal for highly heterogeneous environments and requires a single point of management.
- This approach involves higher initial costs due to replication hardware and software.

Host Based Replication

- Host Based Replication runs on standard servers and uses software to transfer data from a local to remote location.



- An agent is installed on the hosts that communicates with the agents on the other hosts.
- Host based replication can either be block based or file based. Block based replication typically require
  dedicated volumes of the same size on both the local and remote servers.
- With host based replication, entire virtual machines can be replicated in real time.

Monitoring

- Cloud providers provides monitoring service that allows cloud users can monitor their cloud resource usage.
- A monitoring service at the cloud collects data on various system and application metrics from cloud computing instances
- Users can define various actions based on the monitoring data. For eg: Auto scaling when the CPU usage of themonitored resources becomes high.
- Monitoring services also provides various statistics based on the monitoring data collected as shown in below table.

Type	Metrics
CPU	CPU-Usage, CPU-Idle
Disk	Disk-Usage, Bytes/sec (read/write), Operations/sec
Memory	Memory-Used, Memory-Free, Page-Cache
Interface	Packets/sec (incoming/outgoing), Octets/sec(incoming/outgoing)

Typical monitoring metrics

- Monitoring of cloud resources is important because it allows the users to keep track of the health of applications and services deployed in the cloud. For eg: an organization which has its website hosted in the cloud can monitor the performance of the website and also the website traffic with which users can make operational decisions such as scaling up or scaling down cloud resources.

Software Defined Networking

- SDN decouples the network and control forwarding functions.
- SDN stands for Software Defined Network which is networkingarchitecture approach. It enables the control and managementof network using software applications.
- Through Software Defined Network (SDN) networking behavior of entire network and its devices are programmed in centrally controlledmanner through software applications using open APIs.

Traditional Network

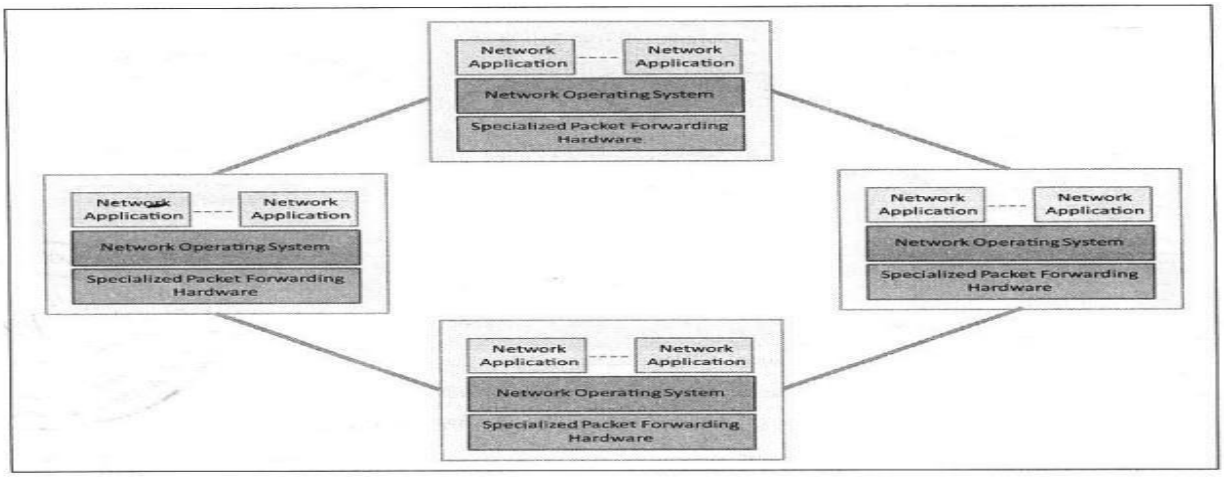
Traditional network refers to the old conventional way of networkingwhich uses fixed and dedicated hardware devices such as routers andswitches to control network traffic.

Inability to scale and network security and performance are the majorconcern now a days in the current growing business situation so that SDN is taking control to traditional network.

Traditional network is static and based on hardware network appliances

- SDN decouples the network and control forwarding functions. It separates the control plane (making traffic decision) from the data plane (packet forwarding).

- The below figure shows the conventional network architecture built with specialized hardware (Switches, routers).



Conventional network architecture

In the conventional network architecture, the control plane and data plane are coupled. Control plane is the part of the network that carries the signaling and routing message traffic while the data plane is the part of thenetwork that carries the payload data traffic.

- Conventional network architecture has following limitations

Complex Network Devices

Conventional networks are getting increasingly complex with more and more protocols being implemented to improve link speed and reliability. The conventional networks are well suited for static traffic patterns. Due to the complexity of conventional network devices, making changes in the networksto meet the dynamic traffic patterns has increasingly difficult.

Management Overhead

Network managers find it increasingly difficult to manage multiple network devices. Upgradation of network requires configurations changes in multiple devices (switches, routers, firewalls etc.)

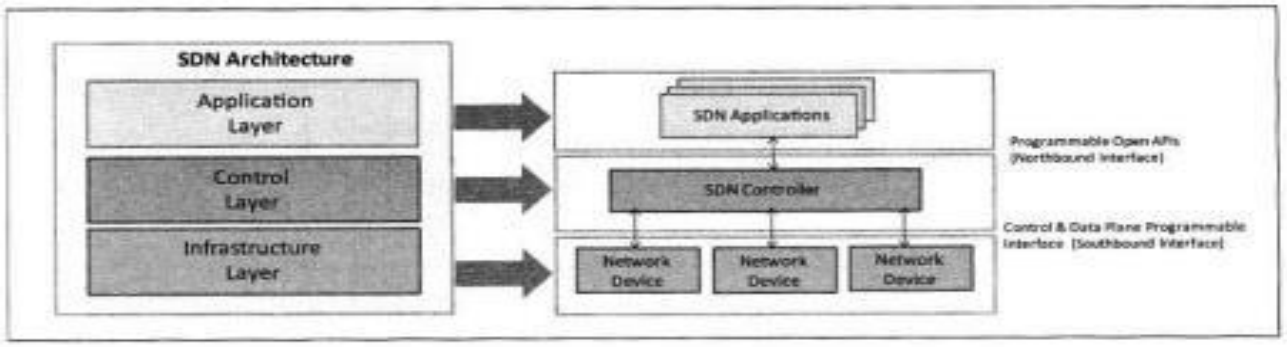
Limited scalability

Big data applications run distributed algorithms on a large number of virtual machines. Such computing environments require highly scalable with minimum manual configurations which is difficult with conventional networks. (Running each module of the applications at different places with multiple copies of data at different locations, need to scaling of the network)

- SDN attempts to create network architecture that are simpler, inexpensive, scalable, agile and easy to manage.
- In SDN architecture, the control plane and data planes are decoupled and network controller is centralized.
- The underlying infrastructure in SDN uses simple packet forwarding hardware as opposed to specializedhardware in conventional networks.
- Network devices become simple with SDN as they do not require implementation of

large number ofprotocols.

- Network devices receive instructions from the SDN controller on how to forward the packets.



SDN layers

Map reduce:

- Map Reduce is a parallel data processing technique for processing and analyzing large scale data. (data intensive problem). (The framework takes care of scheduling tasks, monitoring them and re-executing any failed tasks.)

- The Map Reduce model includes two important phases: Map and Reduce functions.

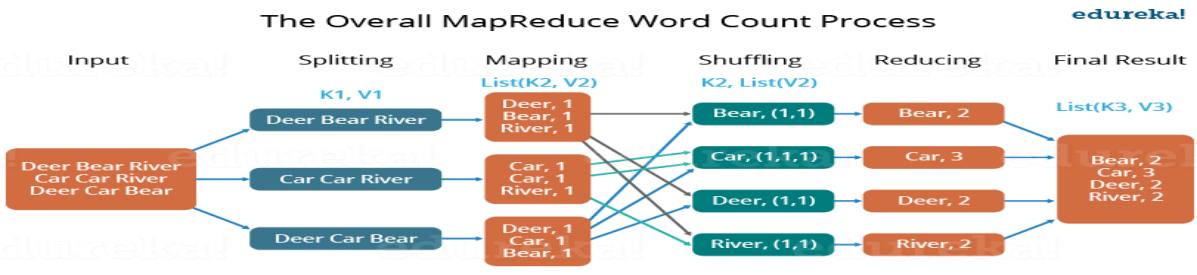
- Map function takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs). Secondly, reduce function, which takes the output from a map as aninput and combines those data tuples into a smaller set of tuples.

MAPREDUCE PROGRAMMING MODEL:

Example taken from net foryour reference A Word Count Example of MapReduce

Dear, Bear, River, Car, Car, River, Deer, Car and Bear

Now, suppose, we have to perform a word count on the example.txt using MapReduce to find the unique words and the number of occurrences of thoseunique words.



- First, we divide the input in three splits as shown in the figure. This will distribute the work among all the map nodes. (Divides into file into 3 subfiles and each file given to each Map node.)
- Then, Map function within each Map node tokenize the words in the file and give a hardcoded value (1) to each of the tokens or words.
- Now, a list of key-value pair will be created where the key is nothing but the individual words and value is one. So, for the first line (Dear BearRiver) we have 3 key-value pairs – (Dear, 1); (Bear, 1); (River, 1). The mapping process remains the same on all the nodes.
- After mapper phase, a partition process takes place where sorting and shuffling happens so that all the tuples with the same key are sent to the corresponding reducer.
- So, after the sorting and shuffling phase, each reducer will have a unique key and a list of values corresponding to that key. For example, Bear,[1,1]; Car, [1,1,1].., etc. (Input to Reduce Node)
- Now, each Reducer counts the values which are present in that list of values. As shown in the figure, reducer gets a list of values which is [1,1] for the key Bear. Then, it counts the number of ones in the list and gives the final output as – Bear, 2.
- Finally, all the output key/value pairs are then collected and written in the output file.
- Among the cluster of nodes, one acts like a "master" and the rest are "workers."
- The master is responsible for scheduling (assigns the map and reduce tasks to the worker) and monitoring (monitors the task progress and the worker health). (Master instructs which nodes should act like a mapper and which nodes should act like a reducer. Also responsible for monitoring health condition of each worker node)
- When map tasks arise, the master assigns the task to an idle worker, taking into account the data locality.
- A worker reads the content of the corresponding input split and emits a key/value pairs. (On what operation performing and corresponding analyzed value)
- The intermediate key/value pairs produced by the Map function are first buffered in memory and then periodically written to a local disk, sorts the intermediate keys so that all occurrences of the same key are grouped together into R sets by the partitioning function.

The master passes the location of these stored pairs to the **reduce worker**, which **reads** the buffered data using remote procedure calls (**RPC**)

- For each key, the worker passes the corresponding intermediate value for its entire occurrence to the **Reduce function**. (worker node givesintermediate set to reduce function, which generates output file)
- Finally, the output is available in **R output files** (one per reduce task).

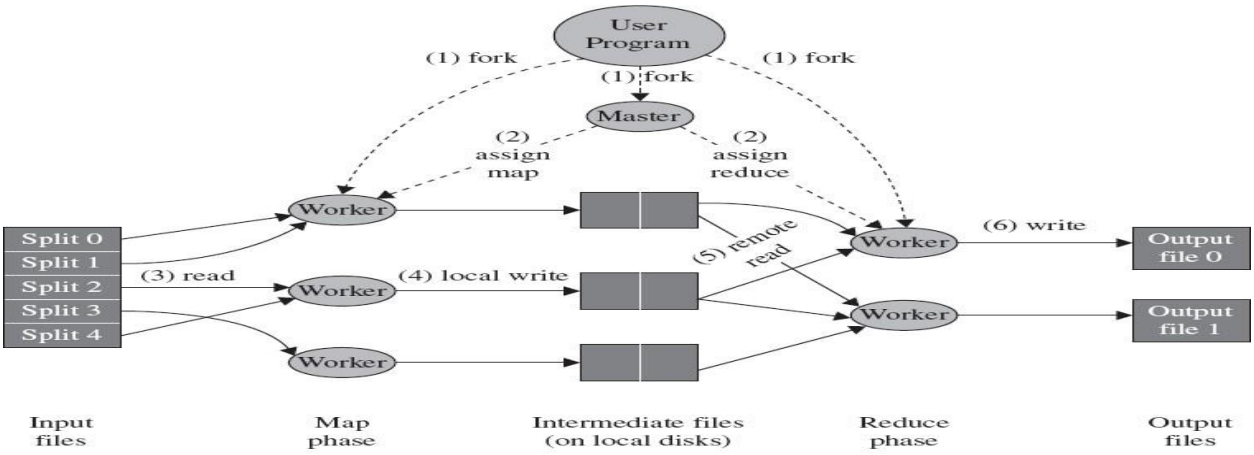


FIGURE 14.3. MapReduce execution overview [4].

- The below figure shows the workflow of MapReduce.
- In the Map phase, data is read from a distributed file system, partitioned among a set of computing nodes.
- The Map tasks process the input records independently of each other and produce intermediate results asKey-Value pairs.
- When all the Map tasks are completed, the Reduce phase begins in which the intermediate data with the same key is aggregated.
- An optional combine task can be used to perform data aggregation on the intermediate data of the same key for the output from the Mapper before transferring to the Reduce task.

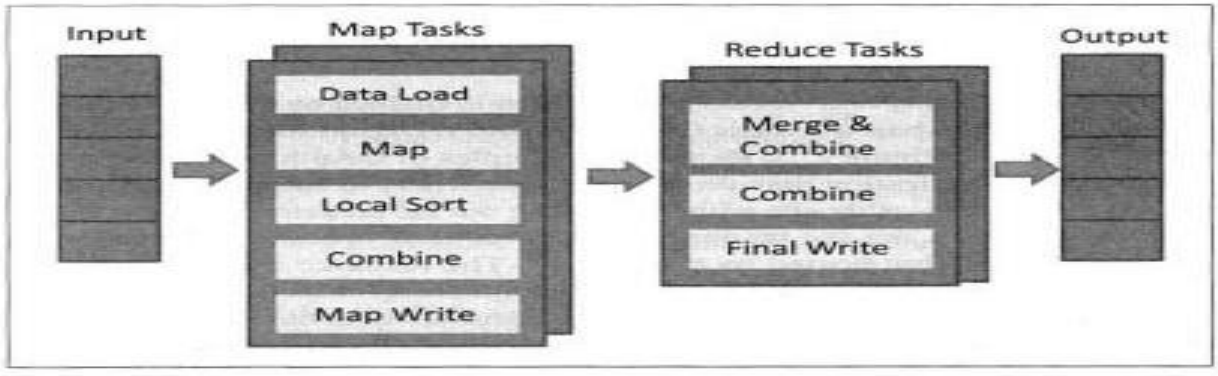


Figure : MapReduce workflow

The below figure shows the flow of data for a MapReduce job.

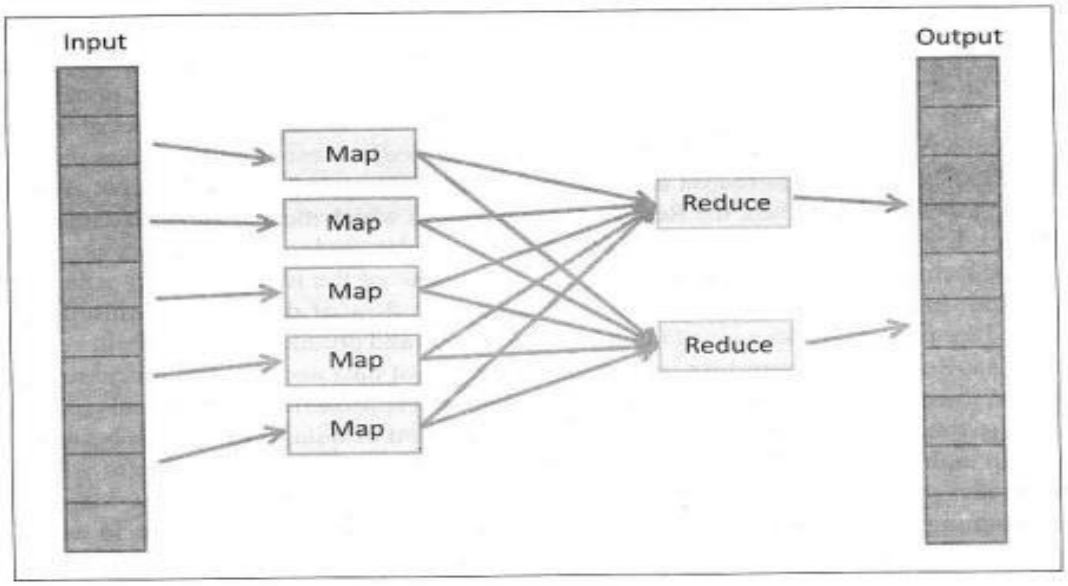


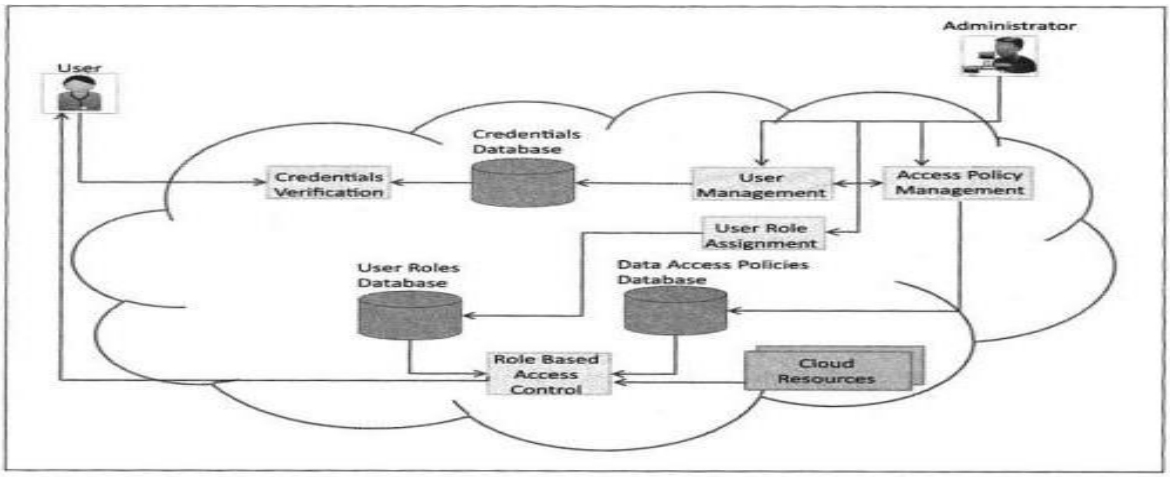
Figure Data flow in MapReduce

- MapReduce model take advantage of locality of data and the data processing takes place on the nodes where the data resides.

In traditional approach for data analysis, data is moved to the compute nodes which results in significant of data transmission between the nodes whereas MapReduce model moves the computation to where the data resides thus decreasing the transmission of data and improving efficiency

Identity access management:

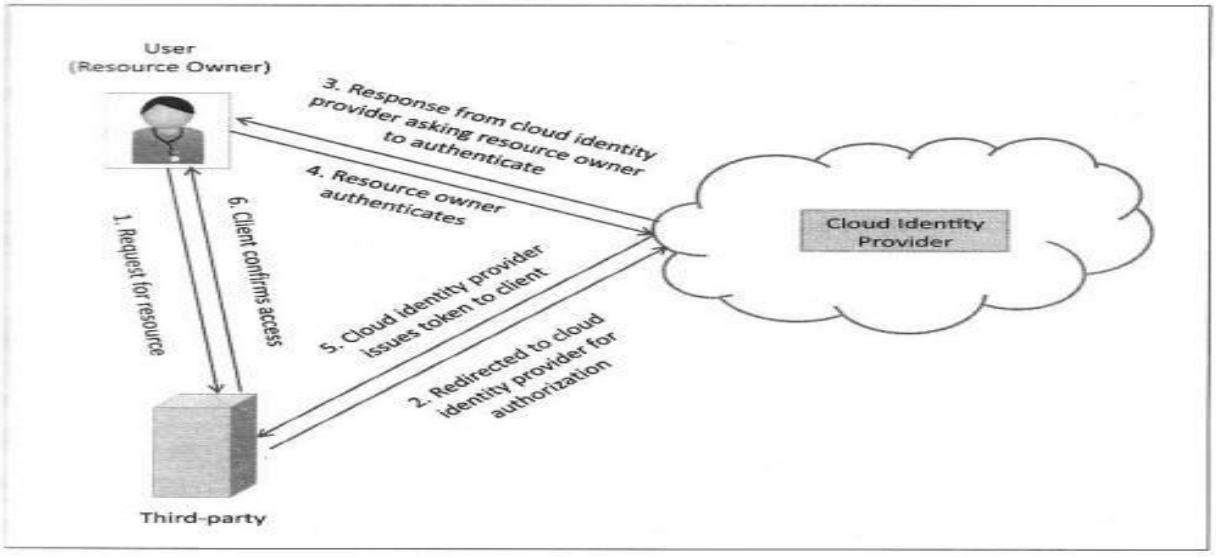
- Identity and Access Management (IDAM) for cloud describes the authentication and authorization of users toprovide secure access to cloud resources.
- Organization with multiple users can use IDAM services provided by the cloud service provider for management of user identifiers and user permissions.
- IDAM services allow organizations to centrally manage users, access permission, security credentials andaccess keys.
- IDAM services allow creation of user groups where all the users in a group have the same access permissions.
- Identity and Management is enabled by a number of technologies such as Open Auth, Role-based Access Control (RBAC), Digital Identities, Security Tokens, Identity Providers etc.



Role-based Access Control example

Identity and access Management:

- The below figure shows the examples of OAuth identity and Access Management Service provided by the cloud service providers.
- Oauth is an open standard for authorization that allows resource owners to share their private resources
  stored on one site with an other site without handling out the credentials.
- In the Oauth model, an application (which is not the resource owner) requests access to resources controlled bythe resource owner (but hosted by the server), the resource owner grants permission to access the resources inthe form of a token and matched shared-secret.
- Tokens can be issued with a restricted scope and limited lifetime and revoked



OAuth example

independently.

Identity and access Management:

- RBAC is an approach for restricting access to authorized users.
- The below figure shows an example of a typical RBAC framework.
- A user who wants to access cloud resources is required to send his/her data to the system administrator who assigns permissions and access control policies which are stored in the User Roles and Data Access Policies databases respectively.

- A Service Level Agreement (SLA) for cloud specifies the level of service that is formally defined as a part of theservice contract with the cloud service provider.

- SLAs provide a level of service for each service which is specified in the form of minimum level of service guaranteed and a target level.

- SLAs contain a number of performance metrics and the corresponding service objectives.

- The below table lists the common criteria cloud SLAs.

Criteria	Details
Availability	Percentage of time the service is guaranteed to be available
Performance	Response time, Throughput
Disaster Recovery	Mean time to recover
Problem resolution	Process to identify problems, support options, resolution expectations
Security and privacy of data	Mechanisms for security of data in storage and transmission

Table : List of criteria for cloud SLAs

Billing

Cloud Service providers offer a number of billing models described as follows:

- Elastic Pricing
- Fixed Pricing
- Spot Pricing

Elastic Pricing

- In Elastic pricing or Pay-as-you-use pricing model, the customers are charged based on the usage of cloud resources.
- Cloud computing provides the benefit of provision resources on demand.
- On-demand provisioning and elastic pricing models bring cost savings for the customers.
- Elastic pricing model is suited for customers who consume cloud resources for the short durations and whocannot predict the usage beforehand.

Fixed Pricing

- In fixed pricing models, customers are charged a fixed amount per month for the cloud resources.
- For eg: Fixed amount can be charged per month for the running a virtual machine instance, irrespective of the actualusage.
- Fixed pricing model is suited for customers who want to use cloud resources for longer durations and want more control over the cloud expenses

Spot Pricing

- Spot pricing models offer variables pricing for the cloud resources which is driven by market demand.
- When the demand for the cloud resources is high, the prices increase and when the demand is lower, the prices decrease.
- The below table lists the billable resources for cloud including virtual machines, network., storage, data services, security services, support, application services, deployment and management services.

Resource	Details
Virtual machines	CPU, memory, storage, disk I/O, network I/O
Network	Network I/O, load balancers, DNS, firewall, VPN
Storage	Cloud storage, storage volumes, storage gateway
Data services	Data import/export services, data encryption, data compression, data backup, data redundancy, content delivery
Security services	Identity and access management, isolation, compliance
Support	Level of support, SLA, fault tolerance
Application services	Queuing service, notification service, workflow service, payment service
Deployment and management services	Monitoring service, deployment service

Table : List of billable resources for cloud

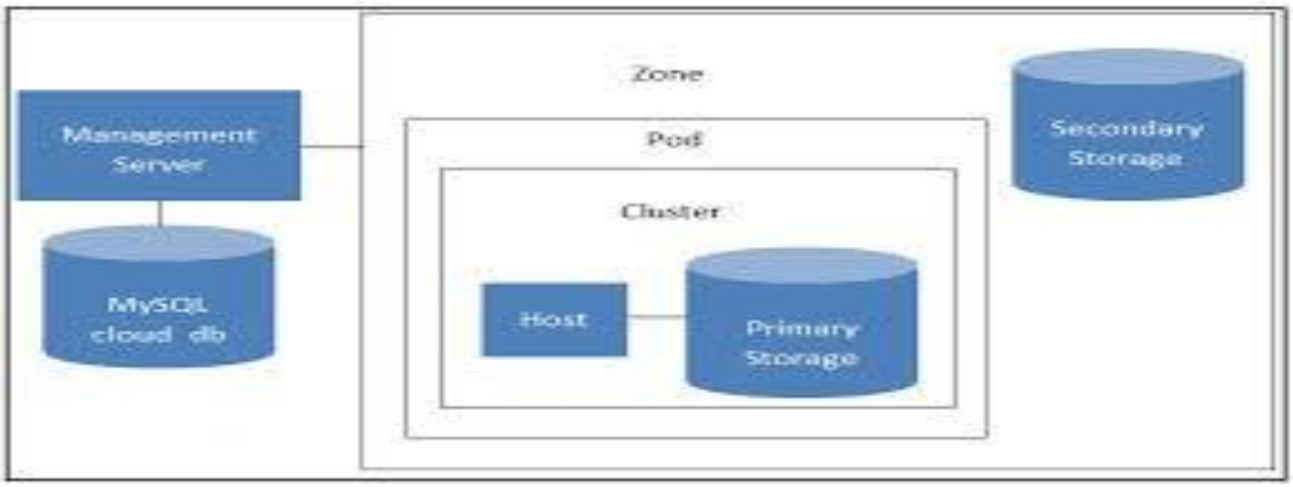
Open-Source Private Cloud Software

In the previous sections you learned about popular public cloud platforms. This section covers open-source cloud software that can be used to build private clouds.

Cloud Stack

Apache Cloud Stack is an open-source cloud software that can be used for creating private cloud offerings [15]. Cloud Stack manages the network, storage, and compute nodes that make up a cloud infrastructure. A Cloud Stack installation consists of a Management Server and the cloud infrastructure that it manages. The cloud infrastructure can be as simple as one host running the hypervisor or a large cluster of hundreds of hosts. The Management Server allows you to

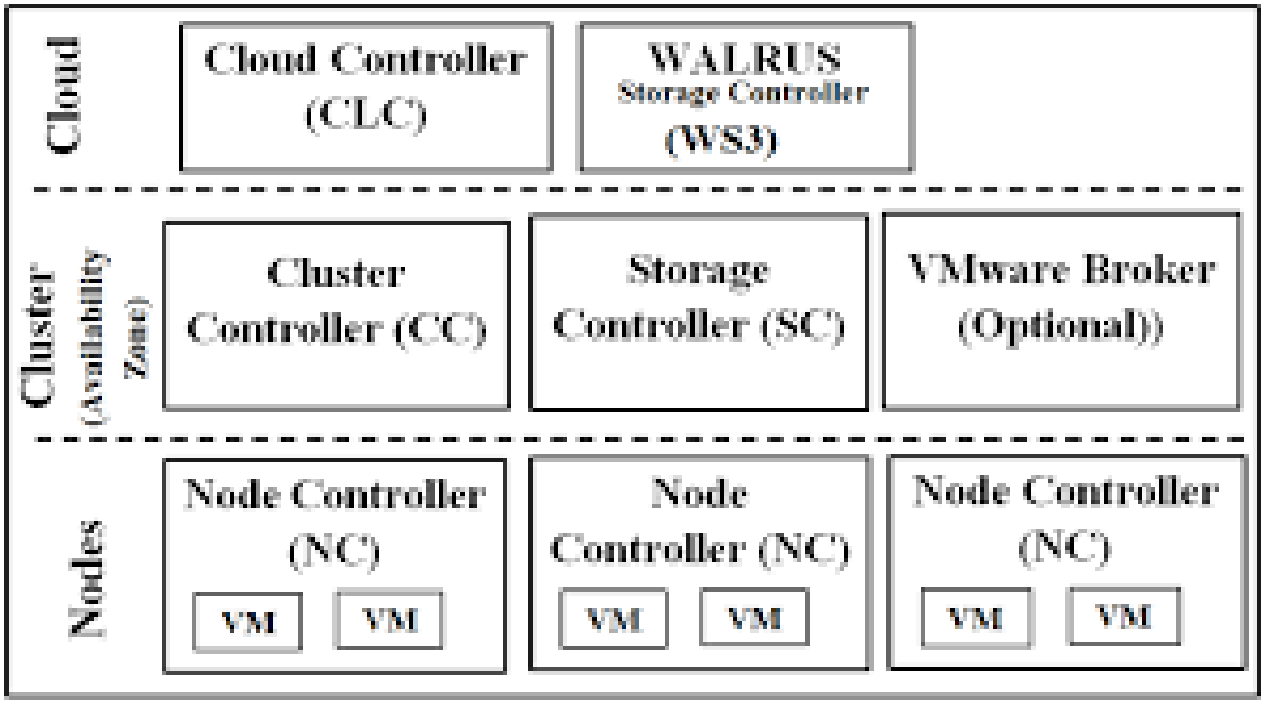
configure and manage the cloud resources. Figure 3.21 shows the architecture of CloudStack which is basically the Management Server. The Management Server manages one or more zones where each zone is typically a single datacenter. Each zone has one or more pods. A pod is a rack of hardware comprising of a switch and one or more clusters. A Cluster consists of one or more hosts and a primary storage. A host is a compute node that runs guest virtual machines. The primary storage of a cluster stores the disk volumes for all the virtual machines running on the hosts in that cluster. Each zone has a secondary storage that stores templates, ISO images, and disk volume snapshots.



Eucalyptus:

Eucalyptus is an open-source private cloud software for building private and hybrid clouds that are compatible with Amazon Web Services (AWS) APIs [16]. Figure 3.22 shows the architecture of Eucalyptus. The Node Controller (NC) hosts the virtual machine instances and manages the virtual network endpoints, The cluster-level (availability-zone) consists of three components - Cluster Controller (CC), Storage Controller (SC) and VMWare Broker. The CC manages the virtual machines and is the front-end for a cluster. The SC manages the Eucalyptus block volumes and snapshots to the instances within its specific cluster. SC is equivalent to AWS Elastic Block Store (EBS). The VMWare Broker is an optional component that provides an AWS-compatible interface for VMware environments. At the cloud-level there are two components - Cloud Controller (CLC) and Walrus. CLC provides an administrative interface for cloud management and performs high-level resource scheduling, system accounting, authentication and quota management.

Walrus is equivalent to Amazon S3 and serves as a persistent storage to all of the virtual machines in the Eucalyptus cloud. Walrus can be used as a simple Storage-as-a-Service solutions.



OpenStack

OpenStack is a cloud operating system comprising of a collection of interacting services that control computing, storage, and networking resources [17]. Figure 3.23 shows the architecture of OpenStack. The OpenStack compute service (called nova-compute) manages networks of virtual machines running on nodes, providing virtual servers on demand. The network service (called nova-networking) provides connectivity between the interfaces of other OpenStack services. The volume service (Cinder) manages storage volumes for virtual machines. The object storage service (swift) allows users to store and retrieve files. The identity service (keystone) provides authentication and authorization for other services. The image registry (glance) acts as a catalog and repository for virtual machine images. The OpenStack scheduler (nova-scheduler) maps the nova-API calls to the appropriate OpenStack components. The scheduler takes the virtual machine requests from the queue and determines where they should run. The messaging service (rabbit-mq) acts as a central node for message passing between daemons. Orchestration activities such as running an instance are performed by the nova-api which accepts and responds to end user compute API calls. The OpenStack dashboard (called horizon) provides web-based interface for managing OpenStack services.

UNIT – II

Hadoop& Map Reduce:
Apache Hadoop, Hadoop Map Reduce Job Execution, Hadoop Schedulers, Hadoop Cluster setup.
Cloud Application Design:
Reference Architecture for Cloud Applications, Cloud Application Design Methodologies, Data Storage Approaches.

Apache Hadoop:

Hadoop is an open source framework from Apache and is used to store process and analyze data which are very huge in volume.

Hadoop is written in Java and is not OLAP (online analytical processing). It is used for batch/offline processing.

It is being used by Facebook, Yahoo, Google, Twitter, LinkedIn and many more. Moreover it can be scaled up just by adding nodes in the cluster.

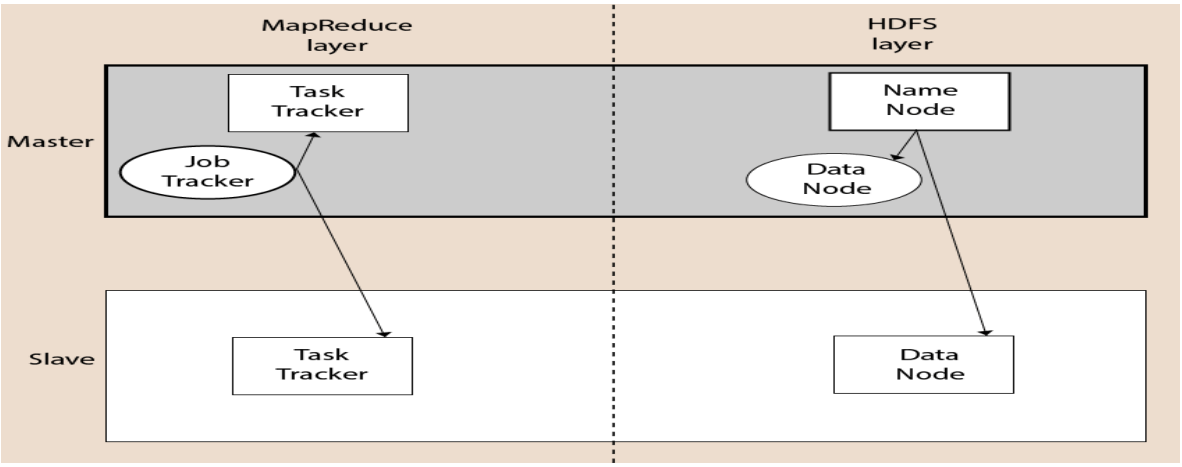
Modules of Hadoop:

1. **HDFS:** Hadoop Distributed File System. Google published its paper GFS and on the basis of that HDFS was developed. It states that the files will be broken into blocks and stored in nodes over the distributed architecture.
2. **Yarn:** Yet another Resource Negotiator is used for job scheduling and manage the cluster.
3. **Map Reduce:** This is a framework which helps Java programs to do the parallel computation on data using key value pair. The Map task takes input data and converts it into a data set which can be computed in Key value pair. The output of Map task is consumed by reduce task and then the out of reducer gives the desired result.
4. **Hadoop Common**: These Java libraries are used to start Hadoop and are used by other Hadoop modules.

Hadoop Architecture:

The Hadoop architecture is a package of the file system, MapReduce engine and the HDFS (Hadoop Distributed File System). The MapReduce engine can be MapReduce/MR1 or YARN/MR2.

A Hadoop cluster consists of a single master and multiple slave nodes. The master node includes Job Tracker, Task Tracker, NameNode, and DataNode whereas the slave node includes DataNode and TaskTracker.



Hadoop Distributed File System:

The Hadoop Distributed File System (HDFS) is a distributed file system for Hadoop. It contains a master/slave architecture. This architecture consist of a single NameNode performs the role of master, and multiple DataNodes performs the role of a slave.

Both NameNode and DataNode are capable enough to run on commodity machines. The Java language is used to develop HDFS. So any machine that supports Java language can easily run the NameNode and DataNode software.

NameNode

- It is a single master server exist in the HDFS cluster.
- As it is a single node, it may become the reason of single point failure.
- It manages the file system namespace by executing an operation like the opening, renaming and closing the files.
- It simplifies the architecture of the system.

DataNode

- The HDFS cluster contains multiple DataNodes.
- Each DataNode contains multiple data blocks.
- These data blocks are used to store data.
- It is the responsibility of DataNode to read and write requests from the file system's clients.
- It performs block creation, deletion, and replication upon instruction from the NameNode.

Job Tracker:

- The role of Job Tracker is to accept the MapReduce jobs from client and process the data by using NameNode.
- In response, NameNode provides metadata to Job Tracker.

Task Tracker:

- It works as a slave node for Job Tracker.
- It receives task and code from Job Tracker and applies that code on the file. This process can also be called as a Mapper.

MapReduce Layer:

The MapReduce comes into existence when the client application submits the MapReduce job to Job Tracker. In response, the Job Tracker sends the request to the appropriate Task Trackers. Sometimes, the TaskTracker fails or time out. In such a case, that part of the job is rescheduled.

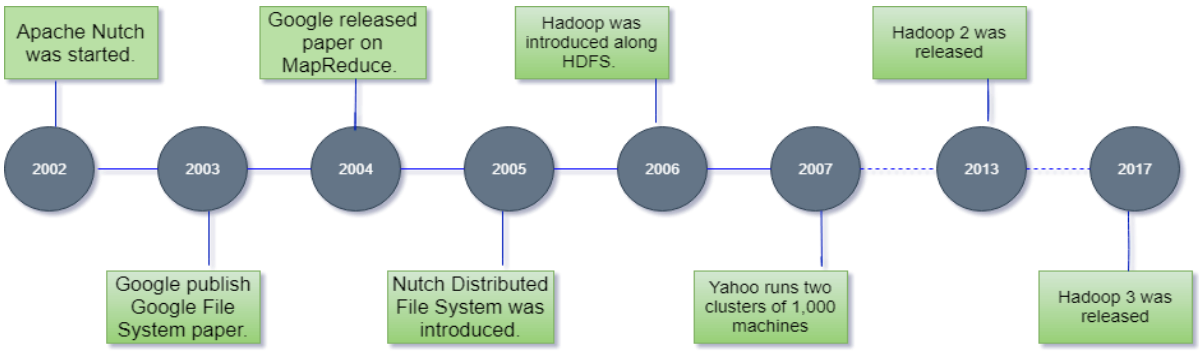
Advantages of Hadoop:

- **Fast:** In HDFS the data distributed over the cluster and are mapped which helps in faster retrieval. Even the tools to process the data are often on the same servers, thus reducing the processing time. It is able to process terabytes of data in minutes and Peta bytes in hours.
- **Scalable:** Hadoop cluster can be extended by just adding nodes in the cluster.
- **Cost Effective:** Hadoop is open source and uses commodity hardware to store data so it really cost effective as compared to traditional relational database management system.

- **Resilient to failure:** HDFS has the property with which it can replicate data over the network, so if one node is down or some other network failure happens, then Hadoop takes the other copy of data and use it. Normally, data are replicated thrice but the replication factor is configurable.

History of Hadoop:

The Hadoop was started by Doug Cutting and Mike Cafarella in 2002. Its origin was the Google File System paper, published by Google.



Let's focus on the history of Hadoop in the following steps: -

- In 2002, Doug Cutting and Mike Cafarella started to work on a project, **Apache Nutch.** It is an open source web crawler software project.
- While working on Apache Nutch, they were dealing with big data. To store that data they have to spend a lot of costs which becomes the consequence of that project. This problem becomes one of the important reason for the emergence of Hadoop.
- In 2003, Google introduced a file system known as GFS (Google file system). It is a proprietary distributed file system developed to provide efficient access to data.
- In 2004, Google released a white paper on Map Reduce. This technique simplifies the data processing on large clusters.
- In 2005, Doug Cutting and Mike Cafarella introduced a new file system known as NDFS (Nutch Distributed File System). This file system also includes Map reduce.
- In 2006, Doug Cutting quit Google and joined Yahoo. On the basis of the Nutch project, Dough Cutting introduces a new project Hadoop with a file system known as HDFS (Hadoop Distributed File System). Hadoop first version 0.1.0 released in this year.
- Doug Cutting gave named his project Hadoop after his son's toy elephant.
- In 2007, Yahoo runs two clusters of 1000 machines.
- In 2008, Hadoop became the fastest system to sort 1 terabyte of data on a 900 node cluster within 209 seconds.
- In 2013, Hadoop 2.2 was released.
- In 2017, Hadoop 3.0 was released.

Year	Event
2003	Google released the paper, Google File System (GFS).
2004	Google released a white paper on Map Reduce.
2006	o Hadoop introduced. o Hadoop 0.1.0 released. o Yahoo deploys 300 machines and within this year reaches 600 machines.
2007	o Yahoo runs 2 clusters of 1000 machines. o Hadoop includes HBase.
2008	o YARN JIRA opened o Hadoop becomes the fastest system to sort 1 terabyte of data on a 900 node cluster within 209 seconds. o Yahoo clusters loaded with 10 terabytes per day. o Cloudera was founded as a Hadoop distributor.
2009	o Yahoo runs 17 clusters of 24,000 machines. o Hadoop becomes capable enough to sort a petabyte. o MapReduce and HDFS become separate subproject.
2010	o Hadoop added the support for Kerberos. o Hadoop operates 4,000 nodes with 40 petabytes. o Apache Hive and Pig released.
2011	o Apache Zookeeper released. o Yahoo has 42,000 Hadoop nodes and hundreds of petabytes of storage.
2012	Apache Hadoop 1.0 version released.
2013	Apache Hadoop 2.2 version released.
2014	Apache Hadoop 2.6 version released.
2015	Apache Hadoop 2.7 version released.
2017	Apache Hadoop 3.0 version released.
2018	Apache Hadoop 3.1 version released.

MapReduce:

Hadoop MapReduce is the data processing layer. It processes the huge amount of structured and unstructured data stored in HDFS. MapReduce handles data in parallel by splitting the job into the set of independent tasks. So, parallel processing increases speed and reliability.

Hadoop MapReduce data processing occurs in 2 phases- Map and Reduce phase.

- **Map phase:** It is the initial phase of data processing. In this phase, we state all the complex logic/business rules/costly code.
- **Reduce phase:** The second phase of processing is the Reduce Phase. In this phase, we state light-weight processing like aggregation/summation.

Steps of MapReduce Job Execution flow:

MapReduce processes the data in various phases with the help of different components. Let us discuss the steps of job execution in Hadoop.

Input Files:

The data for MapReduce job is stored in Input Files. Input files reside in HDFS. The input file format is random. Line-based log files and binary format can also be used.

InputFormat:

After that InputFormat defines how to divide and read these input files. It selects the files or other objects for input. InputFormat creates InputSplit.

InputSplits:

It represents the data that will be processed by an individual Mapper. For each split, one map task is created. Therefore the number of map tasks is equal to the number of InputSplits. Framework divide split into records, which mapper process.

RecordReader:

It communicates with the inputSplit. And then transforms the data into key-value pairs suitable for reading by the Mapper. RecordReader by default uses TextInputFormat to transform data into a key-value pair. It interrelates to the InputSplit until the completion of file reading. It allocates a byte offset to each line present in the file. Then, these key-value pairs are further sent to the mapper for added processing.

Mapper:

It processes input records produced by the RecordReader and generates intermediate key-value pairs. The intermediate output is entirely different from the input pair. The output of the mapper is a full group of key-value pairs. Hadoop framework does not store the output of the mapper on HDFS. Mapper doesn't store, as data is temporary and writing on HDFS will create unnecessary multiple copies. Then Mapper passes the output to the combiner for extra processing.

Combiner:

Combiner is Mini-reducer that performs local aggregation on the mapper's output. It minimizes the data transfer between mapper and reducer. So, when the combiner functionality completes, the framework passes the output to the partitioner for further processing.

Partitioner:

Partitioner comes into existence if we are working with more than one reducer. It grabs the output of the combiner and performs partitioning.

Partitioning of output occurs based on the key in MapReduce. By hash function, the key (or a subset of the key) derives the partition.

Based on key-value in MapReduce, partitioning of each combiner output occur. And then the record having a similar key-value goes into the same partition. After that, each partition is sent to a reducer.

Partitioning in MapReduce execution permits even distribution of the map output over the reducer.

Shuffling and Sorting:

After partitioning, the output is shuffled to the reduced node. The shuffling is the physical transfer of the data which is done over the network. As all the mappers complete and shuffle the output on the reducer nodes. Then the framework joins this intermediate output and sort. This is then offered as input to reduce the phase.

Reducer:

The reducer then takes a set of intermediate key-value pairs produced by the mappers as the input. After that runs a reducer function on each of them to generate the output. The output of the reducer is the decisive output. Then the framework stores the output on HDFS.

RecordWriter:

It writes these output key-value pairs from the Reducer phase to the output files.

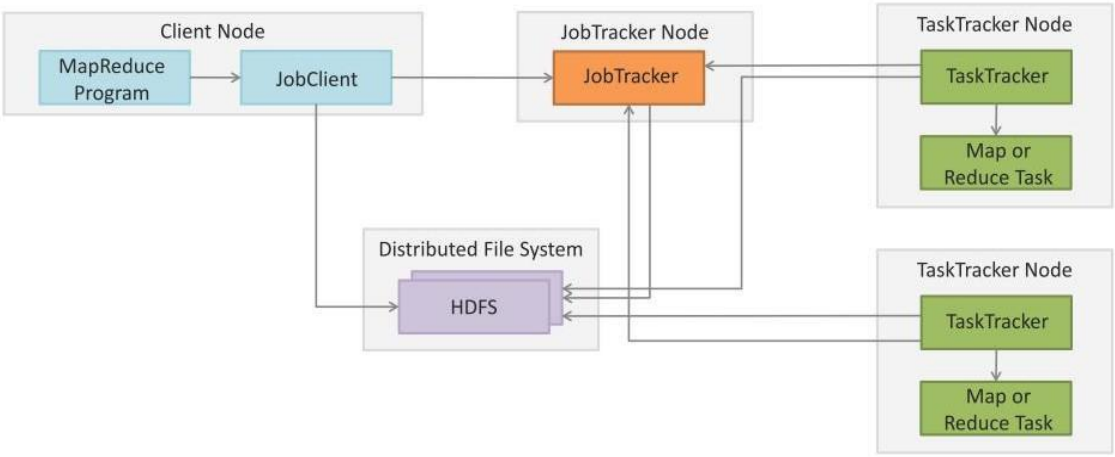
OutputFormat:

OutputFormat defines the way how RecordReader writes these output key-value pairs in output files. So, its instances offered by the Hadoop write files in HDFS. Thus OutputFormat instances write the decisive output of reducer on HDFS.

\

MapReduce Job Execution Workflow:

- MapReduce job execution starts when the client applications submit jobs to the Job tracker.
- The JobTracker returns a JobID to the client application. The JobTracker talks to the NameNode to determine the location of the data.
- The JobTracker locates TaskTracker nodes with available slots at/or near the data.
- The TaskTrackers send out heartbeat messages to the JobTracker, usually every few minutes, to reassure the JobTracker that they are still alive. These messages also inform the JobTracker of the number of available slots, so the JobTracker can stay up to date with where in the cluster, new work can be delegated.
- The JobTracker submits the work to the TaskTracker nodes when they poll for tasks. To choose a task for a TaskTracker, the JobTracker uses various scheduling algorithms (default is FIFO).
- The TaskTracker nodes are monitored using the heartbeat signals that are sent by the TaskTrackers to JobTracker.
- The TaskTracker spawns a separate JVM process for each task so that any task failure does not bring down the TaskTracker.
- The TaskTracker monitors these spawned processes while capturing the output and exit codes. When the process finishes, successfully or not, the TaskTracker notifies the JobTracker. When the job is completed, the JobTracker updates its status.

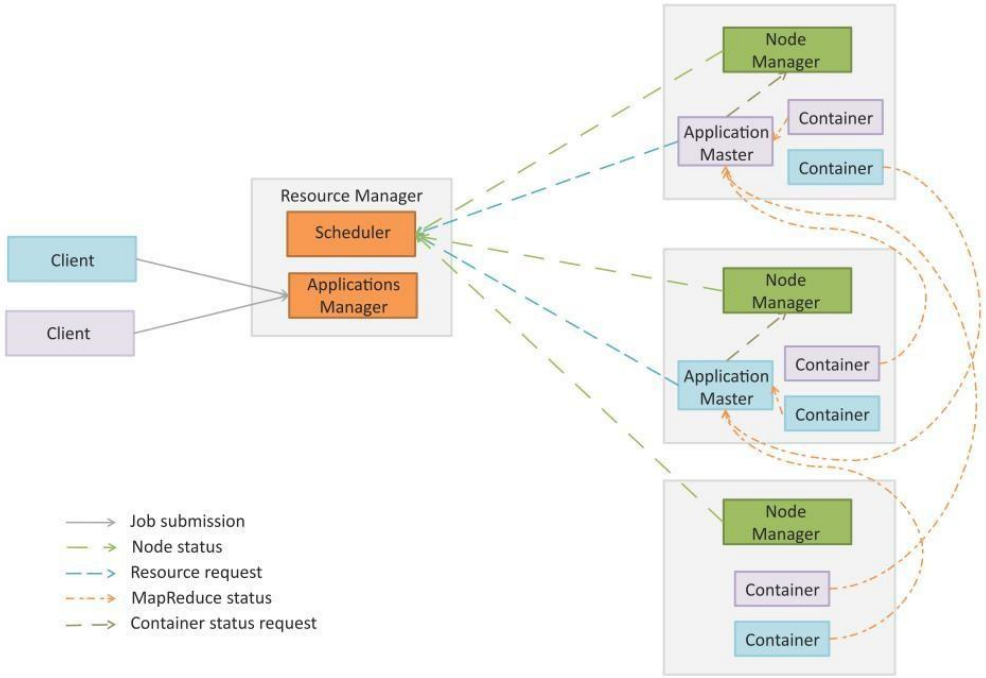


MapReduce 2.0 – YARN:

- In Hadoop 2.0 the original processing engine of Hadoop (MapReduce) has been separated from the resource management (which is now part of YARN).
- This makes YARN effectively an operating system for Hadoop that supports different processing engines on a Hadoop cluster such as MapReduce for batch processing, Apache Tez for interactive queries, Apache Storm for stream processing, etc.
- YARN architecture divides architecture divides the two major functions of the JobTracker - resource management and job life-cycle management - into separate components:
    - ResourceManager
    - ApplicationMaster.

YARN Components

- **Resource Manager (RM):** RM manages the global assignment of compute resources to applications. RM consists of two main services:
    - **Scheduler:** Scheduler is a pluggable service that manages and enforces the resource scheduling policy in the cluster.
    - **Applications Manager(ASM):** AsM manages the running Application Masters in the cluster. AsM is responsible for starting application masters and for monitoring and restarting them on different nodes in case of failures.
- **Application Master (AM):** A per-application AM manages the application's life cycle. AM is responsible for negotiating resources from the RM and working with the NMs to execute and monitor the tasks.
- **Node Manager (NM):** A per-machine NM manages the user processes on that machine.
- **Containers:** Container is a bundle of resources allocated by RM (memory, CPU, network, etc.). A container is a conceptual entity that grants an application the privilege to use a certain amount of resources on a given machine to run a component task.



Hadoop Scheduler:

Prior to Hadoop 2, **Hadoop MapReduce** is a software framework for writing applications that process huge amounts of data (terabytes to petabytes) in-parallel on the large Hadoop cluster. This framework is responsible for scheduling tasks, monitoring them, and re-executes the failed task.

In Hadoop 2, a **YARN** called Yet Another Resource Negotiator was introduced. The basic idea behind the YARN introduction is to split the functionalities of resource management and job scheduling or monitoring into separate daemons that are ResorceManager, ApplicationMaster, and NodeManager.

ResorceManager is the master daemon that arbitrates resources among all the applications in the system. NodeManager is the slave daemon responsible for containers, monitoring their resource usage, and reporting the same to ResourceManager or Schedulers. ApplicationMaster negotiates resources from the ResourceManager and works with NodeManager in order to execute and monitor the task.

The ResourceManager has two main components that are Schedulers and ApplicationsManager.

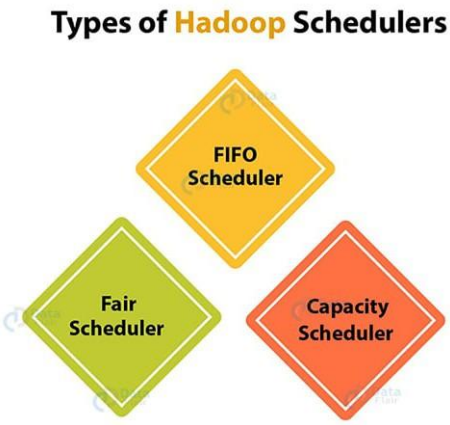
Schedulers in YARN ResourceManager is a pure scheduler which is responsible for allocating resources to the various running applications.

It is not responsible for monitoring or tracking the status of an application. Also, the scheduler does not guarantee about restarting the tasks that are failed either due to hardware failure or application failure.

It has some pluggable policies that are responsible for partitioning the cluster resources among the various queues, applications, etc.

The FIFO Scheduler, CapacityScheduler, and FairScheduler are such pluggable policies that are responsible for allocating resources to the applications. Let us now study each of these Schedulers in detail.

TYPESOF HADOOPSCHEDULER



1. FIFO Scheduler:

First In First Out is the default scheduling policy used in Hadoop. FIFO Scheduler gives more preferences to the application coming first than those coming later. It places the applications in a queue and executes them in the order of their submission (first in, first out).

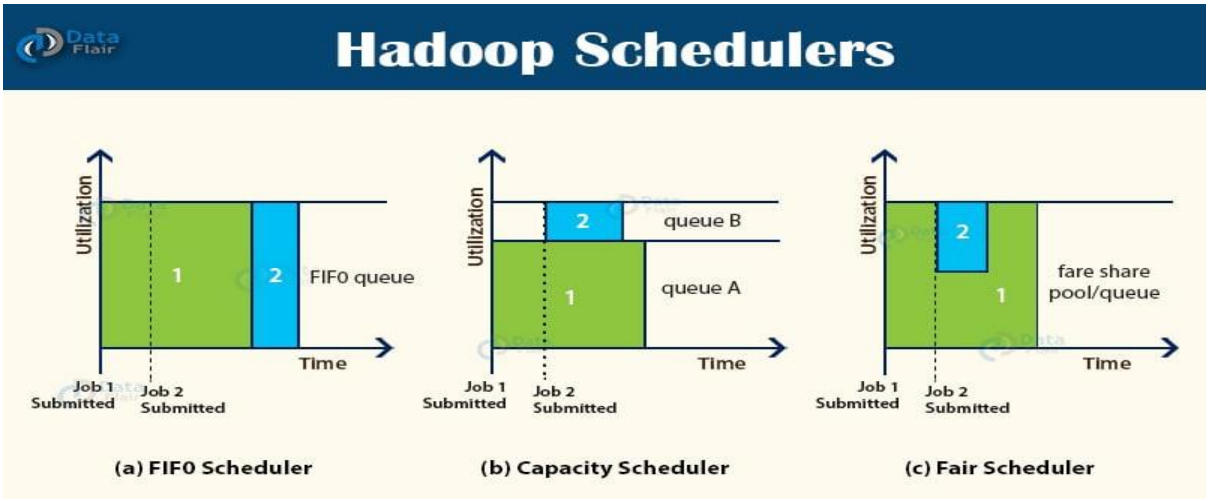
Here, irrespective of the size and priority, the request for the first application in the queue are allocated first. Once the first application request is satisfied, then only the next application in the queue is served.

Advantage:

- It is simple to understand and doesn't need any configuration.
- Jobs are executed in the order of their submission.

Disadvantage:

- It is not suitable for shared clusters. If the large application comes before the shorter one, then the large application will use all the resources in the cluster, and the shorter application has to wait for its turn. This leads to starvation.
- It does not take into account the balance of resource allocation between the long applications and short applications.



2. Capacity Scheduler:

The CapacityScheduler allows multiple-tenants to securely share a large Hadoop cluster. It is designed to run Hadoop applications in a shared, multi-tenant cluster while maximizing the throughput and the utilization of the cluster.

It supports hierarchical queues to reflect the structure of organizations or groups that utilizes the cluster resources. A queue hierarchy contains three types of queues that are root, parent, and leaf.

The root queue represents the cluster itself, parent queue represents organization/group or sub-organization/sub-group, and the leaf accepts application submission.

The Capacity Scheduler allows the sharing of the large cluster while giving capacity guarantees to each organization by allocating a fraction of cluster resources to each queue.

Also, when there is a demand for the free resources that are available on the queue who has completed its task, by the queues running below capacity, then these resources will be assigned to the applications on queues running below capacity. This provides elasticity for the organization in a cost-effective manner.

Apart from it, the Capacity Scheduler provides a comprehensive set of limits to ensure that a single application/user/queue cannot use a disproportionate amount of resources in the cluster.

To ensure fairness and stability, it also provides limits on initialized and pending apps from a single user and queue.

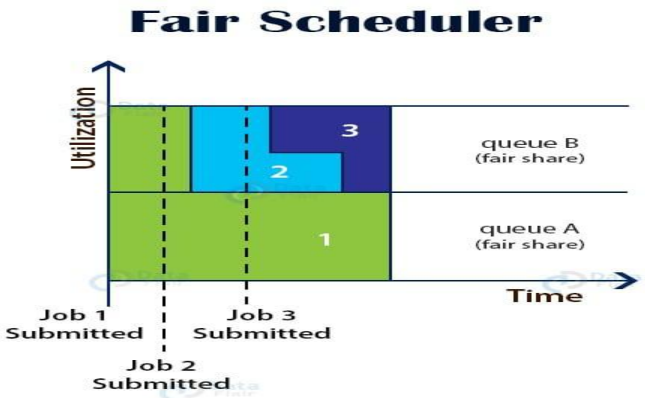
Advantages:

- It maximizes the utilization of resources and throughput in the Hadoop cluster.
- Provides elasticity for groups or organizations in a cost-effective manner.
- It also gives capacity guarantees and safeguards to the organization utilizing cluster.

Disadvantage:

- It is complex amongst the other scheduler.

3. Fair Scheduler:



FairScheduler allows YARN applications to fairly share resources in large Hadoop clusters. With FairScheduler, there is no need for reserving a set amount of capacity because it will dynamically balance resources between all running applications.

It assigns resources to applications in such a way that all applications get, on average, an equal amount of resources over time. The FairScheduler, by default, takes scheduling fairness decisions only on the basis of memory. We can configure it to schedule with both memory and CPU.

When the single application is running, then that app uses the entire cluster resources. When other applications are submitted, the free up resources are assigned to the new apps so that every app eventually gets roughly the same amount of resources. FairScheduler enables short apps to finish in a reasonable time without starving the long-lived apps.

Similar to CapacityScheduler, the FairScheduler supports hierarchical queue to reflect the structure of the long shared cluster.

Apart from fair scheduling, the FairScheduler allows for assigning minimum shares to queues for ensuring that certain users, production, or group applications always get sufficient resources. When an app is present in the queue, then the app gets its minimum share, but when the queue doesn't need its full guaranteed share, then the excess share is split between other running applications.

Advantages:

- It provides a reasonable way to share the Hadoop Cluster between the number of users.
- Also, the FairScheduler can work with app priorities where the priorities are used as weights in determining the fraction of the total resources that each application should get.

Disadvantage: It requires configuration.

Hadoop Cluster setup:

Having installed and configured Hadoop the next step is to start the Hadoop cluster. Box 4.7 lists the commands for starting and stopping the Hadoop cluster.

If the Hadoop cluster is correctly installed, configured and started, the status of the Hadoop daemons can be viewed using the administration web-pages for the daemons. Hadoop publishes the status of HDFS and MapReduce jobs to an internally running web server on the master node of the Hadoop cluster. The default addresses of the web UIs as follows:

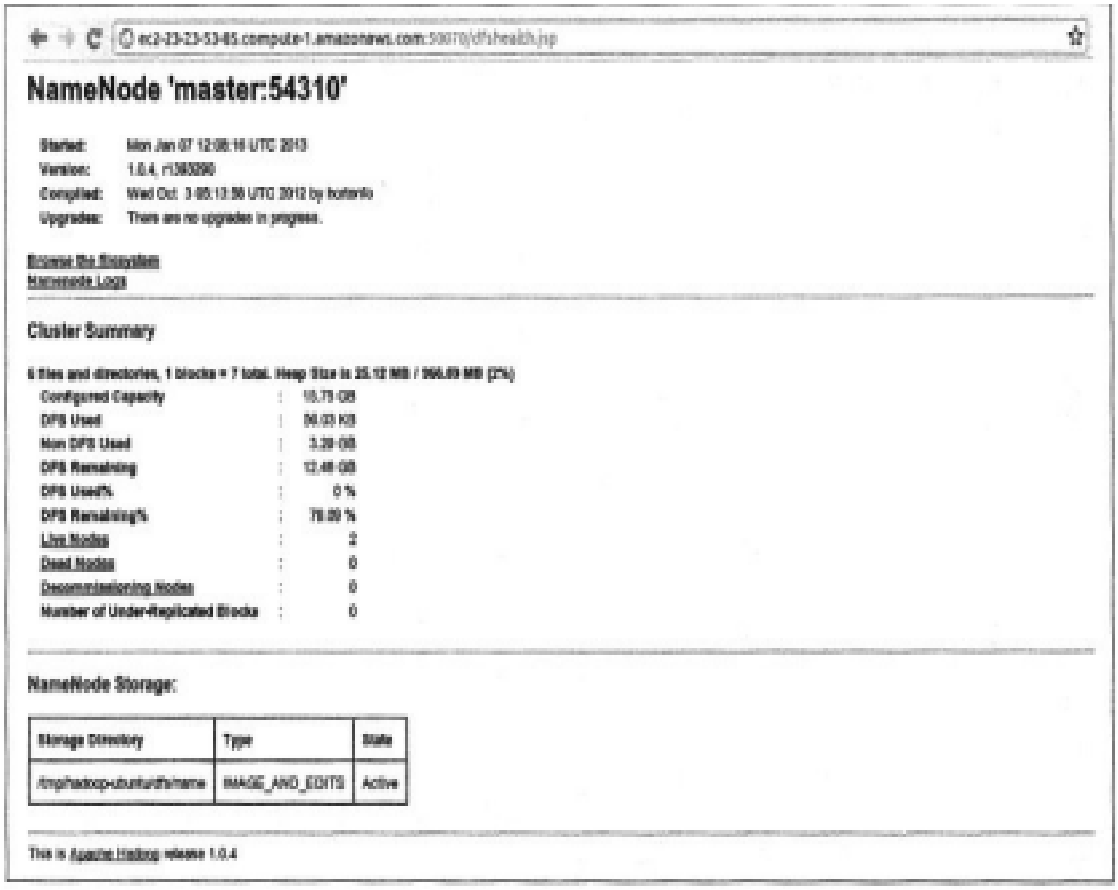


Figure 4.10: Hadoop NameNode status page



Figure 4.11: Hadoop MapReduce administration page

NameNode - http://<NameNodeHostName>:50070/
JobTracker - http://<JobTrackerHostName>:50030/
Figure 4.10 shows the Hadoop NameNode status page which provides information about NameNode uptime, the number of live, dead, and decommissioned nodes, host and port information, safe mode status, heap information, audit logs, garbage collection metrics, total load, file operations, and CPU usage.

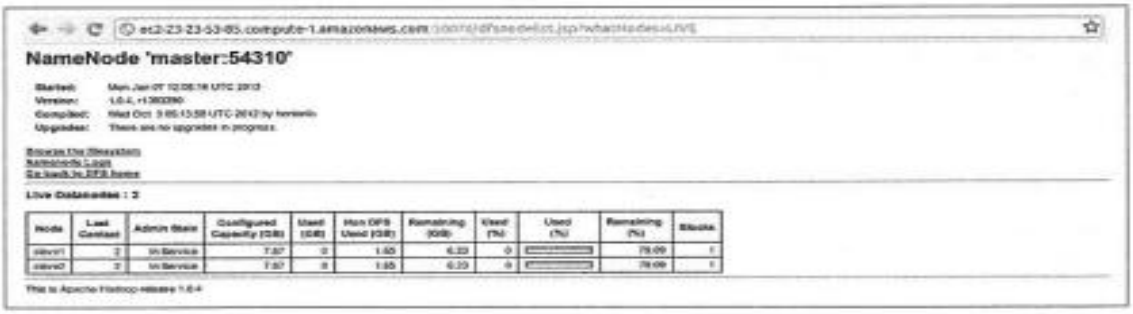


Figure 4.12: Hadoop HDFS status page showing live data nodes

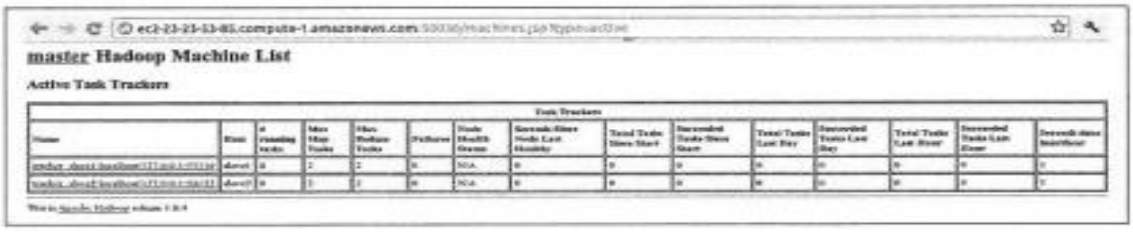


Figure 4.13: Hadoop MapReduce status page showing active TaskTrackers

Figure 4.11 shows the MapReduce administration page which provides host and port information, start time, tracker counts, heap information, scheduling information, current running jobs, retired jobs, job history log, service daemon logs, thread stacks, and a cluster utilization summary.

Figure 4.12 shows the status page of the live data nodes of the Hadoop cluster. The status page shows two live data nodes – slave1 and slave2.

Figure 4.13 shows the status page of the active TaskTrackers of the Hadoop cluster. The status page shows two active TaskTrackers that run on the slave1 and slave2 nodes of the cluster.

Cloud Application Design

Design Considerations for Cloud Applications:

- Scalability
    - Scalability is an important factor that drives the application designers to move to cloud computing environments. Building applications that can serve millions of users without taking a hit on their performance has always been challenging. With the growth of cloud computing application designers can provision adequate resources to meet their workload levels.
    - Reliability & Availability
    - Reliability of a system is defined as the probability that a system will perform the intended functions under stated conditions for a specified amount of time. Availability is the probability that a system will perform a specified function under given conditions at a prescribed time.
- Security
    - Security is an important design consideration for cloud applications given the outsourced nature of cloud computing environments.
    - Maintenance & Upgradation
    - To achieve a rapid time-to-market, businesses typically launch their applications with a core set of features ready and then incrementally add new features as and when they are complete. In such scenarios, it is important to design applications with low maintenance and upgradation costs.
- Performance
    - Applications should be designed while keeping the performance requirements in mind.

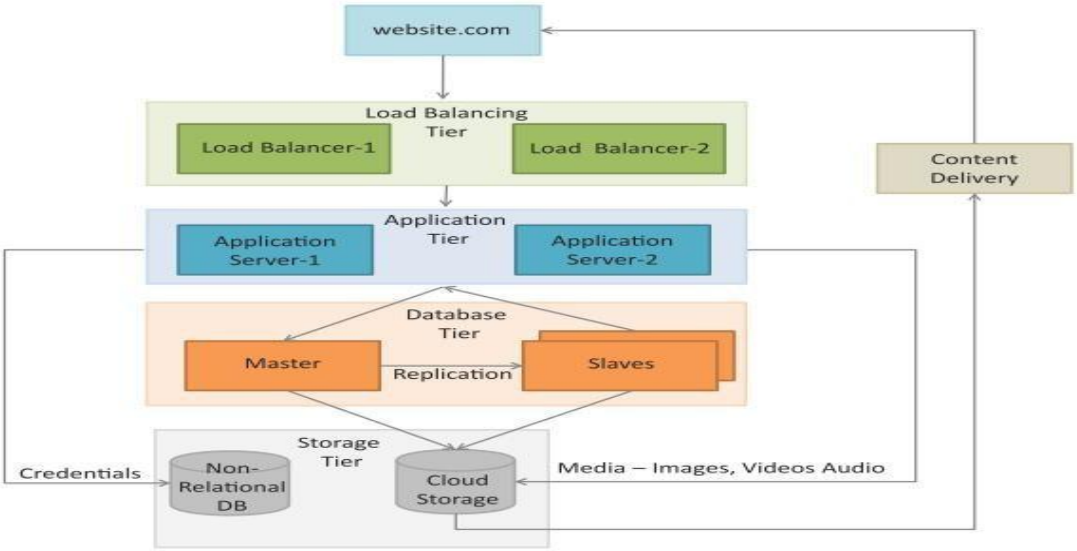
Reference Architectures –e-Commerce, Business-to-Business, Banking and Financial apps

- Load Balancing Tier
    - Load balancing tier consists of one or more load balancers. \
- Application Tier
    - For this tier, it is recommended to configure auto scaling.
    - Auto scaling can be triggered when the recorded values for any of the specified metrics such as CPU usage, memory usage, etc. goes above defined thresholds.
- Database Tier
    - The database tier includes a master database instance and multiple slave instances.
    - The master node serves all the write requests and the read requests are served from the slave nodes.
    - This improves the throughput for the database tier since most applications have a higher number of read requests than write requests.



Reference Architectures –Content delivery apps:

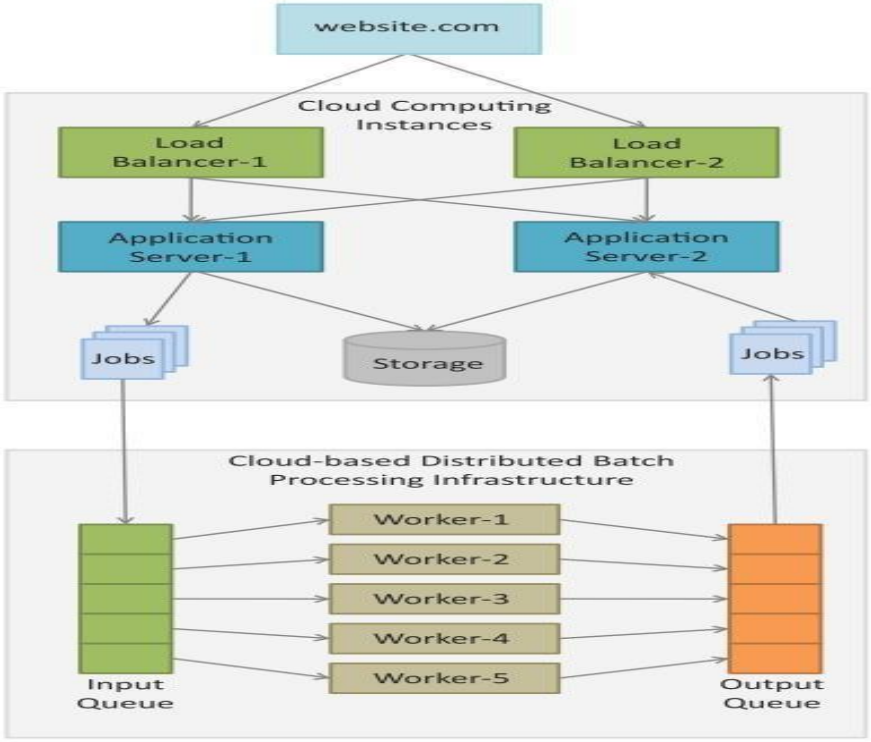
- Figure shows a typical deployment architecture for content delivery applications such as online photo albums, video webcasting, etc.
- Both relational and non-relational data stores are shown in this deployment.
- A content delivery network (CDN) which consists of a global network of edge locations is used for media delivery.
- CDN is used to speed up the delivery of static content such as images and videos.



Reference Architectures –Analytics apps

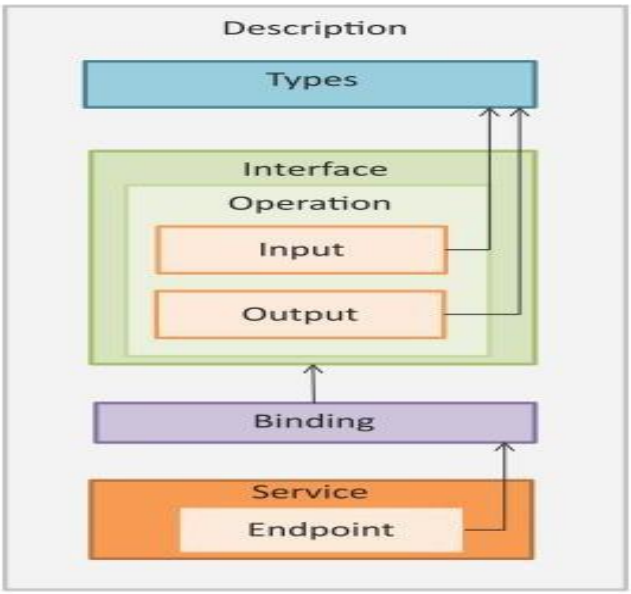
- Figure shows a typical deployment architecture for compute intensive applications such as Data Analytics, Media Transcoding, etc.
- Comprises of web, application, storage, computing/analytics and database tiers.
- The analytics tier consists of cloud-based distributed batch processing frameworks such as Hadoop which are suitable for analyzing big data.
- Data analysis jobs (such as MapReduce) jobs are submitted to the analytics tier from the application servers

- The jobs are queued for execution and upon completion the analyzed data is presented from the application servers.



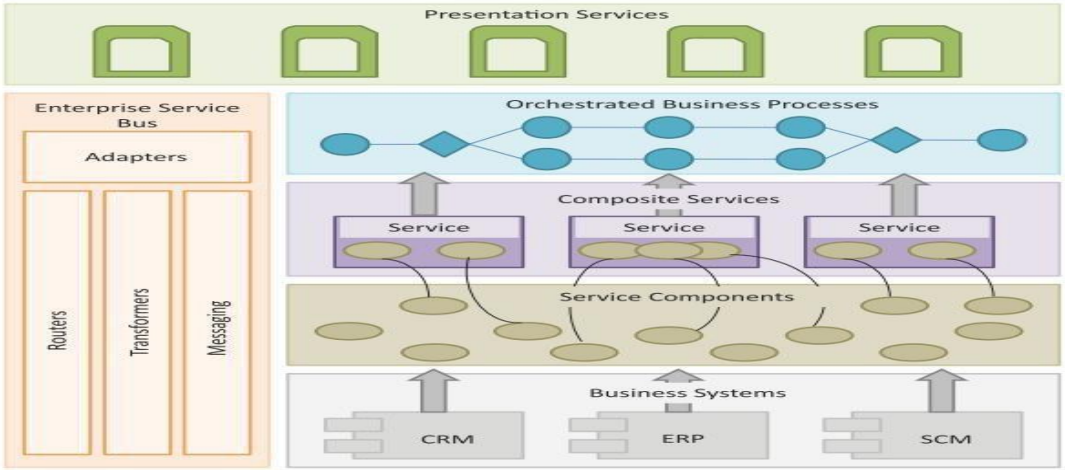
Service Oriented Architecture:

- Service Oriented Architecture (SOA) is a well established architectural approach for designing and developing applications in the form services that can be shared and reused.
- SOA is a collection of discrete software modules or services that form a part of an application and collectively provide the functionality of an application.
- SOA services are developed as loosely coupled modules with no hard- wired calls embedded in the services.
- The services communicate with each other by passing messages.
- Services are described using the Web Services Description Language (WSDL).
- WSDL is an XML-based web services description language that is used to create service descriptions containing information on the functions performed by a service and the inputs and outputs of the service.



SOA Layers:

1. **Business Systems:** This layer consists of custom built applications and legacy systems such as Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), Supply Chain Management (SCM), etc.
2. **Service Components:** The service components allow the layers above to interact with the business systems. The service components are responsible for realizing the functionality of the services exposed.
3. **Composite Services:** These are coarse-grained services which are composed of two or more service components. Composite services can be used to create enterprise scale components or business-unit specific components.
4. **Orchestrated Business Processes:** Composite services can be orchestrated to create higher level business processes. In this layers the compositions and orchestrations of the composite services are defined to create business processes.
5. **Presentation Services:** This is the topmost layer that includes user interfaces that exposes the services and the orchestrated business processes to the users.
6. **Enterprise Service Bus:** This layer integrates the services through adapters, routing, transformation and messaging mechanisms.

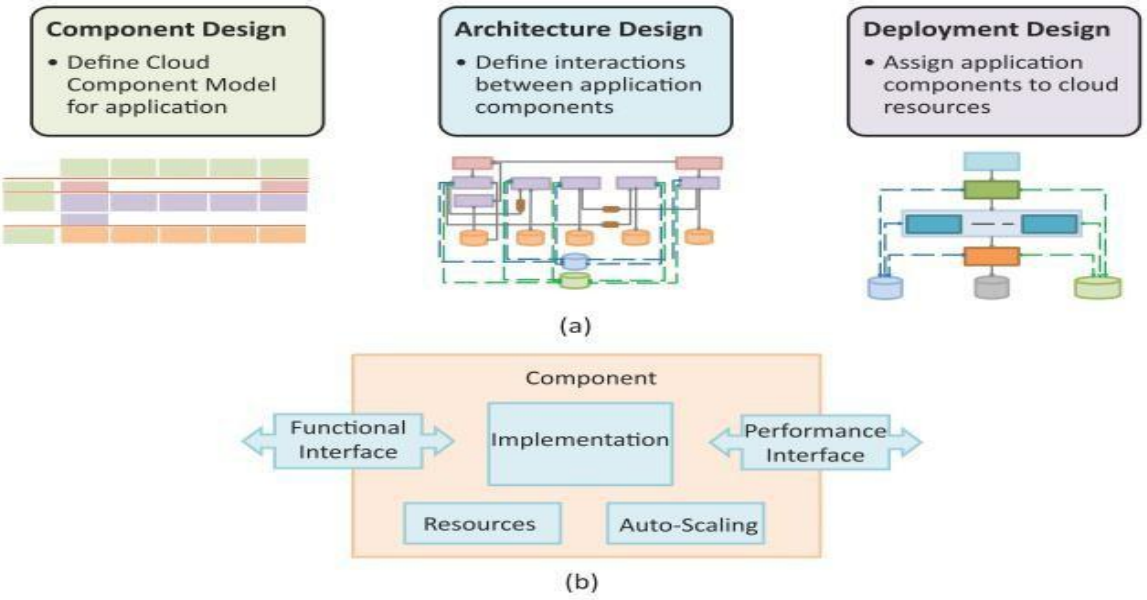


Cloud Component Model:

- Cloud Component Model is an application design methodology that provides a flexible way of creating cloud applications in a rapid, convenient and platform independent manner.
- CCM is an architectural approach for cloud applications that is not tied to any specific programming language or cloud platform.
- Cloud applications designed with CCM approach can have innovative hybrid deployments in which different components of an application can be deployed on cloud infrastructure and platforms of different cloud vendors.
- Applications designed using CCM have better portability and interoperability.
- CCM based applications have better scalability by decoupling application components and providing asynchronous communication mechanisms.

CCM Application Design Methodology:

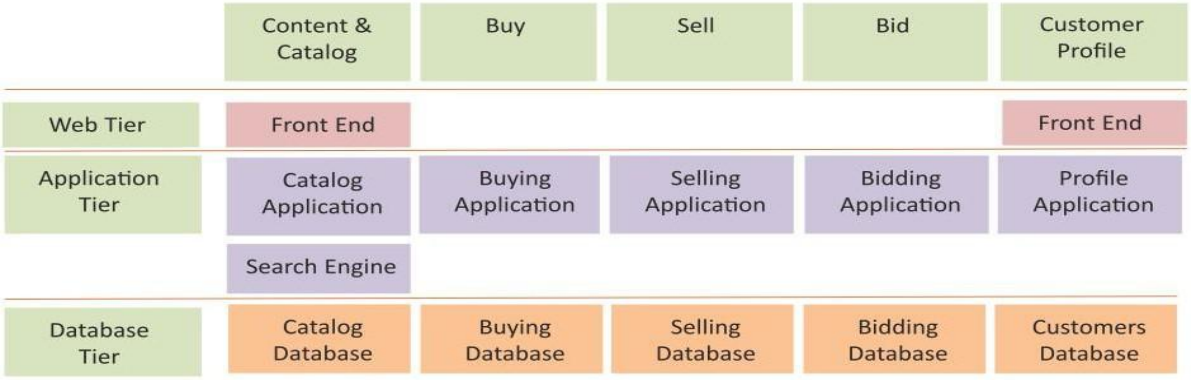
- CCM approach for application design involves:
  - Component Design
  - Architecture Design
  - Deployment Design



CCM Component Design:

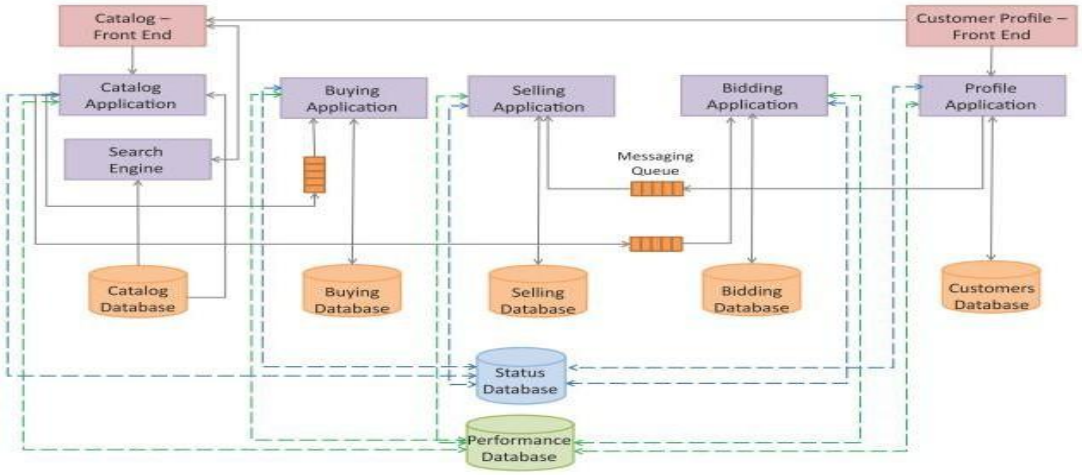
- Cloud Component Model is created for the application based on comprehensive analysis of the application's functions and building blocks.
- Cloud component model allows identifying the building blocks of a cloud application which are classified based on the functions performed and type of cloud resources required.
- Each building block performs a set of actions to produce the desired outputs for other components.

- Each component takes specific inputs, performs a pre- defined set of actions and produces the desired outputs.
- Components offer their functions as services through a functional interface which can be used by other components.
- Components report their performance to a performance database through a performance interface.



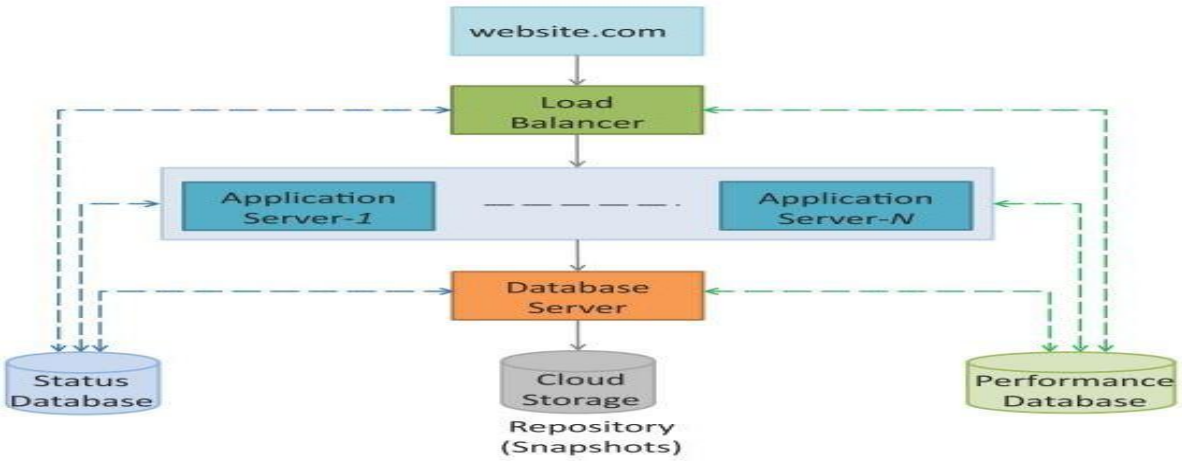
CCM Architecture Design:

- In Architecture Design step, interactions between the application components are defined.
- CCM components have the following characteristics:
  - Loose Coupling
  - Components in the Cloud Component Model are loosely coupled.
- Asynchronous Communication
  - By allowing asynchronous communication between components, it is possible to add capacity by adding additional servers when the application load increases. Asynchronous communication is made possible by using messaging queues.
- Stateless Design
  - Components in the Cloud Component Model are stateless. By storing session state outside of the component (e.g. in a database), stateless component design enables distribution and horizontal scaling.



CCM Deployment Design:

- In Deployment Design step, application components are mapped to specific cloud resources such as web servers, application servers, database servers, etc.
- Since the application components are designed to be loosely coupled and stateless with asynchronous communication, components can be deployed independently of each other.
- This approach makes it easy to migrate application components from one cloud to the other.
- With this fiexibility in application design and deployment, the application developers can ensure that the applications meet the performance and cost requirements with changing contexts.



SOA vs CCM:

Similarities

	SO	CC
Standardization & Re-use	SOA advocates principles of reuse and well defined used relationship between service applications. provider and service	CCM is based on reusable components which can be consumer.by multiple cloud
Loose	SOA is based on loosely coupled services that minimize dependencies.	CCM is based on loosely coupled components that communicate
Statelessnes	SOA services minimize resource consumption by deferring the the management of state information.	CCM components are State is stored outside of components.

Difference

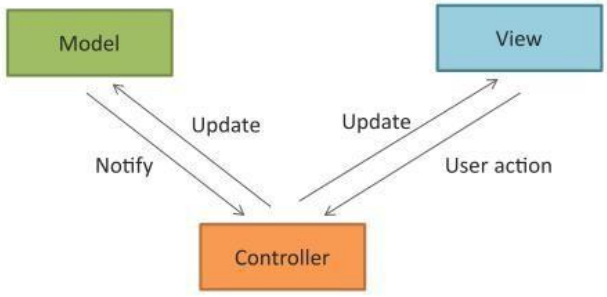
	SOA	CCM
End points	SOA services have small and well-defined set of endpoints through which many types of data can pass.	CCM components have very large number of endpoints. There is an endpoint for each resource in a component, identified by a URI.
Messaging	SOA uses a messaging layer above HTTP by using SOAP which provide prohibitive constraints to developers	CCM components use HTTP and REST for messaging.
Security	Uses WS-Security , SAML and other standards for security	CCM components use HTTPS for security
Interfacing	SOA uses XML for interfacing.	CCM allows resources in components represent different formats for interfacing (HTML, XML, JSON, etc.).
Consumption	Consuming traditional SOA services in a browser is cumbersome.	CCM components and the underlying component resources are exposed as XML, JSON (and other formats) over HTTP or REST, thus easy to consume in the browser.

Model View Controller:

- Model View Controller (MVC) is a popular software design pattern for web applications.

- **Model**
  - Model manages the data and the behavior of the applications. Model processes events sent by the controller. Model has no information about the views and controllers. Model responds to the requests for information about its state (from the view) and responds to the instructions to change state (from controller).
- **View**
  - View prepares the interface which is shown to the user. Users interact with the application through views. Views present the information that the model or controller tell the view to present to the user and also handle user requests and sends them to the controller.

- **Controller**
    - Controller glues the model to the view. Controller processes user requests and updates the model when the user manipulate



RESTful Web Services:

- Representational State Transfer (REST) is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred.
- The REST architectural constraints apply to the components, connectors, and data elements, within a distributed hypermedia system.
- A RESTful web service is a web API implemented using HTTP and REST principles.
- The REST architectural constraints are as follows:
    - Client-Server
    - Stateless
    - Cacheable
    - Layered System
    - Uniform Interface
    - Code on demand

Data Storage Approaches:

Relational Databases:

- A relational database is database that conforms to the relational model that was popularized by Edgar Codd in 1970.
- The 12 rules that Codd introduced for relational databases include:

1. Information rule
2. Guaranteed access rule
3. Systematic treatment of null values
4. Dynamic online catalog based on relational model
5. Comprehensive sub-language rule
6. View updating rule
7. High level insert, update, delete
8. Physical data independence
9. Logical data independence
10. Integrity independence
11. Distribution independence
12. Non-subversion rule.

Relations: A relational database has a collection of relations (or tables). A relation is a set of tuples (or rows).

Schema: Each relation has a fixed schema that defines the set of attributes (or columns in a table) and the constraints on the attributes.

Tuples: Each tuple in a relation has the same attributes (columns). The tuples in a relation can have any order and the relation is not sensitive to the ordering of the tuples.

Attributes: Each attribute has a domain, which is the set of possible values for the attribute.

Insert/Update/Delete: Relations can be modified using insert, update and delete operations. Every relation has a primary key that uniquely identifies each tuple in the relation.

Primary Key: An attribute can be made a primary key if it does not have repeated values in different tuples.

ACID Guarantees:

Relational databases provide ACID guarantees.

- **Atomicity:** Atomicity property ensures that each transaction is either "all or nothing". An atomic transaction ensures that all parts of the transaction complete or the database state is left unchanged.

- **Consistency:** Consistency property ensures that each transaction brings the database from one valid state to another. In other words, the data in a database always conforms to the defined schema and constraints.

- **Isolation:** Isolation property ensures that the database state obtained after a set of concurrent transactions is the same as would have been if the transactions were executed serially. This provides concurrency control, i.e. the results of incomplete transactions are not visible to other transactions. The transactions are isolated from each other until they finish.

- **Durability:** Durability property ensures that once a transaction is committed, the data remains as it is, i.e. it is not affected by system outages such as power loss. Durability guarantees that the database can keep track of changes and can recover from abnormal terminations.

Non-Relational Databases

- Non-relational databases (or popularly called No-SQL databases) are becoming popular with the growth of cloud computing.
- Non-relational databases have better horizontal scaling capability and improved performance for big data at the cost of less rigorous consistency models.
- Unlike relational databases, non-relational databases do not provide ACID guarantees.
- Most non-relational databases offer "eventual" consistency, which means that given a sufficiently long period of time over which no updates are made, all updates can be expected to propagate eventually through the system and the replicas will be consistent.
- The driving force behind the non-relational databases is the need for databases that can achieve high scalability, fault tolerance and availability.
- These databases can be distributed on a large cluster of machines. Fault tolerance is provided by storing multiple replicas of data on different machines.

Non-Relational Databases – Types

- **Key-value store:** Key-value store databases are suited for applications that require storing unstructured data without a fixed schema. Most key-value stores have support for native programming language data types.

- **Document store:** Document store databases store semi-structured data in the form of documents which are encoded in different standards such as JSON, XML, BSON, YAML, etc.

- **Graph store:** Graph stores are designed for storing data that has graph structure (nodes and edges). These solutions are suitable for applications that involve graph data such as social networks, transportation systems, etc.

- **Object store:** Object store solutions are designed for storing data in the form of objects defined in an object-oriented programming language.

Python Basics:

Python is a general-purpose high level programming language and suitable for providing a solid foundation to there ader in the area of cloud computing.

Them a in characteristics of Python are:

Multi-paradigm programming language

Python supports more than one programming paradigms including object-oriented programming and structured programming

Interpreted Language

Python is an interpreted language and does not require an explicit compilation step. The Python interpreter executes the program source code directly, statement by statement, as a processor or scripting engine does.

Interactive Language

Python provides an interactive mode in which the user can submit command sat the Python prompt and interact with the interpreter directly.

Python – Benefits

- **Easy-to-learn, read and maintain**
  - Python is a minimalistic language with relatively few keywords, uses English keywords and has fewer syntactical constructions as compared to other languages. Reading Python programs feels like English with pseudo-code like constructs. Python is easy to learn yet an extremely powerful language for a wide range of applications.
- **Object and Procedure Oriented**
  - Python supports both procedure-oriented programming and object-oriented programming. Procedure oriented paradigm allows programs to be written around procedures or functions that allow reuseof code. Procedure oriented paradigm allows programs to be written around objects that include both data and functionality.
- **Extendable**
  - Python is an extendable language and allows integration of low-level modules written in languages such as C/C++. This is useful when you want to speed up a critical portion of a program.
- **Scalable**
  - Due to the minimalistic nature of Python, it provides a manageable structure for large programs.
- **Portable**
  - Since Python is an interpreted language, programmers do not have to worry about compilation, linking and loading of programs. Python programs can be directly executed from source
- **Broad Library Support**
  - Python has a broad library support and works on various platforms such as Windows, Linux, Mac, etc.

Python – Setup

- Windows
    - Python binaries for Windows can be downloaded from http://www.python.org/getit .
    - For the examples and exercise in this book, you would require Python 2.7 which can be directly downloaded from: http://www.python.org/ftp/python/2.7.5/python-2.7.5.msi
    - Once the python binary is installed you can run the python shell at the command prompt using > python
    - Linux

#Install Dependencies
sudo apt-get install build-essential
sudo apt-get install libreadline-gplv2-dev libncursesw5-dev libssl-dev libsqlite3-devtk-dev libgdbm-dev libc6-dev libbz2-dev
#Download Python
wget http://python.org/ftp/python/2.7.5/Python-2.7.5.tgz tar -xvf Python-2.7.5.tgz
cd Python-2.7.5
#Install Python
./configure make
sudo make install

Numbers

- Numbers
    - Number data type is used to store numeric values. Numbers are immutable data types, therefore changing the value of a number data type results in a newly allocated object.

Unit-3

Python for Cloud: Python for Amazon web services, Python for Google Cloud Platform, Python for windows Azure, Python for Map Reduce, Python packages of Interest, Python web Application Framework, Designing a REST full web API.

Cloud Application Development in Python: Design Approaches, Image Processing APP, Document Storage App, MapReduce App, Social Media Analytics App.

Python for Amazon web services

AmazonEC2–Python Example:

• Boto is a Python package that provides interfaces to Amazon Web Services (AWS)

• In this example, a connection to EC2 service is first established by calling boto.ec2.connect_to_region

• TheEC2 region, AWS access key and AWS secret key are passed to this function. After connecting to EC2, a new instance is launched using the conn.run_instances function.

• The AMI-ID, instance type, EC2 key handle and security group are passed to this function

#Python program for launching an EC2 instance

```
importboto.ec2
 from time import sleep
ACCESS_KEY="<enter access key>"
SECRET_KEY="<entersecretkey>"
 REGION="us-east-1" AMI_ID="ami
d0f89fb9"
 EC2_KEY_HANDLE = "<enter key handle>"
INSTANCE_TYPE="t1.micro"
 SECGROUP_HANDLE="default"
 conn=boto.ec2.connect_to_region(REGION,aws_access_key_id=ACCESS_KEY,
 aws_secret_access_key=SECRET_KEY)
 reservation=conn.run_instances(image_id=AMI_ID,key_name=EC2_KEY_HANDLE,
```

```
 instance_type=INSTANCE_TYPE,
security_groups=[SECGROUP_HANDLE,])
```

Amazon Auto Scaling–Python Example:

Auto Scaling Service

A connection to Auto Scaling service is first established by calling the boto.ec2.autoscale.connect_to_region function.

Launch Configuration

After connecting to Auto Scaling service, a new launch configuration is created by calling conn.create_launch_configuration.

Launch configuration contains instructions on how to launch new instances, including the AMI-ID, instance type, security groups, etc.

Auto Scaling Group

After creating a launch configuration, it is then associated with a new Auto Scaling group.

Auto Scaling group is created by calling conn.create_auto_scaling_group.

The settings for Auto Scaling group such as the maximum and minimum number of instances in the group, the launch configuration, availability zones, optional load balancer to use with the group, etc.

#Python program for creating an Auto Scaling group (code excerpt)

```
importboto.ec2.autoscale :
 print"Connectingto AutoscalingService"
 conn = boto.ec2.autoscale.connect_to_region(REGION,
aws_access_key_id=ACCESS_KEY,
aws_secret_access_key=SECRET_KEY)
 print"Creatinglaunchconfiguration"
 lc =LaunchConfiguration(name='My-Launch-Config-2',
 image_id=AMI_ID,
key_name=EC2_KEY_HANDLE,
instance_type=INSTANCE_TYPE,
security_groups=[SECGROUP_HANDLE,])
```

```
 conn.create_launch_configuration(lc)
 print"Creatingauto-scalinggroup"
 ag = AutoScalingGroup(group_name='My-Group',
availability_zones=['us-east-1b'],
launch_config=lc, min_size=1,max_size=2,
connection=conn)
 conn.create_auto_scaling_group(ag)
```

Amazon Auto Scaling–Python Example:

Auto Scaling Policies

- After creating an Auto Scaling group, the policies for scaling up and scaling down are defined.
- In this example, a scale-up policy with adjustment type ChangeInCapacity and scaling_adjustment=1 is defined.
- Similarly, a scale-down policy with adjustment type ChangeInCapacity and scaling_adjustment=-1 is defined.

#Creatingauto-scalingpolicies

```
scale_up_policy = ScalingPolicy(
    name='scale_up',
    adjustment_type='ChangeInCapacity',
    as_name='My-Group',
    scaling_adjustment=1,
    cooldown=180
)
scale_down_policy = ScalingPolicy(
    name='scale_down',
    adjustment_type='ChangeInCapacity',
    as_name='My-Group',
    scaling_adjustment=-1,
```

```
    cooldown=180
)
conn.create_scaling_policy(scale_up_policy)
conn.create_scaling_policy(scale_down_policy)
```

Amazon Auto Scaling–Python Example:

Cloud Watch Alarms

- With the scaling policies defined, the next step is to create Amazon CloudWatch alarms that trigger these policies.
- The scale-up alarm is defined using the CPU Utilization metric with the Average statistic and a threshold greater than 70% for a period of 60 seconds. The scale-up policy created previously is associated with this alarm. This alarm is triggered when the average CPU utilization of the instances in the group becomes greater than 70% for more than 60 seconds.
- The scale-down alarm is defined in a similar manner with a threshold less than 50%.

```
cloudwatch = boto.ec2.cloudwatch.connect_to_region(
    REGION,
    aws_access_key_id=ACCESS_KEY,
    aws_secret_access_key=SECRET_KEY
)

alarm_dimensions = {"AutoScalingGroupName": 'My-Group'}

scale_up_alarm = MetricAlarm(
    name='scale_up_on_cpu',
    namespace='AWS/EC2',
    metric='CPUUtilization',
    statistic='Average',
    comparison='>',
    threshold='70',
    period='60',
```

```
    evaluation_periods=2,
    alarm_actions=[scale_up_policy.policy_arn],
    dimensions=alarm_dimensions
)
cloudwatch.create_alarm(scale_up_alarm)

scale_down_alarm = MetricAlarm(
    name='scale_down_on_cpu',
    namespace='AWS/EC2',
    metric='CPUUtilization',
    statistic='Average',
    comparison='<',
    threshold='40',
    period='60',
    evaluation_periods=2,
    alarm_actions=[scale_down_policy.policy_arn],
    dimensions=alarm_dimensions
)
cloudwatch.create_alarm(scale_down_alarm)
```

Amazon S3 – Python Example;

- In this example, a connection to the S3 service is first established by calling the boto.connect_s3 function.
- The upload_to_s3_bucket_path function uploads the file to the S3 bucket specified at the given path.

Python program for uploading a file to an S3bucket

```
import boto.s3
conn = boto.connect_s3(
    aws_access_key_id='<enter>',
    aws_secret_access_key='<enter>'
```

```
)

def percent_cb(complete, total):
    print('.')

def upload_to_s3_bucket_path(bucketname, path, filename):
    mybucket = conn.get_bucket(bucketname)
    fullkeyname = os.path.join(path, filename)
    key = mybucket.new_key(fullkeyname)
    key.set_contents_from_filename(filename, cb=percent_cb, num_cb=10)
```

Amazon RDS – Python Example;

• In this example, a connection to RDS (Relational Database Service) service is first established by calling the boto.rds.connect_to_region function.

• The RDS region, AWS access key, and AWS secret key are passed to this function.

• After connecting to RDS service, the conn.create_dbinstance function is called to launch a new RDS instance.

• The input parameters to this function include the instance ID, database size, instance type, database username, database password, database port, database engine (e.g., MySQL 5.1), database name, security groups, etc.

Python program for launching an RDS instance (excerpt)

```
import boto.rds

ACCESS_KEY = "<enter>"
SECRET_KEY = "<enter>"
REGION = "us-east-1"
INSTANCE_TYPE = "db.t1.micro"
```

```
ID = "MySQL-db-instance-3"
USERNAME = 'root'
PASSWORD = 'password'
DB_PORT = 3306
DB_SIZE = 5
DB_ENGINE = 'MySQL5.1'
DB_NAME = 'mytestdb'
SECGROUP_HANDLE = "default"

# Connecting to RDS
conn = boto.rds.connect_to_region(
   REGION,
   aws_access_key_id=ACCESS_KEY,
   aws_secret_access_key=SECRET_KEY
)

# Creating an RDS instance
db = conn.create_dbinstance(
   ID,
   DB_SIZE,
   INSTANCE_TYPE,
   USERNAME,
   PASSWORD,
   port=DB_PORT,
   engine=DB_ENGINE,
   db_name=DB_NAME,
   security_groups=[SECGROUP_HANDLE]
)
```

Amazon Dynamo DB – Python Example;

• In this example, a connection to DynamoDB service is first established by calling boto.dynamodb.connect_to_region.

• After connecting to DynamoDB service, a schema for the new table is created by calling conn.create_schema.

• The schema includes the hash key and range key names and types.

• A DynamoDB table is then created by calling the conn.create_table function with the table schema, read units, and write units as input parameters.

Python program for creating a DynamoDB table (excerpt)

```
import boto.dynamodb

ACCESS_KEY = "<enter>"
SECRET_KEY = "<enter>"
REGION = "us-east-1"

# Connecting to DynamoDB
conn = boto.dynamodb.connect_to_region(
    REGION,
    aws_access_key_id=ACCESS_KEY,
    aws_secret_access_key=SECRET_KEY
)
table_schema = conn.create_schema(
    hash_key_name='msgid',
    hash_key_proto_value=str,
    range_key_name='date',
    range_key_proto_value=str
)
```

Creating table with schema

```
table = conn.create_table(
    name='my-test-table',
    schema=table_schema,
    read_units=1,
    write_units=1
)
```

Google Compute Engine–PythonExample;

• This example uses the OAuth 2.0 scope (https://www.googleapis.com/auth/compute) and credentials in the credentials file to request a refresh and access token, which is then stored in the oauth2.dat file.

• After completing the OAuth authorization, an instance of the Google Compute Engine service is obtained.

• To launch a new instance, the instances().insert method of the Google Compute Engine API is used.

• The request body to this method contains the properties such as instance name, machine type, zone, network interfaces, etc., specified in JSON format.

```
# Python program for launching a GCE instance (excerpt)
API_VERSION = 'v1beta15'
GCE_SCOPE = 'https://www.googleapis.com/auth/compute'
GCE_URL = 'https://www.googleapis.com/compute/%s/projects/' % (API_VERSION)
DEFAULT_ZONE = 'us-central1-b'
CLIENT_SECRETS = 'client_secrets.json'
OAUTH2_STORAGE = 'oauth2.dat'
def main():
    # OAuth 2.0 authorization.
```

```
flow = flow_from_clientsecrets(CLIENT_SECRETS, scope=GCE_SCOPE)
storage = Storage(OAUTH2_STORAGE)
credentials = storage.get()
if credentials is None or credentials.invalid:
    credentials = run(flow, storage)
http = httplib2.Http()
auth_http = credentials.authorize(http)
gce_service = build('compute', API_VERSION)
# Create the instance
request = gce_service.instances().insert(
    project=PROJECT_ID,
    body=instance,
    zone=DEFAULT_ZONE
)
response = request.execute(auth_http)
```

Google Cloud Storage –Python Example :

• This example uses the OAuth 2.0 scope (https://www.googleapis.com/auth/devstorage.full_control) and credentials in the credentials file to request a refresh and access token, which is then stored in the oauth2.dat file.

• After completing the OAuth authorization, an instance of the Google Cloud Storage service is obtained.

• To upload a file, the objects().insert method of the Google Cloud Storage API is used.

• The request to this method contains the bucket name, file name, and media body, which includes the MediaIo Base Upload object created from the file contents.

```
def main():
    flow = flow_from_clientsecrets(CLIENT_SECRETS, scope=GS_SCOPE)
```

```
    storage = Storage(OAUTH2_STORAGE)
    credentials = storage.get()
    if credentials is None or credentials.invalid:
        credentials = run(flow, storage)
    http = httplib2.Http()
    auth_http = credentials.authorize(http)
    gs_service = build('storage', API_VERSION, http=auth_http)
    fp = open(FILENAME, 'r')
    fh = io.BytesIO(fp.read())
    media = MediaIoBaseUpload(fh, FILE_TYPE)
    request = gs_service.objects().insert(bucket=BUCKET, name=FILENAME,
media_body=media)
    response = request.execute()
```

Google Cloud SQL–Python Example:

• This example uses the OAuth 2.0 scope (https://www.googleapis.com/auth/compute) and credentials in the credentials file to request a refresh and access token, which is then stored in the oauth2.dat file.

• After completing the OAuth authorization, an instance of the Google Cloud SQL service is obtained.

• To launch a new instance, the instances().insert method of the Google Cloud SQL API is used.

• The request body of this method contains properties such as instance, project, tier, pricing plan, and replication type.

Python program for launching a Google Cloud SQL instance (excerpt)

```
def main():
    flow = flow_from_clientsecrets(CLIENT_SECRETS, scope=GS_SCOPE)
    storage = Storage(OAUTH2_STORAGE)
    credentials = storage.get()
    if credentials is None or credentials.invalid:
```

```
    credentials = run(flow, storage)
http = httplib2.Http()
auth_http = credentials.authorize(http)
gcs_service = build('sqladmin', API_VERSION, http=auth_http)
instance = {
    "instance": "mydb",
    "project": "bahgacloud",
    "settings": {
        "tier": "D0",
        "pricingPlan": "PER_USE",
        "replicationType": "SYNCHRONOUS"
    }
}
request = gcs_service.instances().insert(project=PROJECT_ID, body=instance)
response = request.execute()
```

Azure Virtual Machines–Python Example :

• To create a virtual machine, a cloud service is first created.

• A virtual machine is created using the create_virtual_machine_deployment method of the Azure Service Management API.

Python program for launching an Azure VM instance (excerpt)

```
from azure import *
sms = ServiceManagementService(subscription_id, certificate_path)
name = '<enter>'
location = 'WestUS'
# Name of an OS image as returned by list_os_images
```

```
image_name = '<enter>'
# Destination storage account container/blob where the VM disk will be created
media_link = '<enter>'
# Linux VM configuration
linux_config = LinuxConfigurationSet('bahga', 'arshdeepbahga', 'Arsh~2483', True)
os_hd = OSVirtualHardDisk(image_name, media_link)
# Create instance
sms.create_virtual_machine_deployment(
    service_name=name,
    deployment_name=name,
    deployment_slot='production',
    label=name,
    role_name=name,
    system_config=linux_config,
    os_virtual_hard_disk=os_hd,
    role_size='Small')
```

Azure Storage–Python Example:

• Azure Blob service allows you to store large amounts of unstructured text or binary data, such as video, audio, and images.

• This example demonstrates how to use the Blob service to store a file.

• Blobs are organized into containers. The create_container method is used to create a new container.

• After creating a container, the blob is uploaded using the put_blob method.

• Blobs can be listed using the list_blobs method.

• To download a blob, the get_blob method is used.

```
# Python example of using Azure Blob Service (excerpt)
from azure.storage import *
blob_service = BlobService(account_name='<enter>', account_key='<enter>')
# Create Container
blob_service.create_container('mycontainer')
# Upload Blob
filename = 'images.txt'
myblob = open(filename, 'r').read()
blob_service.put_blob('mycontainer', filename, myblob, x_ms_blob_type='BlockBlob')
# List Blobs
blobs = blob_service.list_blobs('mycontainer')
for blob in blobs:
    print(blob.name)
    print(blob.url)
# Download Blob
output_filename = 'output.txt'
blob = blob_service.get_blob('mycontainer', 'myblob')
with open(output_filename, 'w') as f:
    f.write(blob)
```

Python for Map Reduce:

The example shows inverted index mapper program.

•The map function reads the data from the standard input (stdin) and splits the tab-limited data into document-ID and contents of the document.

•Themapfunctionemitskey-valuepairswherekeyiseachwordinthedocumentandvalue is the document-ID.

```
# Inverted Index Mapper in Python
#!/usr/bin/env python
import sys
for line in sys.stdin:
    doc_id, content = line.split(' ', 1)  # Split only on the first space
    words = content.split()
     for word in words:
        print('%s\t%s' % (word, doc_id))
```

Python for MapReduce:

• This example demonstrates an inverted index reducer program.

• The key-value pairs emitted by the map phase are shuffled to the reducers and grouped by the key.

• The reducer reads the key-value pairs grouped by the same key from the standard input (stdin) and creates a list of document IDs where the word occurs.

• The output of the reducer contains key-value pairs, where the key is a unique word and the value is the list of document IDs in which the word occurs.

```
# Inverted Index Reducer in Python
#!/usr/bin/env python
import sys
current_word = None
current_docids = []
word = None
for line in sys.stdin:
    # Remove leading and trailing whitespace
    line = line.strip()
```

```
# Parse the input received from the mapper
    word, doc_id = line.split(' ', 1)
        if current_word == word:
        current_docids.append(doc_id)
    else:
        if current_word:
            print('%s\t%s' % (current_word, current_docids))
        current_docids = [doc_id]
        current_word = word
# Print the last word and its document IDs
if current_word:
    print('%s\t%s' % (current_word, current_docids))
```

Python Packages of Interest :

• **JSON**
JavaScript Object Notation (JSON) is an easy-to-read and write data-interchange format. JSON is used as an alternative to XML and is easy for machines to parse and generate. JSON is built on two structures: a collection of name-value pairs (e.g., a Python dictionary) and ordered lists of values (e.g., a Python list).

• **XML**
Extensible Markup Language (XML) is a data format for structured document interchange. The Python minidom library provides a minimal implementation of the Document Object Model interface and has an API similar to that in other languages.

• **HTTPLib & URLLib**
httplib2 and urllib2 are Python libraries used for network and internet programming.

• **SMTPLib**
Simple Mail Transfer Protocol (SMTP) is a protocol for sending email and routing emails between mail servers. The Python smtplib module provides an SMTP client session object that can be used to send emails.

• **NumPy**
NumPy is a package for scientific computing in Python. It provides support for large multi-dimensional arrays and matrices.

• **Scikit-learn**
Scikit-learn is an open-source machine learning library for Python that provides implementations of various machine learning algorithms for classification, clustering, regression, and dimension reduction problems.

Python Web Application Framework – Django:

• Django is an open-source web application framework for developing web applications in Python.

• A web application framework is a collection of solutions, packages, and best practices that facilitate the development of web applications and dynamic websites.

• Django is based on the Model-Template-View (MTV) architecture, which separates the data model, business logic, and user interface.

• Django provides a unified API for interacting with a database backend.

• Web applications built with Django can work with different databases without requiring any code changes.

• With its flexibility in web application design, combined with the powerful capabilities of Python and its ecosystem, Django is well-suited for cloud applications.

• Django consists of an Object-Relational Mapper (ORM), a web templating system, and a regular-expression-based URL dispatcher.

Django Architecture:

• Django is a Model-Template-View (MTV) framework.

Model

• The model defines the structure of stored data and handles database interactions.
• In a web application, data can be stored in a relational database, non-relational database, XML file, etc.
• A Django model is a Python class that outlines the variables and methods for a particular type of data.

Template

• In a typical Django web application, the template is an HTML page with additional placeholders.
• Django's template language can be used to create various types of text-based files, such as XML, email templates, CSS, JavaScript, and CSV.

View

• The view connects the model to the template.
• The view contains the logic to generate web pages dynamically.
• It determines what data needs to be displayed, retrieves it from the database, and passes it to the template for rendering.

Django Setup on Amazon EC2:



Cloud Application Development in Python

Designing a REST full web API:

In Chapter 5 you learned about the characteristics of Representational State Transfer (REST) systems and RESTful web APIs. In this section you will learn how to develop a RESTful web API. The example in this section uses the Django REST framework [33] for building a REST

API. With the Django framework already installed, the Django REST framework can be installed as follows:

pip install Django rest framework

pip install markdown

pip install django-filter

After installing the Django REST framework, create a new Django project named rest fulapi, and then start a new app called myapp, as follows:

django-admin.py startproject restfulapi

cd restfulapi

python manage.py startapp myapp

The REST API described in this section allows you to create, view, update and delete a collection of resources where each resource represents a Book object. Box 7.74 shows the Django model for a Book. The Book model contains four fields - book name, ISBN number, publication date and publisher name. Box 7.75 shows the Django views for the REST API. ViewSets are used for the views that allow you to combine the logic for a set of related views in a single class.

Box 7.76 shows the serializers for the REST API. Serializers allow complex data (such as query sets and model instances) to be converted to native Python datatypes that can then be easily rendered into JSON, XML or other content types. Serializers also provide deserialization, allowing parsed data to be converted back into complex types, after first validating the incoming data.

Box 7.77 shows the URL patterns for the REST API. Since View Sets are used instead of views, we can automatically generate the URL conf for our API, by simply registering the view sets with a router class. Routers automatically determining how the URLs for an application should be mapped to the logic that deals with handling incoming requests.

Box 7.78 shows the settings for the REST API Django project.

Box 7.74: Django model for Book - models.py

```
from django.db import models

class Book(models.Model):
    name = models.CharField(max_length=50)
    isbn = models.CharField(max_length=50)
    pub_date = models.DateField()
    pub_name = models.CharField(max_length=50)
```

Box 7.75: Django views for Book REST API - views.py

```
from myapp.models import Book
from rest_framework import viewsets
from myapp.serializers import BookSerializer

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

Box 7.76: Serializers for Book REST API- serializers.py

```
from myapp.models import Book
from rest_framework import serializers

class BookSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Book
        fields = ('url', 'name', 'isbn', 'pub_date', 'pub_name')
```

Box 7.77: Django URL patterns for Book REST API - urls.py

```
from django.conf.urls import patterns, include, url
from django.contrib import admin
admin.autodiscover()
from rest_framework import routers
from myapp import views
```

```
router = routers.DefaultRouter()
router.register(r'books', views.BookViewSet)

# Wire up our API using automatic URL routing.
# Additionally, we include login URLs for the browseable API.
urlpatterns = patterns('',
    url(r'^', include(router.urls)),
    url(r'^api-auth/', include('rest_framework.urls', namespace='rest_framework')),
    url(r'^admin/', include(admin.site.urls)),
)
```

■ Box 7.78: Django project settings for Book REST API project - settings.py

```
DATABASES =
    'default':
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),


REST_FRAMEWORK =
    'DEFAULT_PERMISSION_CLASSES': ('rest_framework.permissions.IsAdminUser',),
    'PAGINATE_BY': 10


INSTALLED_APPS = (
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp',
    'rest_framework',
)
```

After creating the Books REST API source files, the next step is to setup the database and then run the Django development web server as follows:

python manage.py syncdb

python manage.py runserver

■ Box 7.79: Using the Book REST API - CURL examples

```
#----------POST Example----------

$curl -i -H "Content-Type: application/json" -H "Accept: application/json; indent=4"
-X POST -d '{"name":"Cloud Computing","isbn":"989-1-9881818","pub_date":"2013-12-04",
"pub_name":"XYZ Publishers"}' -u arshdeep http://127.0.0.1:8000/books/
```

```
HTTP/1.0 201 CREATED
Server: WSGIServer/0.1 Python/2.7.3
Vary: Accept, Cookie
X-Frame-Options: SAMEORIGIN
Content-Type: application/json; indent=4
Location: http://127.0.0.1:8000/books/1/
Allow: GET, POST, HEAD, OPTIONS

    "url": "http://127.0.0.1:8000/books/1/",
    "name": "Cloud Computing",
    "isbn": "989-1-9881818",
    "pub_date": "2013-12-04",
    "pub_name": "XYZ Publishers"


#———GET Examples———

$curl -I -H "Accept: application/json; indent=4" -u arshdeep http://127.0.0.1:8000/books/

HTTP/1.0 200 OK
Server: WSGIServer/0.1 Python/2.7.3
Vary: Accept, Cookie
X-Frame-Options: SAMEORIGIN
Content-Type: application/json; indent=4
Allow: GET, POST, HEAD, OPTIONS

    "count": 2,
    "next": null,
    "previous": null,
    "results": [

        {
            "url": "http://127.0.0.1:8000/books/1/",
            "name": "Cloud Computing",
            "isbn": "989-1-9881818",
            "pub_date": "2013-12-04",
            "pub_name": "XYZ Publishers"
        },

        {
            "url": "http://127.0.0.1:8000/books/2/",
            "name": "Database Systems",
            "isbn": "981-3-8128989",
            "pub_date": "2013-02-14",
            "pub_name": "ABC Publishers"
        }
    ]


$curl -I -H "Accept: application/json; indent=4" -u arshdeep http://127.0.0.1:8000/books/1/

HTTP/1.0 200 OK
Server: WSGIServer/0.1 Python/2.7.3
```

```
Vary: Accept, Cookie
X-Frame-Options: SAMEORIGIN
Content-Type: application/json; indent=4
Allow: GET, PUT, PATCH, DELETE, HEAD, OPTIONS

    "url": "http://127.0.0.1:8000/books/1/",
    "name": "Cloud Computing",
    "isbn": "989-1-9881818",
    "pub_date": "2013-12-04",
    "pub_name": "XYZ Publishers"


#------PUT Example-------

$curl -i -H "Content-Type: application/json" -H "Accep: application/json; indent=4"
-X PUT -d '"name":"Cloud Computing: Concepts & Technologies", "isbn":"989-1-9881818",
"pub_date":"2013-12-04","pub_name":"XYZ Publishers"'
-u arshdeep http://127.0.0.1:8000/books/1/

HTTP/1.0 200 OK
Server: WSGIServer/0.1 Python/2.7.3
Vary: Accept, Cookie
X-Frame-Options: SAMEORIGIN
Content-Type: application/json; indent=4
Allow: GET, PUT, PATCH, DELETE, HEAD, OPTIONS

    "url": "http://127.0.0.1:8000/books/1/",
    "name": "Cloud Computing: Concepts & Technologies",
    "isbn": "989-1-9881818",
    "pub_date": "2013-12-04",
    "pub_name": "XYZ Publishers"


#------DELETE Example-------

$curl -i -X DELETE -H "Accept: application/json; indent=4"
-u arshdeep http://127.0.0.1:8000/books/2/

HTTP/1.0 204 NO CONTENT
Server: WSGIServer/0.1 Python/2.7.3
Vary: Accept, Cookie
X-Frame-Options: SAMEORIGIN
Content-Type: application/json; indent=4
Content-Length: 0
Allow: GET, PUT, PATCH, DELETE, HEAD, OPTIONS
```

Box 7.79 shows examples of interacting with the Books REST API using CURL. The HTTP POST method is used to create a new resource, GET method is used to obtain information about a resource, PUT method is used to update a resource and DELETE method is used to delete a resource. Figures 7.11 and 7.12 show the screenshots from the web browsable Books REST API.

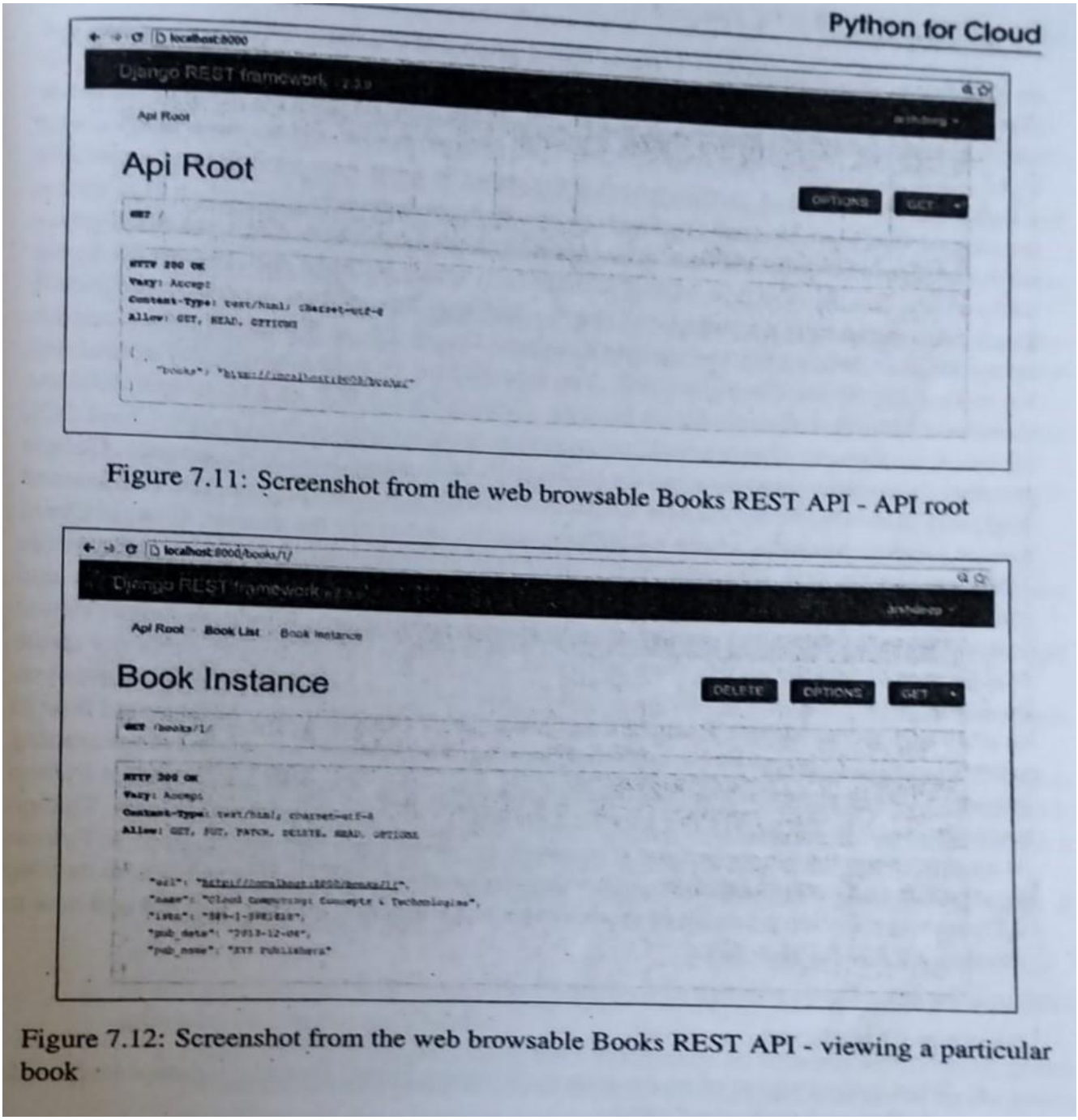


Figure 7.11: Screenshot from the web browsable Books REST API - API root

Figure 7.12: Screenshot from the web browsable Books REST API - viewing a particular book

1. **Design Approaches**
    - Overview of different cloud application design approaches
    - Key considerations for scalability, security, and performance
2. **Design Methodology for IaaS Service Model**
    - Infrastructure as a Service (IaaS) architecture
    - Deployment strategies and resource management
    - Virtualization and automation in IaaS
3. **Design Methodology for PaaS Service Model**
    - Platform as a Service (PaaS) architecture
    - Application development and deployment in PaaS
    - Integration with cloud services
4. **Cloud Application Case Studies**
    - Image Processing App
        - Use of cloud storage and compute resources for processing
        - Example workflow and implementation
    - Document Storage App
        - Cloud-based document management system
        - Security and access control considerations
    - MapReduce App
        - Distributed computing and data processing using MapReduce
        - Example use cases and implementation
    - Social Media Analytics App
        - Data collection, processing, and analysis in the cloud
        - Scalability and real-time analytics solutions

Design methodology for IaaS service model:

Component Design

1. **Identify the Building Blocks**
    - Determine the core functionalities of the application.
    - Define what each block is responsible for (e.g., data processing, storage, user authentication).
2. **Group the Building Blocks**
    - Categorize blocks based on their functions (e.g., UI, database, business logic).
    - Identify the cloud resources required for each group (e.g., compute, storage, networking).
    - Define application components based on these groupings.
3. **Identify Inputs and Outputs**
    - Determine the input data each component receives.
    - Define the output each component produces.
    - Establish data flow between components.
4. **List Component Interfaces**
    - Define the APIs or methods exposed by each component.
    - Specify how components will communicate with each other (e.g., REST APIs, message queues).
5. **Evaluate Implementation Alternatives**
    - Explore design patterns such as Model-View-Controller (MVC), Microservices, or Serverless architecture.
    - Compare different technology stacks and frameworks for implementation.
    - Consider factors like scalability, security, and performance in selecting alternatives.

Architecture Design:

1. **Define the Interactions Between Application Components**
    - Establish communication pathways between components (e.g., APIs, event-driven messaging).
    - Determine data flow and dependencies among services.
    - Identify synchronous vs. asynchronous communication requirements.
2. **Guidelines for Loosely Coupled and Stateless Designs**
    - Use messaging queues (e.g., Kafka, RabbitMQ, AWS SQS) for asynchronous communication.
    - Implement functional interfaces (e.g., REST APIs, GraphQL) to ensure loose coupling between components.
    - Store application state in an external database (e.g., Redis, DynamoDB) to maintain statelessness and enable scalability.
    - Consider serverless functions (e.g., AWS Lambda, Google Cloud Functions) for independent execution.

Deployment Design:

1. **Map Application Components to Specific Cloud Resources**
    - Web Servers → Serve static and dynamic content (e.g., Nginx, Apache, AWS CloudFront).
    - Application Servers → Process business logic (e.g., Node.js, Flask, Django, Spring Boot).
    - Database Servers → Store structured or unstructured data (e.g., MySQL, PostgreSQL, MongoDB).
    - Load Balancers → Distribute traffic across instances (e.g., AWS ELB, Nginx, HAProxy).
    - Storage Services → Handle file and object storage (e.g., AWS S3, Google Cloud Storage).
    - Container Orchestration → Manage microservices and scalability (e.g., Kubernetes, Docker Swarm).

2. **Considerations for Deployment**
    - Scalability: Auto-scaling groups, containerized deployment (e.g., Kubernetes, Docker).
    - Security: IAM roles, encryption, firewalls, API gateways.
    - Monitoring & Logging: Use tools like Prometheus, ELK Stack, CloudWatch, Stack driver.
    - High Availability: Deploy across multiple availability zones or regions.

Design methodology for IaaS service mode:

Component Design

• Identify the building blocks of the application and to be performed by each block

• Group the building blocks based on the functions performed and type of cloud resources required and identify the application components based on the groupings

• Identify the inputs and outputs of each component

• List the interfaces that each component will expose

• Evaluate the implementation alternatives for each component (design patterns such as MVC, etc.)

Architecture Design

• Define the interactions between the application components

• Guidelines for loosely coupled and stateless designs - use messaging queues (for asynchronous communication), functional interfaces (such as REST for loose coupling) and external status database (for clean stateless design)

Deployment Design

• Map the application components to specific cloud resources (such as web servers, application servers, database servers, etc.)

Design Methodology for PaaS Service Model

• For applications that use the Platform-as-a-Service (PaaS) cloud service model, the architecture and deployment design steps are not required since the platform takes care of the architecture and deployment.

• **Component Design**

• In the component design step, the developers have to take into consideration the platform-specific features.

• **Platform Specific Software**

• Different PaaS offerings such as Google App Engine, Windows Azure Web Sites, etc., provide platform-specific software development kits (SDKs) for developing cloud applications.

• **Sandbox Environments**

• Applications designed for specific PaaS offerings run in sandbox environments and are allowed to perform only those actions that do not interfere with the performance of other applications.

• **Deployment & Scaling**

• The deployment and scaling are handled by the platform while the developers focus on the application development using the platform-specific SDKs.

• **Portability**

• Portability is a major constraint for PaaS-based applications as it is difficult to move the

Image Processing App – Component Design

• **Functionality:**

• A cloud-based Image Processing application.

• This application provides online image filtering capability.

• Users can upload image files and choose the filters to apply.

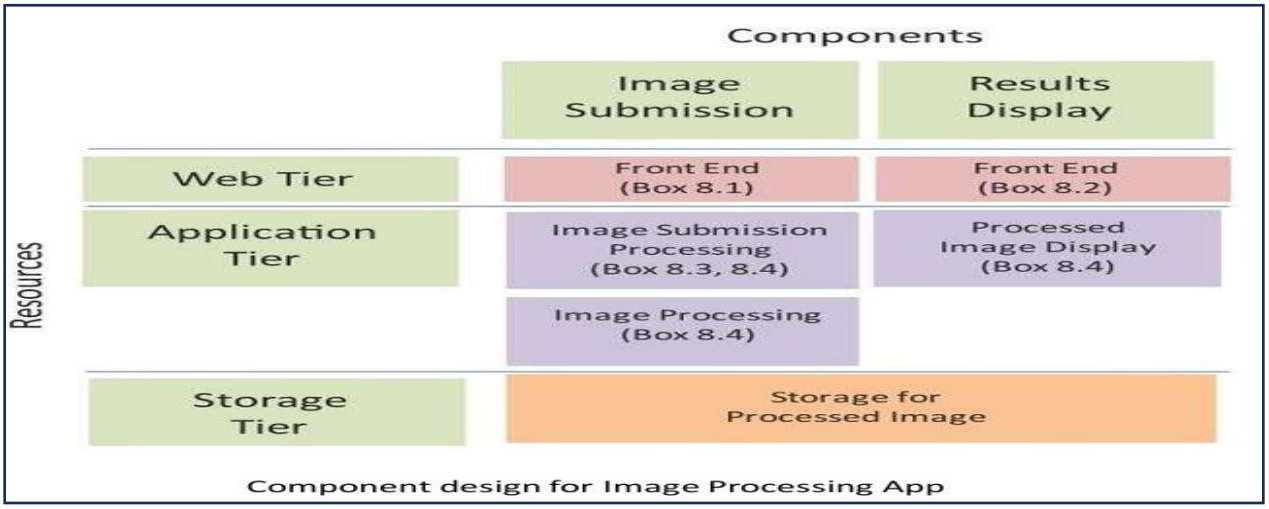
• The selected filters are applied to the image, and the processed image can then be downloaded.

• **Component Design**

• **Web Tier:** The web tier for the image processing app has front ends for image submission and displaying processed images.

• **Application Tier:** The application tier has components for processing the image submission requests, processing the submitted image, and processing requests for displaying the results.

• **Storage Tier:** The storage tier comprises the storage for processed images.



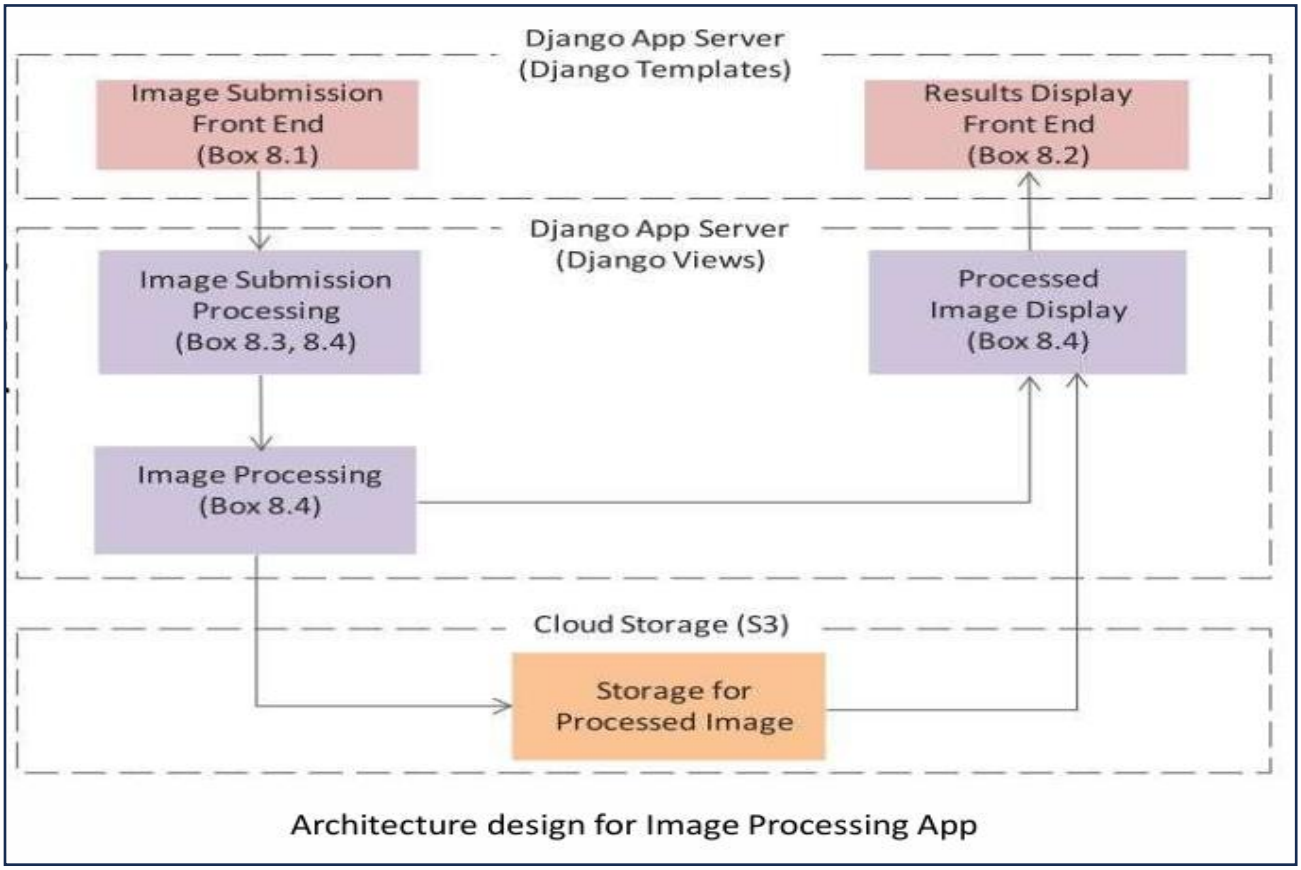
Component design for Image Processing App

Image Processing App – Architecture Design:

• Architecture design step which defines the interactions between the application components.

• This application uses the Django framework, therefore, the web tier components map to the Django templates and the application tier components map to the Django views.

• A cloud storage is used for the storage tier. For each component, the corresponding code box numbers are mentioned.

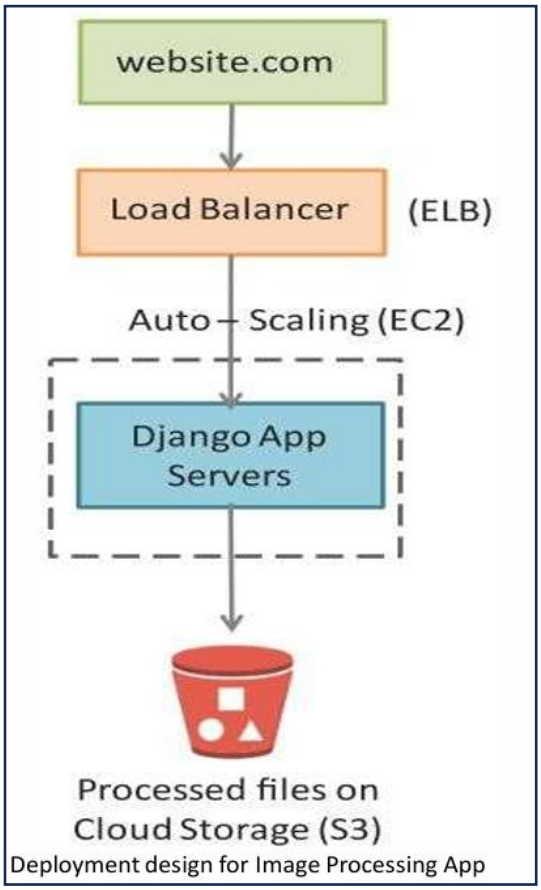


Architecture design for Image Processing App

Image Processing App – Deployment Design

• Deployment for the app is a multi-tier architecture comprising of load balancer, application servers, and a cloud storage for processed images.

• For each resource in the deployment, the corresponding Amazon Web Services (AWS) cloud service is mentioned.



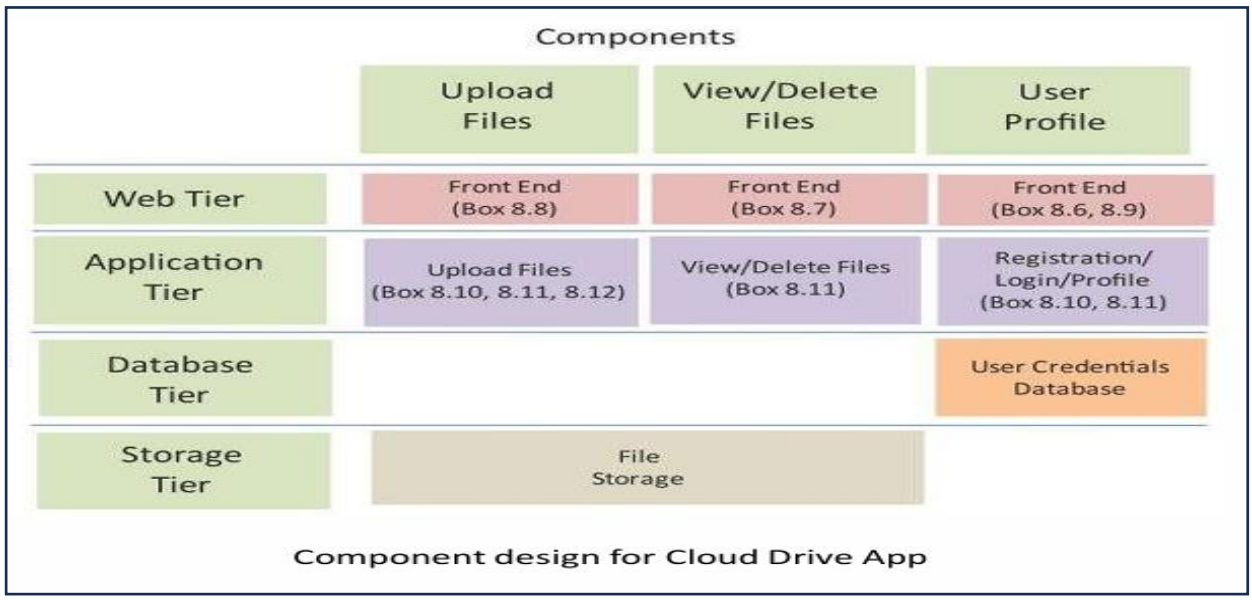
Deployment design for Image Processing App

Cloud Drive App – Component Design

• Functionality:
• A cloud-based document storage (Cloud Drive) application.
• This application allows users to store documents on cloud-based storage.

• Component Design:
• Web Tier: The web tier for the Cloud Drive app has front ends for uploading files, viewing/deleting files, and user profile.
• Application Tier: The application tier has components for processing requests for uploading files, viewing/deleting files, and the component that handles registration, profile, and login functions.

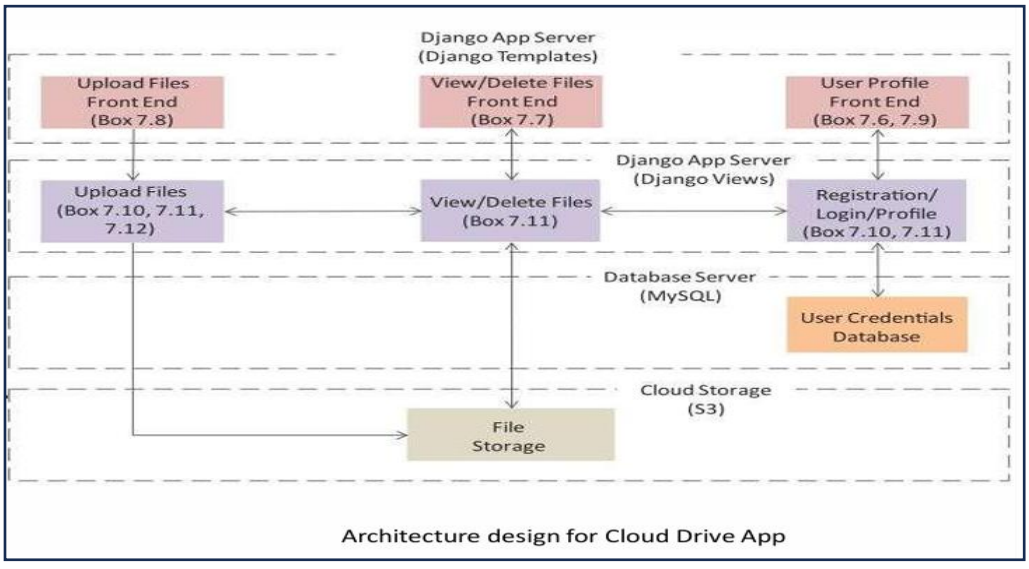
• Database Tier: The database tier comprises a user credentials database.
• Storage Tier: The storage tier comprises the storage for files.



Component design for Cloud Drive App

Cloud Drive App – Architecture Design

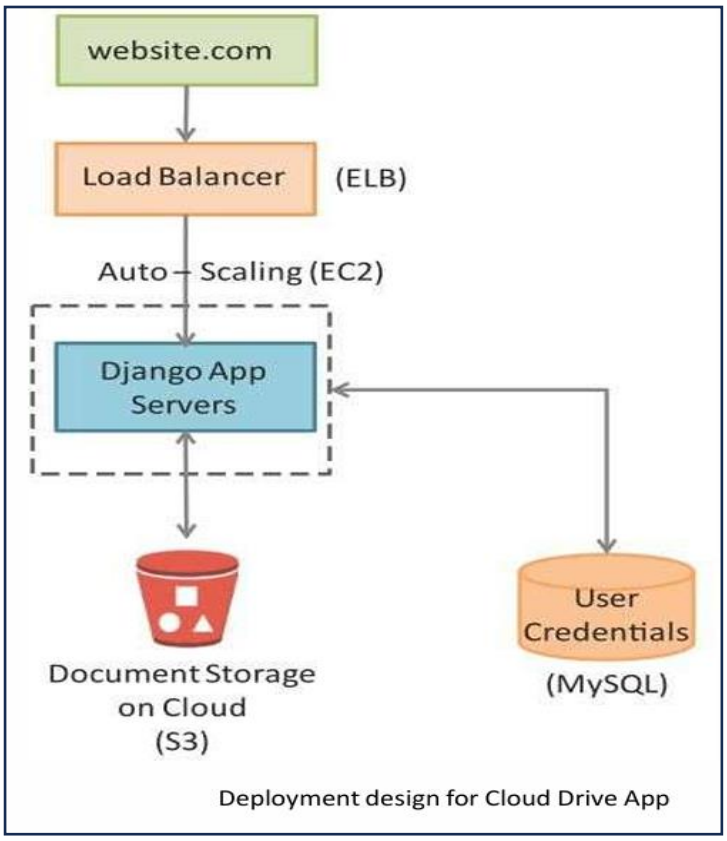
• Architecture design step defines the interactions between the application components.
• This application uses the Django framework, therefore:
• The web tier components map to the Django templates.
• The application tier components map to the Django views.
• A MySQL database is used for the database tier, and cloud storage is used for the storage tier.
• For each component, the corresponding code box numbers are mentioned.



Architecture design for Cloud Drive App

Cloud Drive App – Deployment Design

• Deployment for the app is a multi-tier architecture comprising of:
• Load balancer
• Application servers
• Cloud storage for storing documents
• Database server for storing user credentials
• For each resource in the reference architecture, the corresponding Amazon Web Services (AWS) cloud service is mentioned.



Deployment design for Cloud Drive App

MapReduce App – Component Design

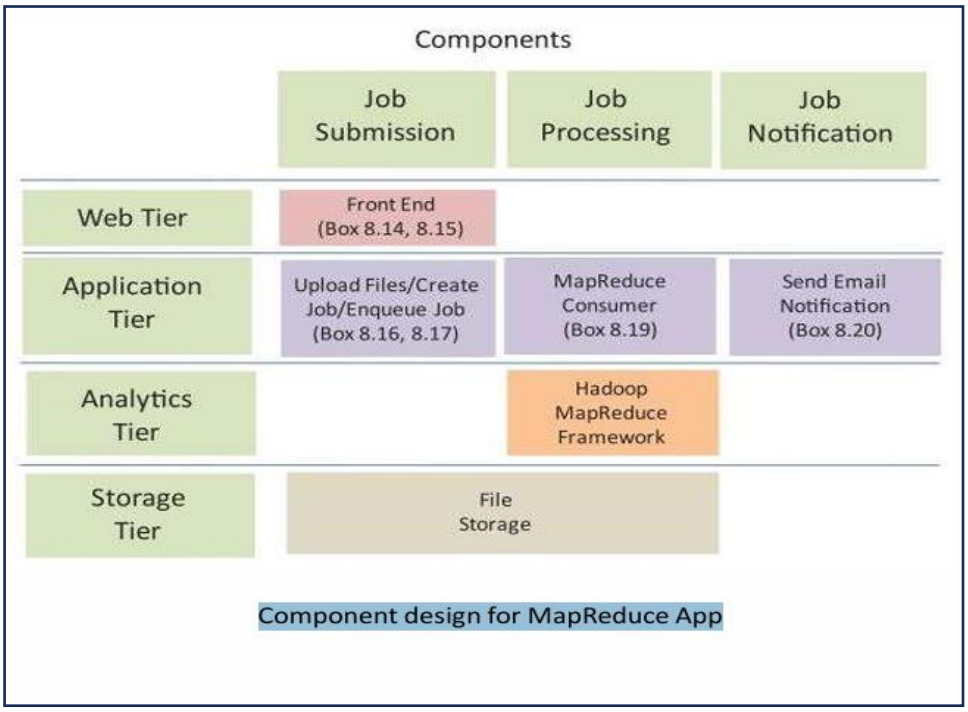
Functionality:

• This application allows users to submit MapReduce jobs for data analysis.
• It is based on the Amazon Elastic MapReduce (EMR) service.
• Users can upload data files for analysis and choose/upload Map and Reduce programs.
• The selected Map and Reduce programs, along with the input data, are submitted to a queue for processing.

Component Design:

• Web Tier: Provides a front-end interface for MapReduce job submission.
• Application Tier: Contains components for:

- Processing requests for uploading files
- Creating MapReduce jobs
- Enqueuing jobs
- MapReduce consumer
- Sending email notifications
- Analytics Tier: Uses the Hadoop framework for analytics and cloud storage for data.
- Storage Tier: Responsible for storing uploaded and processed files.



Component design for MapReduce App

MapReduce App – Architecture Design

• Architecture design step which defines the interactions between the application components.

• This application uses the Django framework, therefore, the web tier components map to the Django templates and the application tier components map to the Django views.

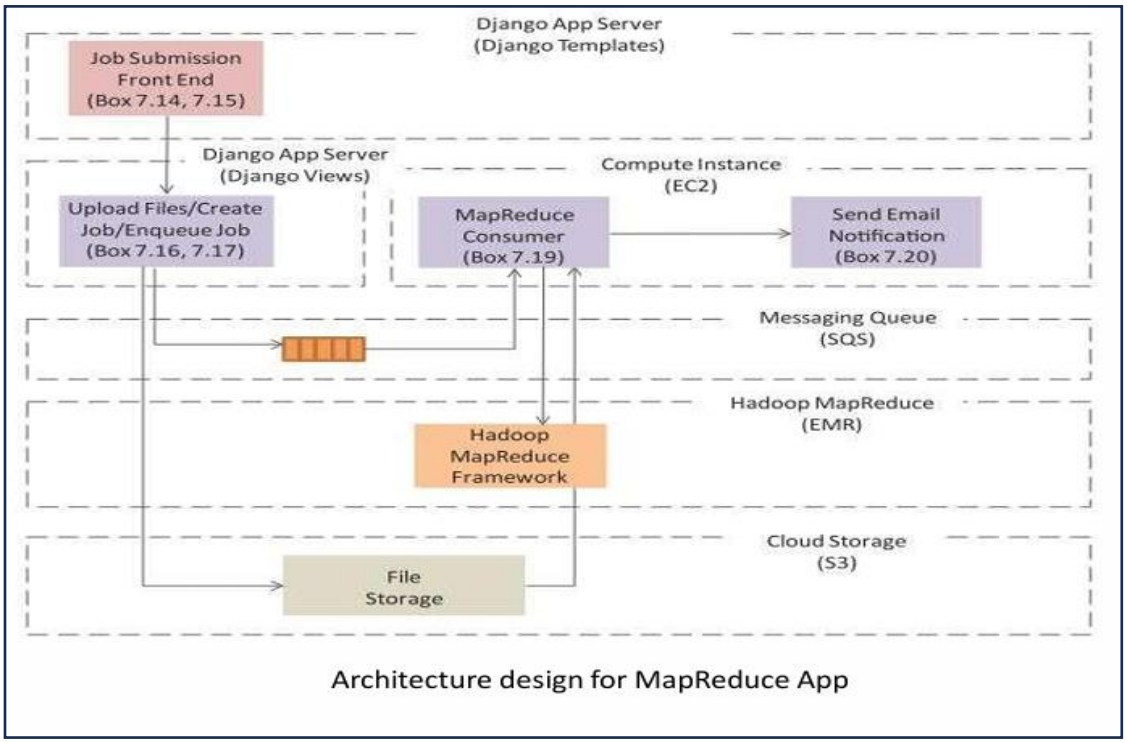
• For each component, the corresponding code box numbers are mentioned.

• To make the application scalable the job submission and job processing components are separated.

• The MapReduce job requests are submitted to a queue.

• A consumer component that runs on a separate instance retrieves the MapReduce job requests from the queue and creates the MapReduce jobs and submits them to the Amazon EMR service.

• The user receives an email notification with the download link for the results when the job is complete.

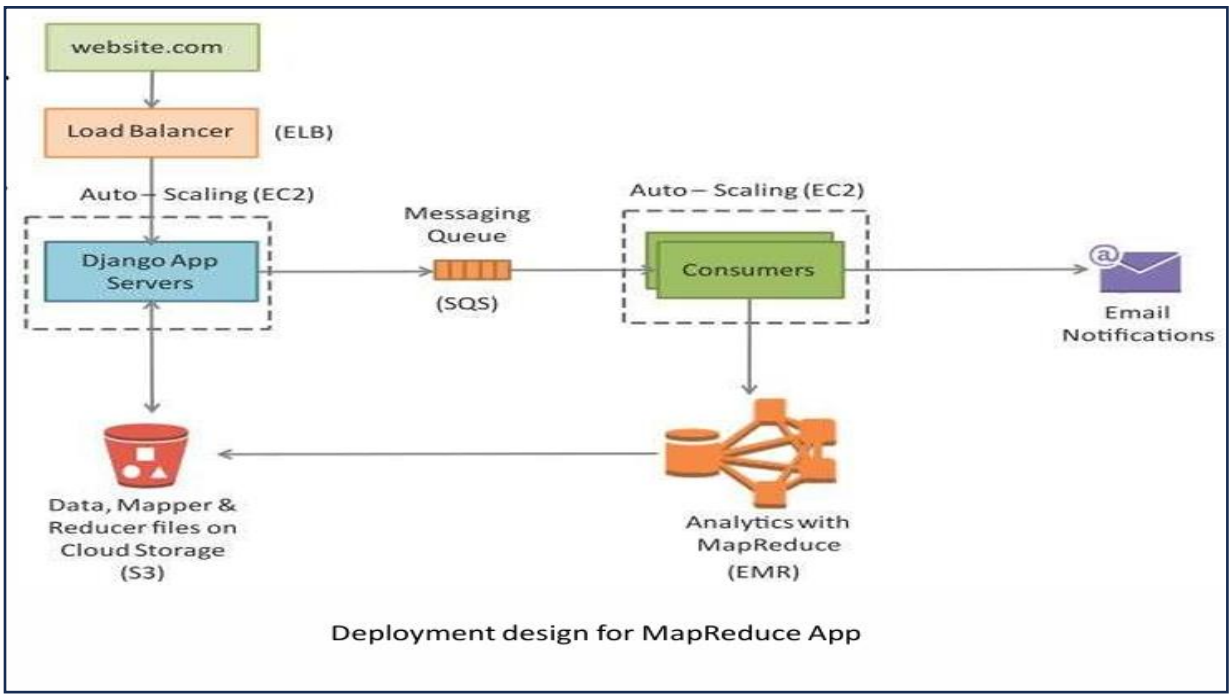


Architecture design for MapReduce App

MapReduce App – Deployment Design:

• Deployment for the app is a multi-tier architecture comprising of load balancer, application servers and a cloud storage for storing MapReduce programs, input data and MapReduce output.

• For each resource in the deployment the corresponding Amazon Web Services (AWS) cloud service is mentioned.

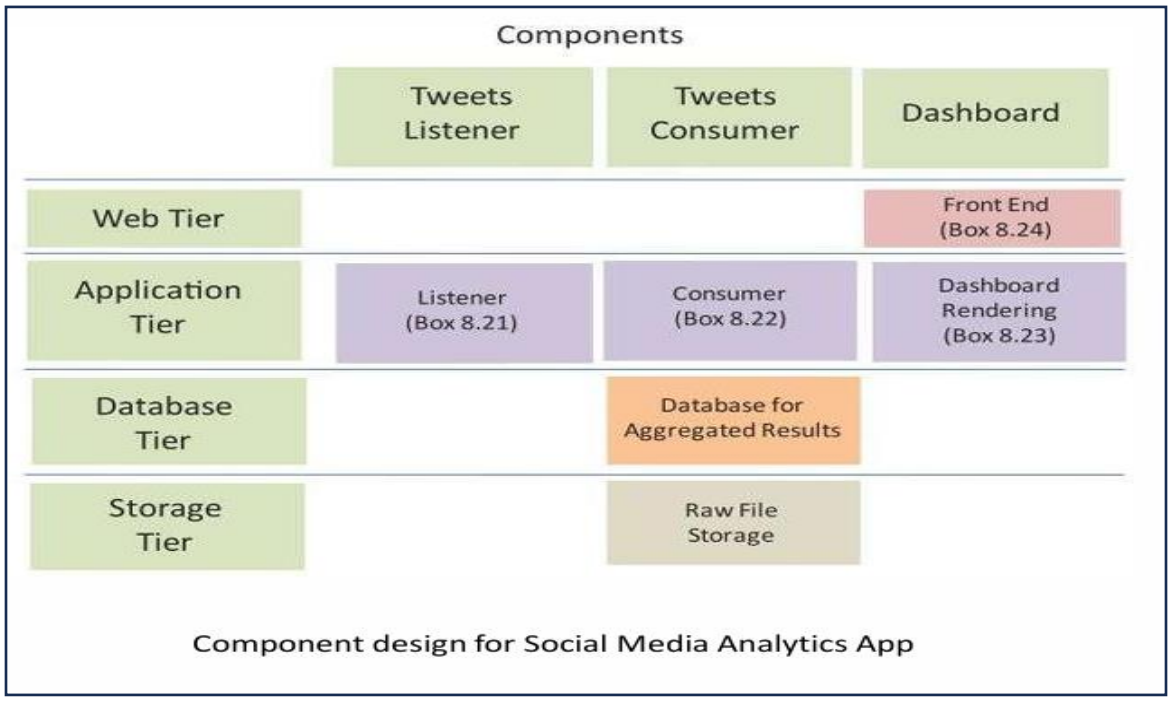


Deployment design for MapReduce App

Social Media Analytics App – Component Design:

• Functionality:
• A cloud-based Social Media Analytics application.
• This application collects the social media feeds (Twitter tweets) on a specified keyword in real time and analyzes the sentiments of the tweets and provides aggregate results.

• Component Design
• Web Tier: The web tier has a front end for displaying results.
• Application Tier: The application tier has a listener component that collects social media feeds, a consumer component that analyzes tweets and a component for rendering the results in the dashboard.
• Database Tier: A MongoDB database is used for the database tier and a cloud storage is used for the storage tier.
• Storage Tier: The storage tier comprises of the storage for files.



Component design for Social Media Analytics App

Social Media Analytics App – Architecture Design:

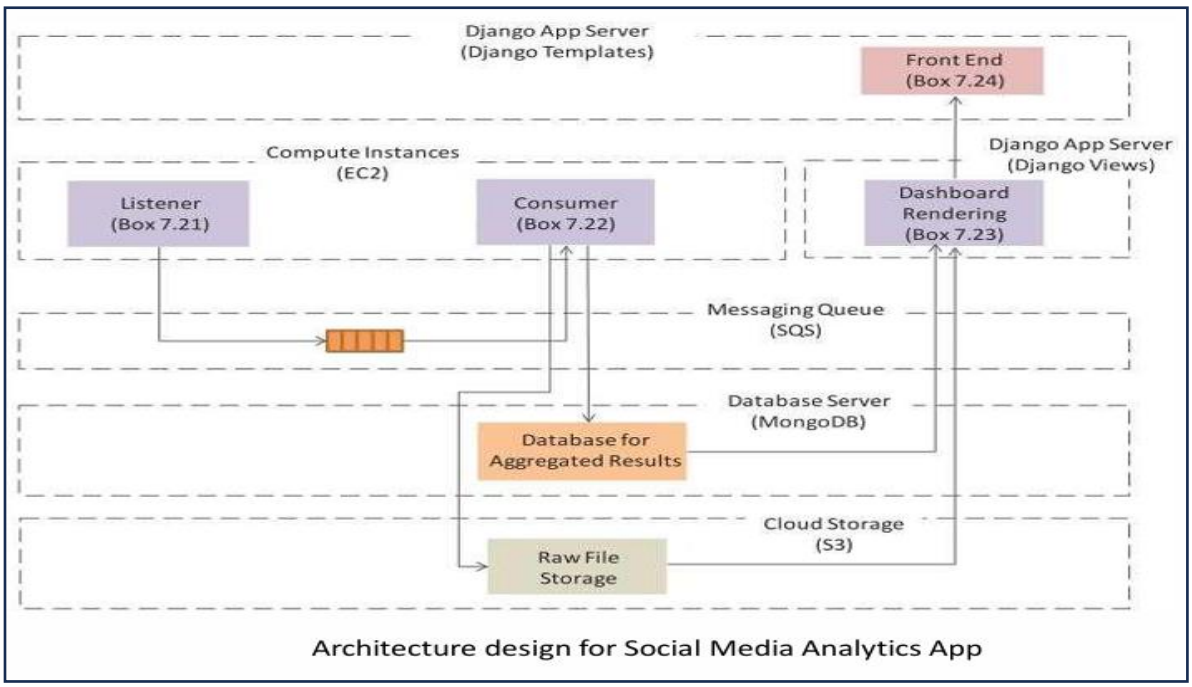
• Architecture design step which defines the interactions between the application components.

• To make the application scalable the feeds collection component (Listener) and feeds processing component (Consumer) are separated.

• The Listener component uses the Twitter API to get feeds on a specific keyword (or a list of keywords) and enqueues the feeds to a queue.

• The Consumer component (that runs on a separate instance) retrieves the feeds from the queue and analyzes the feeds and stores the aggregated results in a separate database.

• The aggregate results are displayed to the users from a Django application.

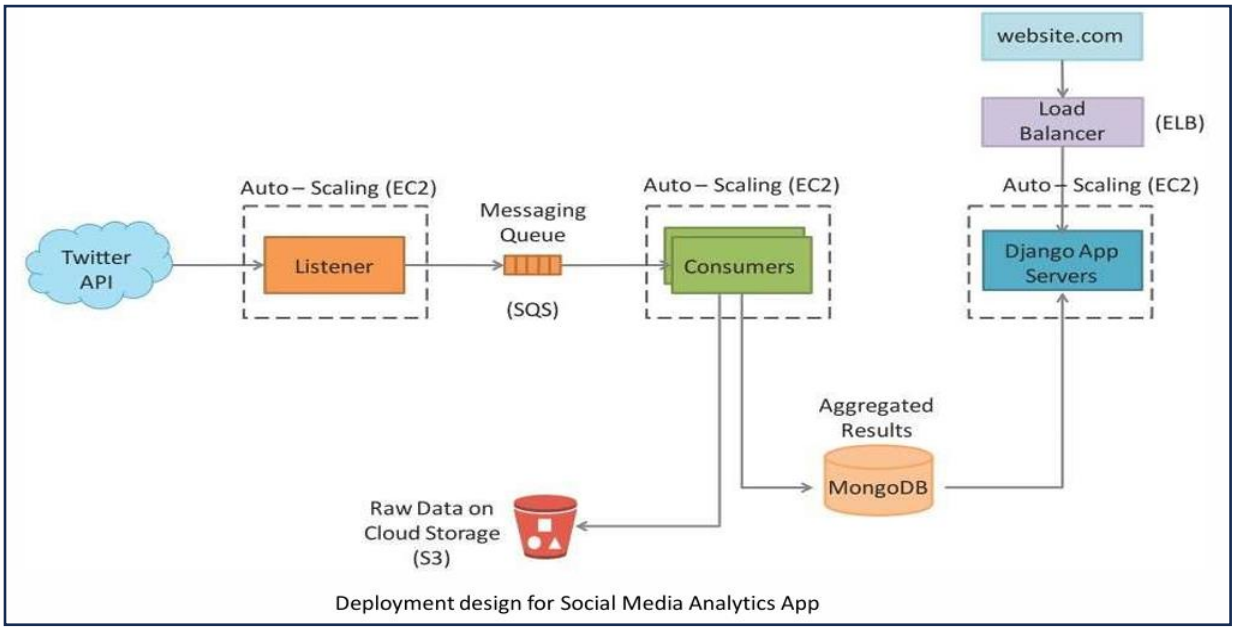


Architecture design for Social Media Analytics App

Social Media Analytics App – Deployment Design:

• Deployment for the app is a multi-tier architecture comprising of load balancer, application servers, listener and
consumer instances, a cloud storage for storing raw data and a database server for storing aggregated results.

• For each resource in the deployment the corresponding Amazon Web Services (AWS) cloud service is mentioned.



Deployment design for Social Media Analytics App

Social Media Analytics App – Dashboard



UNIT-4

Big Data Analytics: Introduction, Clustering Bigdata, Classification of Bigdata, Recommendation of Systems.
Cloud Application Bench marking and Tuning: Introduction, Workload Characteristics, Application Performance Metrics, Design Considerations for a Bench marking Methodology, Bench marking Tools, Deployment Prototyping, Load Testing & Bottleneck Detection case Study, Hadoop bench marking case Study.

Big Data Analytics

Outline:

•Big Data analytics approaches

•Approaches for clustering big data

•Approaches for classification of big data

•Recommendation Systems

Big Data:

• Big data is defined as collections of data sets whose volume, velocity in terms of time variation, or variety is so large that it is difficult to store, manage, process and analyse the data using traditional databases and data processing tools.

Characteristics of big data:

Volume:

• Though there is no fixed threshold for the volume of data to be considered as big data, however, typically, the term big data is used for massive scale data that is difficult to store, manage and process using traditional databases and data processing architectures. The volumes of data generated by modern IT, industrial, healthcare and systems are growing exponentially driven by the lowering costs of data storage and processing architectures and the need to extract valuable insights from the data to improve business processes, efficiency and service to consumers.

Velocity:

• Velocity is another important characteristic of big data and the primary reason for exponential growth of data. Velocity of data refers to how fast the data is generated. Modern IT, industrial and other systems are generating data at increasingly higher speeds generating big data.

Variety

• Variety refers to the forms of the data. Big data comes in different forms such as structured or unstructured data, including text data, image, audio, video and sensor data.

Clustering Big Data

• Clustering is the process of grouping similar data items together such that data items that are more similar to each other (with respect to some similarity criteria) than other data items are put in one cluster.

• Clustering big data is of much interest, and happens in applications such as:

• Clustering social network data to find a group of similar users

• Clustering electronic health record (EHR) data to find similar patients.

• Clustering sensor data to group similar or related faults in a machine

• Clustering market research data to group similar customers

• Clustering clickstream data to group similar users

• Clustering is achieved by clustering algorithms that belong to a broad category algorithm called unsupervised machine learning.

• Unsupervised machine learning algorithms find the patterns and hidden structure in data for which no training data is available.

k-means Clustering:

• k-means is a clustering algorithm that groups data items into k clusters, where k is user defined.

• Each cluster is defined by a centroid point.

• k-means clustering begins with a set of k centroid points which are either randomly chosen from the dataset or chosen using some initialization algorithm such as canopy clustering.

• The algorithm proceeds by finding the distance between each data point in the data set and the centroid points.

• Based on the distance measure, each data point is assigned to a cluster belonging to the closest centroid.

• In the next step the centroids are recomputed by taking the mean value of all the data points in a cluster.

• This process is repeated till the centroids no longer move more than a specified threshold.

k-means Clustering Algorithm:

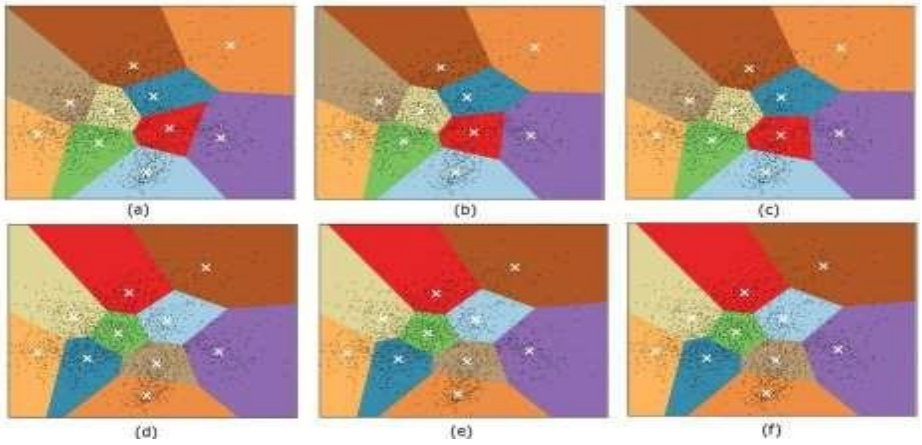
Start with k centroid points
while the centroids no longer move beyond a threshold or maximum number of iterations reached:
for each point in the dataset:

    for each centroid:

        find the distance between the point and the centroid
        assign the point to the cluster belonging to the nearest centroid for each

cluster:

recompute the centroid point by taking mean value of all points in the cluster



Clustering Documents with k-means

• Document clustering is the most commonly used application of k-means clustering algorithm.

• Document clustering problem occurs in many big data applications such as finding similar news articles, finding similar patients using electronic health records, etc.

• Before applying k-means algorithm for document clustering, the documents need to be vectorized. Since documents contain textual information, the process of vectorization is required for clustering documents.

• **The process of generating document vectors involves several steps**:

- A dictionary of all words used in the tokenized records is generated. Each word in the dictionary has a dimension number assigned to it which is used to represent the dimension the word occupies in the document vector.
- The number of occurrences or term frequency (TF) of each word is computed.
- Inverse Document Frequency (IDF) for each word is computed. Document Frequency (DF) for a word is the number of documents (or records) in which the word occurs.
- Weight for each word is computed. The term weight Wi is used in the document vector as the value for the dimension-i.
- Similarity between documents is computed using a distance measure such as Euclidean distance measure.

k-means with MapReduce:

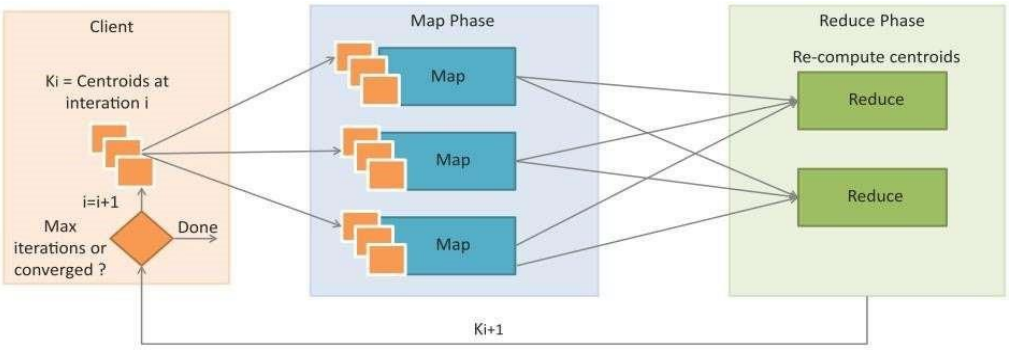
• The data to be clustered is distributed on a distributed file system such as HDFS and split into blocks which are replicated across different nodes in the cluster.

• Clustering begins with an initial set of centroids. The client program controls the clustering process.

• In the Map phase, the distances between the data samples and centroids are calculated and each sample is assigned to the nearest centroid.

• In the Reduce phase, the centroids are recomputed using the mean of all the points in each cluster.

• The new centroids are then fed back to the client which checks whether convergence is reached or maximum number of iterations are completed.



DBSCAN clustering:

• DBSCAN is a density clustering algorithm that works on the notions of density reachability and density connectivity.

• Density Reachability

• Is defined on the basis of Eps-neighbourhood, where Eps-neighbourhood means that for every point p in a cluster C there is a point q in C so that p is inside of the Eps-neighbourhood of q and there are at least a minimum number (Min Pts) of points in an Eps-neighbourhood of that point.

• A point p is called directly density-reachable from a point q if it is not farther away than a given distance (Eps) and if it is surrounded by at least a minimum number (Min Pts) of points that may be considered to be part of a cluster.

• Density Connectivity

• A point p is density connected to a point q if there is a point o such that both, p and q are density-reachable from o wrt. Eps and Min Pts.

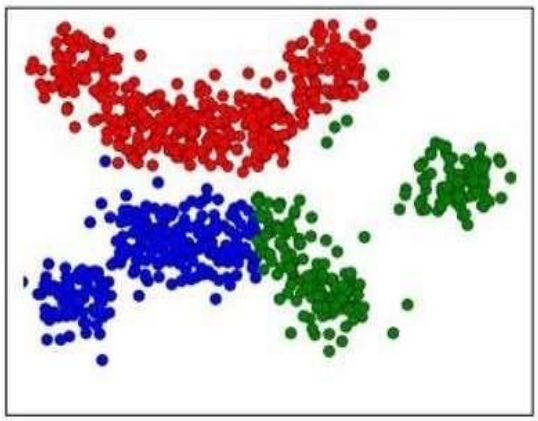
• A cluster, is then defined based on the following two properties:

• **Maximality**: For all point p, q if p belongs to cluster C and q is density-reachable from p (wrt. Eps and MinPts), then q also belongs to the cluster C.

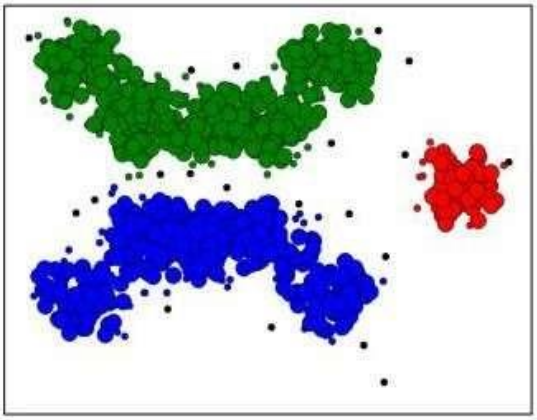
• **Connectivity**: For all point p, q in cluster C, p is density-connected to q (wrt. Eps and MinPts).

DBSCAN vs K-means:

• DBSCAN can find irregular shaped clusters as seen from this example and can even find a cluster completely surrounded by a different cluster.



(a) kmeans



(b) DBSCAN

Classification of Big Data

• Classification is the process of categorizing objects into predefined categories.

• Classification is achieved by classification algorithms that belong to a broad category of algorithms called supervised machine learning.

• Supervised learning involves inferring a model from a set of input data and known responses to the data (training data) and then using the inferred model to predict responses to new data.

• Binary classification

• Binary classification involves categorizing the data into two categories. For example, classifying the sentiment of a news article into positive or negative, classifying the state of a machine into good or faulty, classifying the heath test into positive or negative, etc.

• Multi-class classification

• Multi-class classification involves more than two classes into which the data is categorized. For example, gene expression classification problem involves multiple classes.

• Document classification

• Document classification is a type of multi-class classification approach in which the data to the classified is in the form of text document. For classifying news articles into different categories such as politics, sports, etc

Performance of Classification Algorithms:

• **Precision:**

Precision is the fraction of objects that are classified correctly.

$$Precision = \frac{TruePositive}{(TruePositive + FalsePositive)}$$

• **Recall:**

Recall is the fraction of objects belonging to a category that are classified correctly.

$$Recall = \frac{TruePositive}{(TruePositive + FalseNegative)}$$

• **Accuracy:**

$$Recall = \frac{TruePositive}{(TruePositive + FalseNegative)}$$

• **F1-score**:

F1-score is a measure of accuracy that considers both precision and recall. F1-score is the harmonic means of precision and recall given as

$$Recall = \frac{TruePositive}{(TruePositive + FalseNegative)}$$

Naive Bayes:

• Naive Bayes is a probabilistic classification algorithm based on the Bayes theorem with a naive assumption about the independence of feature attributes. Given a class variable C and feature variables F1,...,Fn , the conditional probability (posterior) according to Bayes theorem is given as,

$$P(C|F_1,...,F_n) = \frac{P(F_1,...,F_n|C)P(C)}{P(F_1,...,F_n)}$$

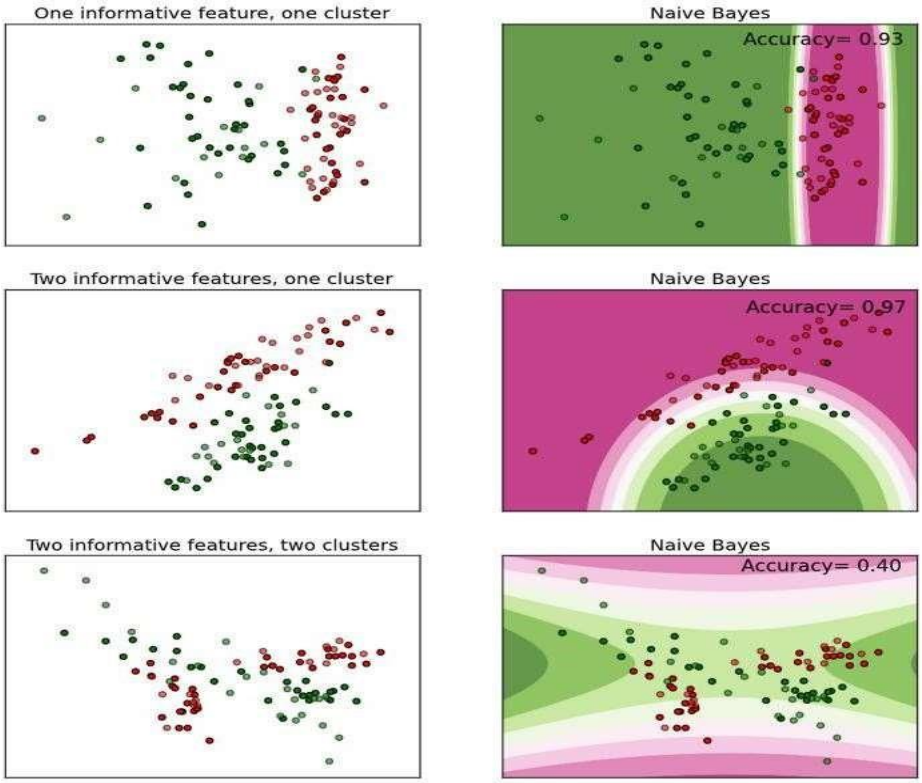
• where, P(C|F1,...,Fn) is the posterior probability, P(F1,...,Fn |C) is the likelihood and P(C) is the prior probability and P(F1,...,Fn) is the evidence. Naive Bayes makes a naïve assumption about the independence every pair of features given as,

$$P(F_1,...,F_n|C) = \prod_{i=1}^{n} P(F_i|C)$$

• Since the evidence P(F1,...,Fn) is constant for a given input and does not depend on the class variable C, only the numerator of the posterior probability is important for classification.

• With this simplification, classification can then be done as follows,

$$C = argmax_C P(C) \prod_{i=1}^{n} P(F_i|C)$$



Decision Trees:

• Decision Trees are a supervised learning method that use a tree created from simple decision rules learned from the training data as a predictive model.

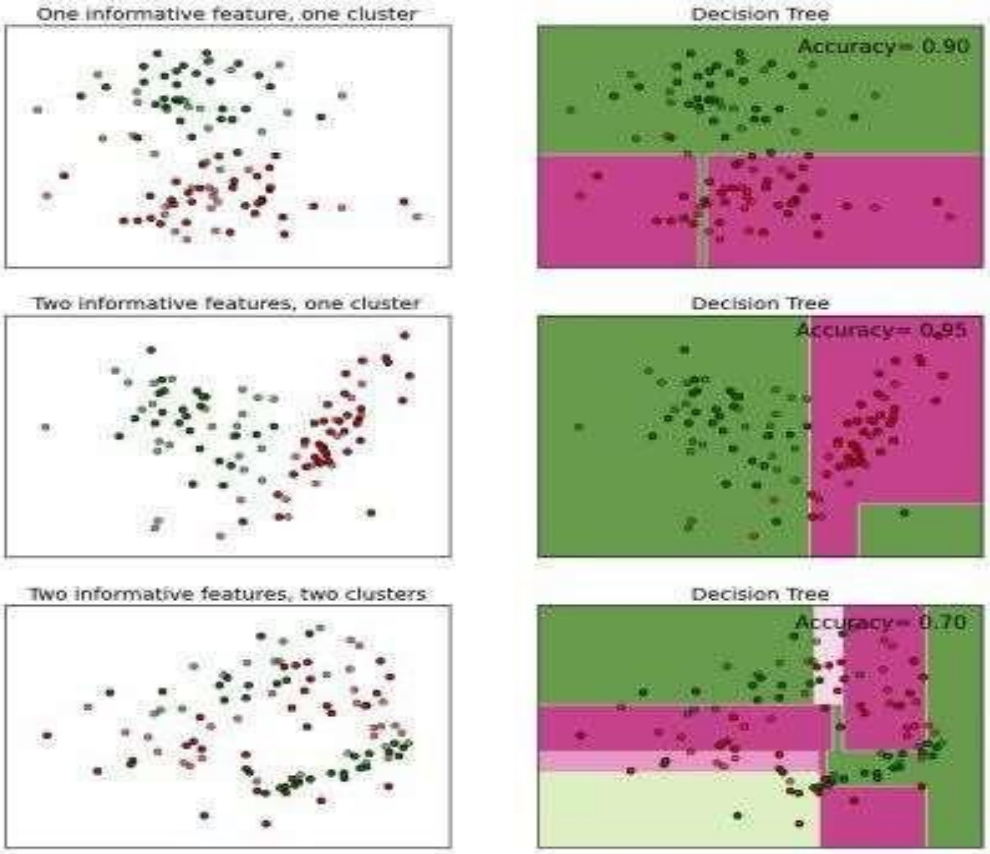
• The predictive model is in the form of a tree that can be used to predict the value of a target variable based on a several attribute variables.

• Each node in the tree corresponds to one attribute in the dataset on which the "split" is performed.

• Each leaf in a decision tree represents a value of the target variable.

• The learning process involves recursively splitting on the attributes until all the samples in the child node have the same value of the target variable or splitting further results in no further information gain.

• To select the best attribute for splitting at each stage, different metrics can be used.



Splitting Attributes in Decision Trees:

To select the best attribute for splitting at each stage, different metrics can be used such as:

• Information Gain :

• Information content of a discrete random variable X with probability mass function (PMF), P(X), is defined as,

$$I(X) = -\log_2 P(X)$$

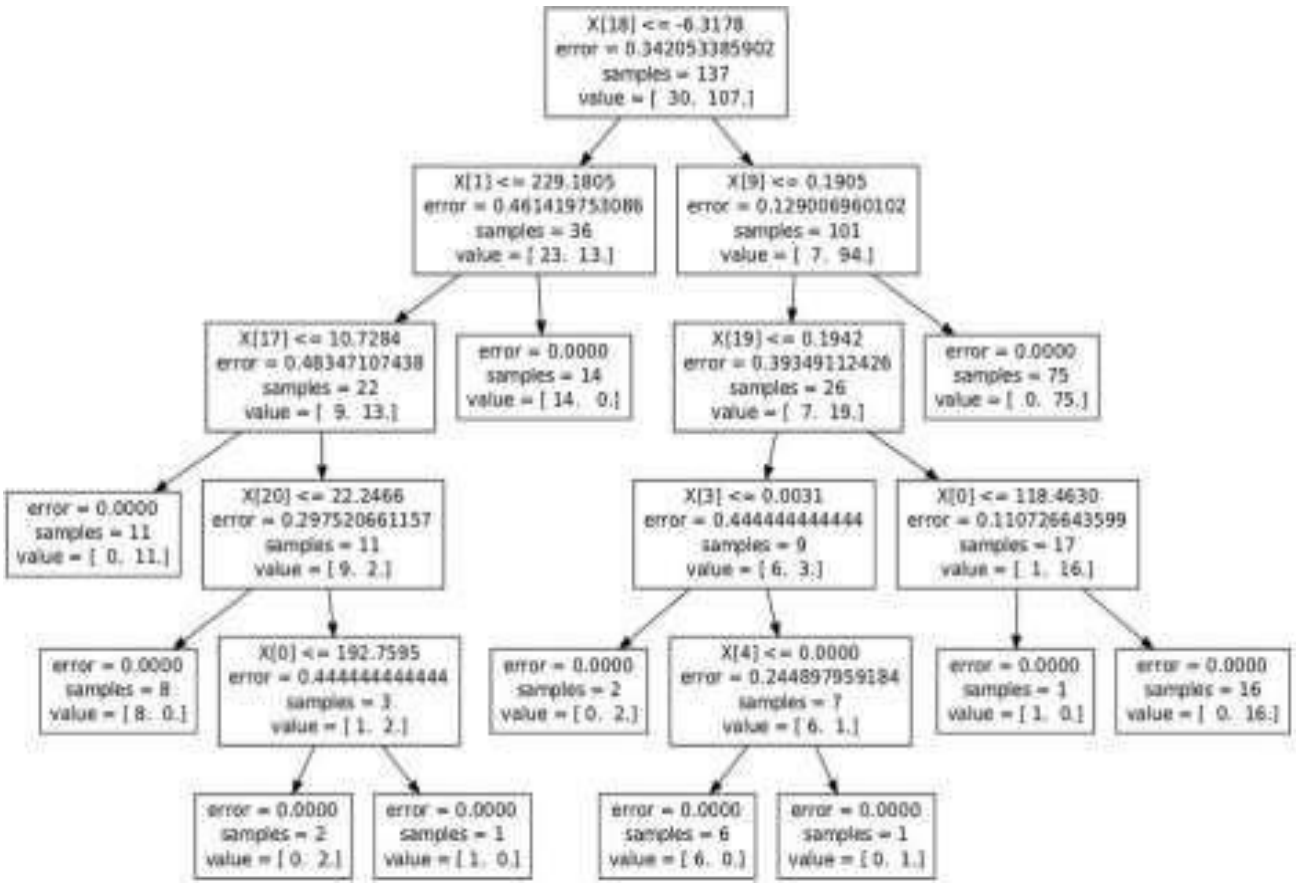
• Information gain is defined based on the entropy of the random variable which is defined as,

$$H(X) = E[I(X)] = E[-\log_2 P(X)] = -\sum_i \log_2 P(x_i)$$

• Entropy is a measure of uncertainty in a random variable and choosing the attribute with the highest information gain results in a split that reduces the uncertainty the most at that stage.

• Gini Coefficient

• Gini coefficient measures the inequality, i.e. how often a randomly chosen sample that is labelled based on the distribution of labels, would be labelled incorrectly. Gini coefficient is defined as $G(X) = 1 - \sum_i P(x_i)^2$



Decision Tree Algorithms :

• There are different algorithms for building decisions trees, popular ones being ID3 and C4.5.

ID3:

• Attributes are discrete. If not, discretize the continuous attributes.

• Calculate the entropy of every attribute using the dataset.

• Choose the attribute with the highest information gain.

• Create branches for each value of the selected attribute.

• Repeat with the remaining attributes.

• The ID3 algorithm can be result in over-fitting to the training data and can be expensive to train especially for continuous attributes.

C4.5:

• The C4.5 algorithm is an extension of the ID3 algorithm. C4.5 supports both discrete and continuous attributes.

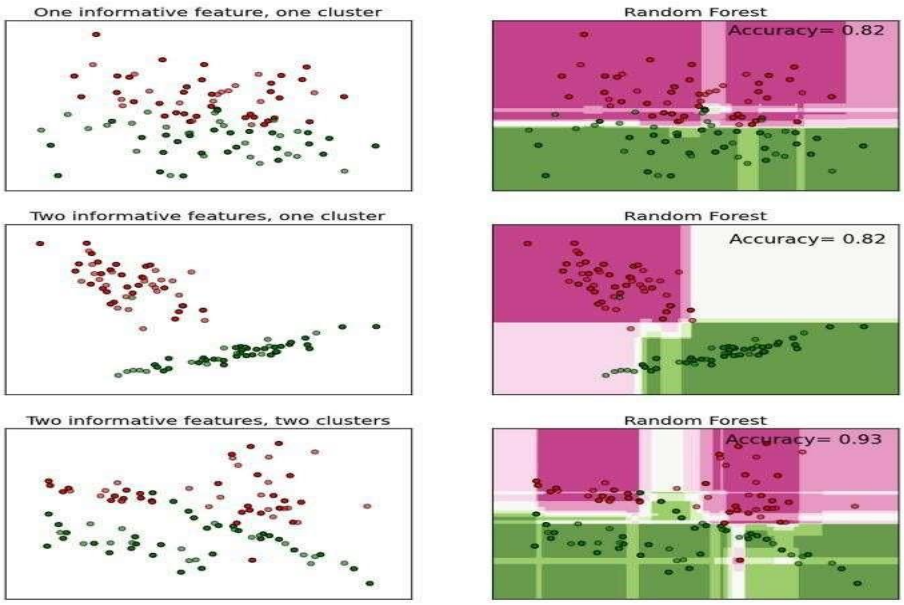
• To support continuous attributes, C4.5 finds thresholds for the continuous attributes and then splits based on the threshold values. C4.5 prevents over-fitting by pruning trees after they have been created.

• Pruning involves removing or aggregating those branches which provide little discriminatory power.

Random Forest:

• Random Forest is an ensemble learning method that is based on randomized decision trees.

• Random Forest trains a number decision trees and then takes the majority vote by using the mode of the class predicted by the individual trees.



Breiman's Algorithm:

1.Draw a bootstrap sample (n times with replacement from the N samples in the training set) from the dataset

2.Train a decision tree -Until the tree is fully grown (maximum size) -Choose next leaf node -Select m attributes (m is much less than the total number of attributes M) at random. -Choose the best attribute and split as usual

3.Measure out-of-bag error -Use the rest of the samples (not selected in the bootstrap) to estimate the error of the tree, by predicting their classes.

4.Repeat steps 1-3 k times to generate k trees.

5.Make a prediction by majority vote among the k trees

Support Vector Machine:

• Support Vector Machine (SVM) is a supervised machine learning approach used for classification and regression.

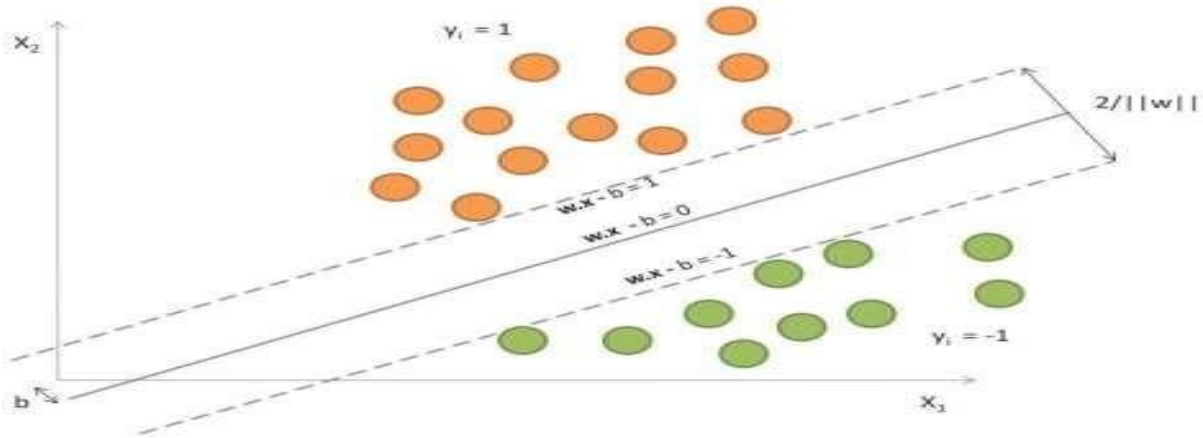
• The basic form is SVM is a binary classifier that classifies the data points into one of the two classes.

• SVM training involves determining the maximum margin hyperplane that separates the two classes.

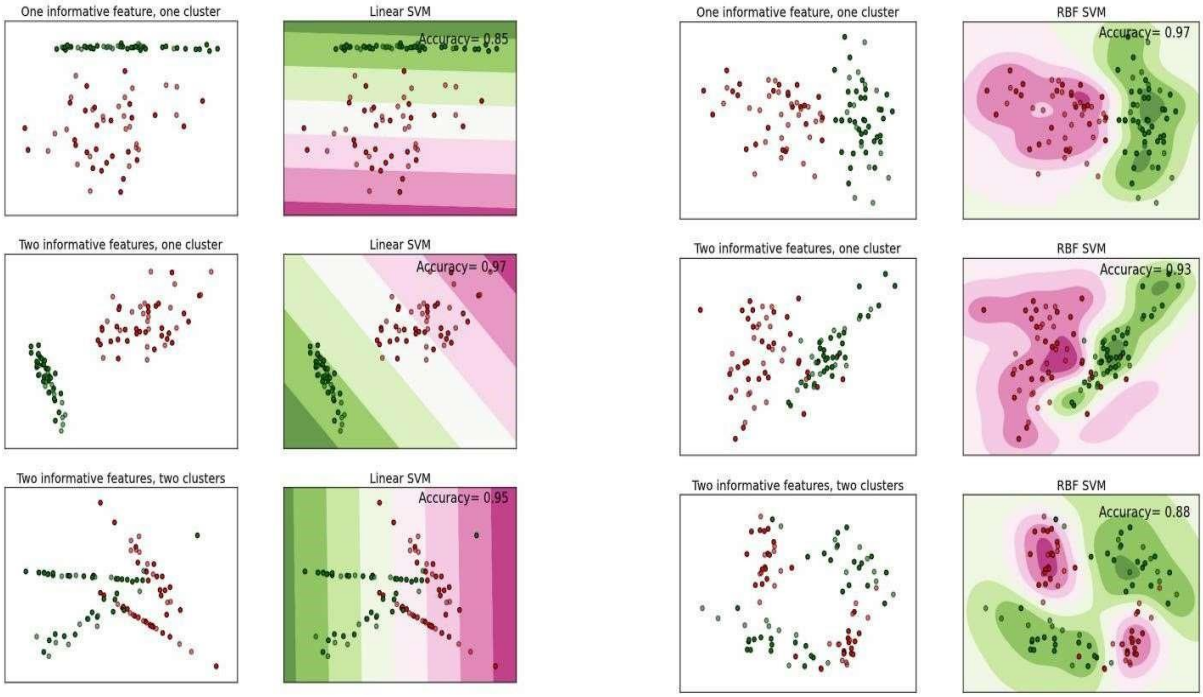
• The maximum margin hyperplane is one which has the largest separation from the nearest training data point.

• Given a training data set (xi ,yi) where xi is an n dimensional vector and yi = 1 if xi is in class 1 and yi = -1 if xi is in class 2.

• A standard SVM finds a hyperplane w.x-b = 0, which correctly separates the training data points and has a maximum margin which is the distance between the two hyperplanes w.x-b = 1 and w.x-b = -1.



Support Vector Machine:



Recommendation Systems

• Recommendation systems are an important part of modern cloud applications such as e-Commerce, social networks, content delivery networks, etc.

• Item-based or Content-based Recommendation

• Provides recommendations to users (for items such as books, movies, songs, or restaurants) for unrated items based on the characteristics of the item.

• Collaborative Filtering

• Provides recommendations based on the ratings given by the user and other users to similar items.

Cloud Application Benchmarking & Tuning

Outline:

• Cloud application workload characteristics

• Performance metrics for cloud applications

• Cloud application testing

• Performance testing tools

• Load test and bottleneck detection case study

Benchmarking:

• Benchmarking of cloud applications is important or the following reasons:

• Provisioning and capacity planning:

• The process of provisioning and capacity planning for cloud applications involves determining the amount of computing, memory and network resources to provision for the application.

• Benchmarking can help in comparing alternative deployment architectures and choosing the best and most cost-effective deployment architecture that can meet the application performance requirements.

• Ensure proper utilization of resources:

• Benchmarking can help in determining the utilization of computing, memory and network resources for applications and identify resources which are either under-utilized or overprovisioned and hence save deployments costs.

• Market readiness of applications:

• Performance of an application depends on the characteristics of the workloads it experiences. Different types of workloads can dead to different performance for the same application.

• To ensure the market readiness of an application it is important to model all types of workloads the application can experience and benchmark the application with such workloads. Cloud Application Benchmarking.

Cloud Application Benchmarking - Steps

• Trace Collection/Generation:

• The first step in benchmarking cloud applications is to collect/generate traces of real application workloads. For generating a trace of workload, the application is instrumented to log information such as the requests submitted by the users, the time-stamps of the requests, etc.

• **Workload Modelling**:

• Workload modelling involves creation of mathematical models that can be used for generation of synthetic workloads.

• Workload Specification:

• Since the workload models of each class of cloud computing applications can have different workload attributes, a Workload Specification Language (WSL) is often used for specification of application workloads. WSL can provide a structured way for specifying the workload attributes that are critical to the performance of the applications. WSL can be used by synthetic workload generators for generating workloads with slightly varying the characteristics.

• Synthetic Workload Generation:

• Synthetic workloads are used for benchmarking cloud applications. An important requirement for a synthetic workload generator is that the generated workloads should be representative of the real workloads. Synthetic Workload Generation Approaches • Empirical approach

• In this approach traces of applications are sampled and replayed to generate the synthetic workloads.

• The empirical approach lacks flexibility as the real traces obtained from a particular system are used for workload generation which may not well represent the workloads on other systems with different configurations and load conditions.

• Analytical approach:

• Uses mathematical models to define the workload characteristics that are used by a synthetic workload generator.

• Analytical approach is flexible and allows generation of workloads with different characteristics by varying the workload model attributes.

• With the analytical approach it is possible to modify the workload model parameters one at a time and investigate the effect on application performance to measure the application sensitivity to different parameters.

User Emulation vs Aggregate Workloads

The commonly used techniques for workload generation are:

• User Emulation:

• Each user is emulated by a separate thread that mimics the actions of a user by alternating between making requests and lying idle.

• The attributes for workload generation in the user emulation method include think time, request types, InterQuest dependencies, for instance.

• User emulation allows fine grained control over modelling the behavioural aspects of the users interacting with the system under test, however, it does not allow controlling the exact time instants at which the requests arrive the system.

• Aggregate Workload Generation:

• Allows specifying the exact time instants at which the requests should arrive the system under test.

• However, there is no notion of an individual user in aggregate workload generation, therefore, it is not possible to use this approach when dependencies between requests need to be satisfied.

• Dependencies can be of two types inter-request and data dependencies.

• An inter-request dependency exists when the current request depends on the previous request, whereas a data dependency exists when the current requests require input data which is obtained from the response of the previous request.

Workload Characteristics

• Session:

• A set of successive requests submitted by a user constitutes a session.

• Inter-Session Interval:

• Inter-session interval is the time interval between successive sessions.

• Think Time:

• In a session, a user submits a series of requests in succession. The time interval between two successive requests is called think time.

• Session Length:

• The number of requests submitted by a user in a session is called the session length.

• Workload Mix:

• Workload mix defines the transitions between different pages of an application and the proportion in which the pages are visited.

Application Performance Metrics

The most commonly used performance metrics for cloud applications are:

• Response Time:

• Response time is the time interval between the moment when the user submits a request to the application and the moment when the user receives a response.

• Throughput:

• Throughput is the number of requests that can be serviced per second.

Design Considerations for Benchmarking Methodology

• Accuracy:

• Accuracy of a benchmarking methodology is determined by how closely the generated synthetic workloads mimic the realistic workloads.

• Ease of Use:

• A good benchmarking methodology should be user friendly and should involve minimal hand coding effort for writing scripts for workload generation that take into account the dependencies between requests, workload attributes, for instance.

• Flexibility:

• A good benchmarking methodology should allow fine grained control over the workload attributes such as think time, inter-session interval, session length, workload mix, for instance, to perform sensitivity analysis.

• Sensitivity analysis is performed by varying one workload characteristic at a time while keeping the others constant.

• Wide Application Coverage:

• A good benchmarking methodology is one that works for a wide range of applications and not tied to the application architecture or workload types.

Bench Marking Tool

There are several workload generation tools developed to study web applications such as http erf [52], SURGE [53], SWAT [54], HP LoadRunner [55], and SPECweb99 [57]. Such workload generation tools repeatedly send requests from machines configured as clients to the intended systems under test. Table 11.1 provides a comparison of few workload generation tools. Several other tools generate synthetic workloads through transformation (eg. permutation) of empirical workload traces [58, 59, 60]. Several studies on analysis and modelling of web workloads have been done [61, 62]. Since obtaining real traces from complex multi-tier systems is difficult, a number of benchmarks have been developed to

model the real systems [63, 64, 65). Figure 11.1 shows a workflow used by traditional performance evaluation approaches, which require a real user to interact with the application to record scripts that are used by load generators. Figure 11.2 shows the performance evaluation workflow used by recent tools such as GT-CAT [56] developed by the authors.

Capturing Workload Characteristics: In traditional approach shown in Figure 11.1, to capture workload characteristics, a real user's interactions with a cloud application are first recorded as virtual user scripts. The recorded virtual user scripts then are parameterized to account for randomness in application and workload parameters. There is no underlying statistical model involved in such approaches as recorded scripts are used to drive the load generators. In the automated approach shown in Figure 11.2, real traces of a multi-tier application which are logged on web servers, application servers and database servers are analysed to generate benchmark and workload models that capture the cloud application and workload characteristics. A statistical analysis of the user requests in the real traces is performed to identify the right distributions that can be used to model the workload model attributes.

Automated Performance Evaluation: In the traditional approach, multiple scripts have to be recorded to create different workload scenarios. This approach involves a lot of manual effort. In order to add new specifications for workload mix and new requests, new scripts need to be recorded and parameterized. Writing additional scripts for new requests may be complex and time consuming as inter-request dependencies need to be taken care of. In the automated approach, real traces are analysed to generate benchmark and workload models. Various workload scenarios can be created by changing the specifications of the workload model. New specifications for workload mix and new requests can be specified by making changes in the benchmark model. This approach is faster as compared to the traditional approach in which multiple virtual user scripts have to be recorded and parameterized to generate various workload scenarios. The benchmark and workload models drive the synthetic workload generator. The approach shown in Figure 11.2 automates the entire performance evaluation workflow right from capturing user behaviour into workload and benchmark models to generating synthetic workloads which have the same characteristics as real workloads.

Realistic Workloads: Traditional approaches which are based on manually generating virtual user scripts by interacting with a cloud application, are not able to generate synthetic workloads which have the same characteristics as real workloads. Although the traditional approaches allow creation of various workload scenarios using multiple recorded virtual user scripts, however, these workload scenarios are generally over simplifications of real-world scenarios in which a very large number of users may be simultaneously interacting with a cloud application. In the automated approach, since real traces from a cloud application are used to capture workload and application characteristics into workload and benchmark models, the generated synthetic work-loads have the same characteristics as real workloads. By statistical analysis of the user requests in the real traces, the automated approach is able to

identify the right distributions that can be used to model the workload model attributes such as think time, inter-session interval and session length.

Rapid Deployment Prototyping: Traditional approaches do now allow rapidly comparing various deployment architectures. Based on the performance evaluation results, the deployments have to be refined manually and additional virtual user scripts have to be generated with new deployments. In the automated approach, an architecture model captures the deployment configurations of multi-tier applications. With this approach complex deployments can be created rapidly, and a comparative performance analysis on various deployment configurations can be accomplished.

Types of Tests

There are various types of tests that can be done using the benchmarking tools described in

• **Baseline Tests**: Baseline tests are done to collect the performance metrics data of the entire application or a component of the application. The performance metrics data collected from baseline tests is used to compare various performance tuning changes which are subsequently made to the application or a component. Base line tests are usually done with a small number of users (eg. 1, 10 and 100) and the data on performance metrics such as response time, throughput, CPU, memory and network utilization of various servers, for instance. is collected. Whenever a change in the application is done, the baseline tests can be used to compare if the performance improves or degrades with the changes.

• **Load Tests**: Load tests evaluate the performance of the system with multiple users and workload levels that are encountered in the production phase. The number of users and workload mix are usually specified in the load test configuration.

Stress Tests: Stress tests load the application to a point where it breaks down. The workload levels in stress tests are set beyond what are anticipated in the production phase. These tests are done to determine how the application fails, the conditions in which the application fails and the metrics to monitor which can warn about impending failures under elevated workload levels.

Soak Tests: Soak tests involve subjecting the application to a fixed workload level for long periods of time. Soak tests help in determining the stability of the application under prolonged use and how the performance changes with time.

Deployment Prototyping

• Deployment prototyping can help in making deployment architecture design choices.

• By comparing performance of alternative deployment architectures, deployment prototyping can help in choosing the best and most cost-effective deployment architecture that can meet the application performance requirements.

• Deployment design is an iterative process that involves the following steps:

• **Deployment Design:**

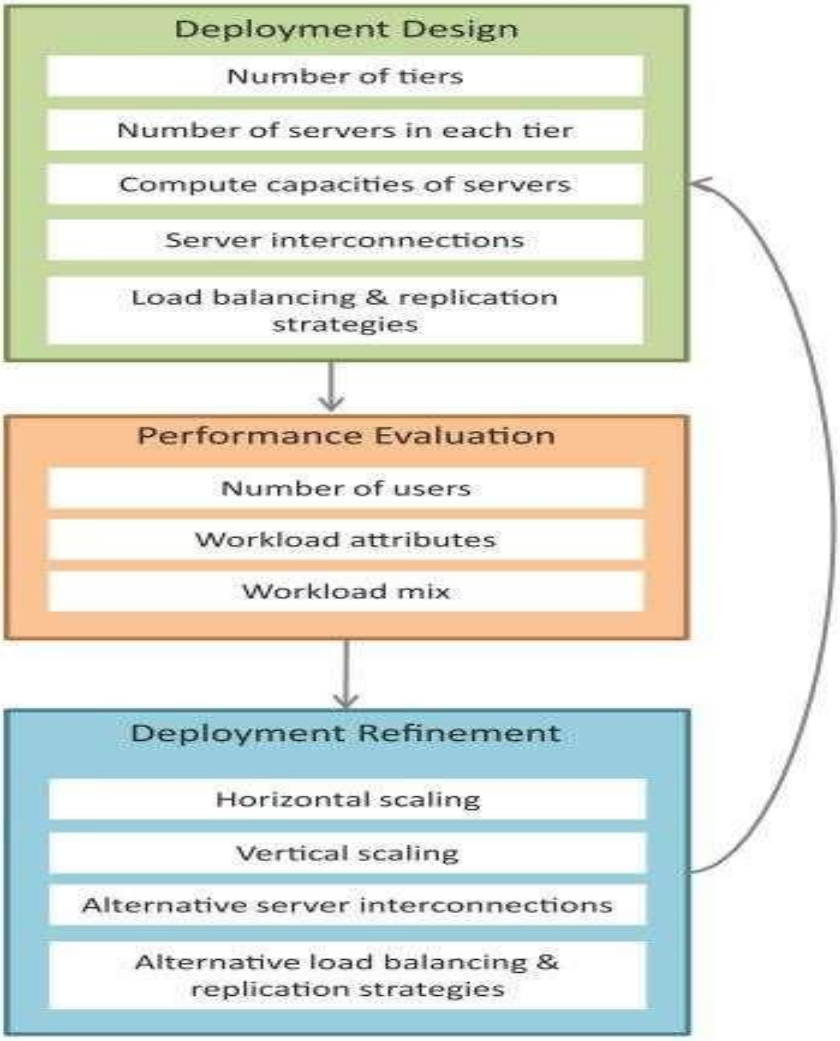
• Create the deployment with various tiers as specified in the deployment configuration and deploy the application.

• **Performance Evaluation:**

• Verify whether the application meets the performance requirements with the deployment.

• **Deployment Refinement:**

• Deployments are refined based on the performance evaluations. Various alternatives can exist in this step such as vertical scaling, horizontal scaling, for instance.



Performance Evaluation Workflow

Semi-Automated Workflow (Traditional Approach)

• In traditional approach to capture workload characteristics, a real user's interactions with a cloud application are first recorded as virtual user scripts.

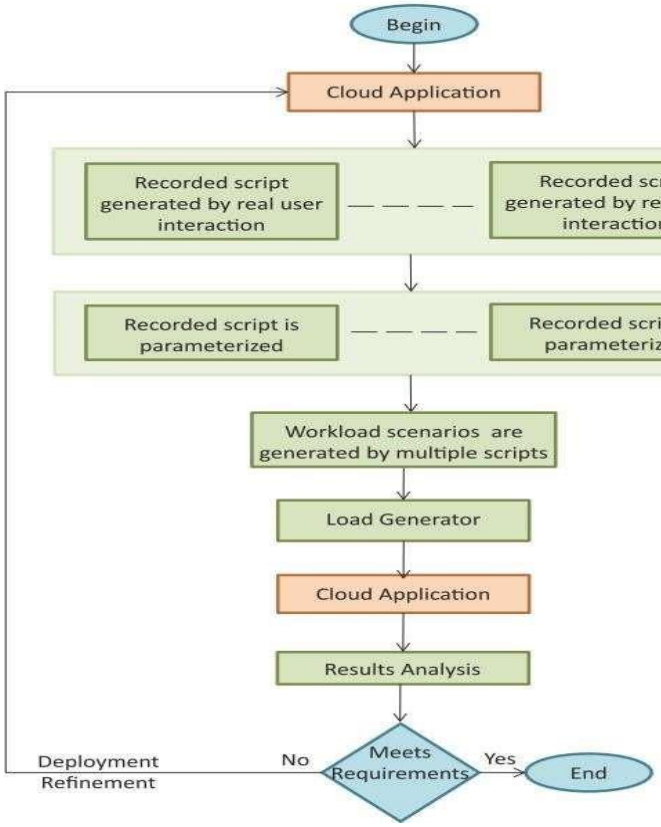
• The recorded virtual user scripts then are parameterized to account for randomness in application and workload parameters.

• Multiple scripts have to be recorded to create different workload scenarios. This approach involves a lot of manual effort.

• To add new specifications for workload, mix and new requests, new scripts need to be recorded and parameterized.

• Traditional approaches which are based on manually generating virtual user scripts by interacting with a cloud application, are not able to generate synthetic workloads which have the same characteristics as real workloads.

• Traditional approaches do now allow rapidly comparing various deployment architectures.



Performance Evaluation Workflow

Fully-Automated Workflow (Modern Approach)

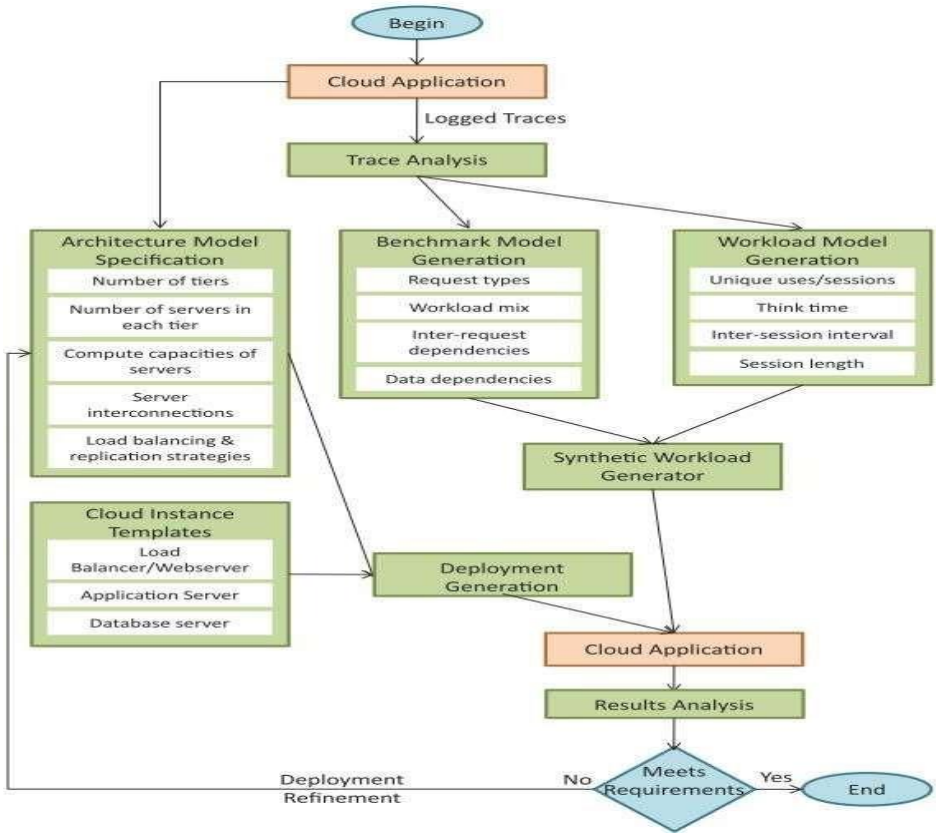
• In the automated approach real traces of a multi-tier application which are logged on web servers, application servers and database servers are analysed to generate benchmark and workload models that capture the cloud application and workload characteristics.

• A statistical analysis of the user requests in the real traces is performed to identify the right distributions that can be used to model the workload model attributes.

• Real traces are analysed to generate benchmark and workload models.

• Various workload scenarios can be created by changing the specifications of the workload model.

• Since real traces from a cloud application are used to capture workload and application characteristics into workload and benchmark models, the generated synthetic workloads have the same characteristics as real workloads.



Load Testing & Bottleneck Detection case Study

This section provides a case study of load testing and benchmarking an application with a multi-tier cloud deployment as shown in Figure 11.4. For this case study, the Rice University Bidding System [64] benchmark is used. RUBIS is an auction site prototype which has been loosely modelled after the internet auction website eBay. The RUBIS PHP application is deployed on the application servers and a MySQL database is setup on the database server. For measuring system statistics collected utility is used. collected is a daemon which collects system performance statistics periodically and provides mechanisms to store the values in a variety of ways, for example in RRD files, CSV files, for instance, collected gathers statistics about the system it is running on and stores this information. The collected statistics can then be used to find performance bottlenecks and predict future system load. collected comes with over 90 plugins including plugins for monitoring CPU, Disk, Memory, Network, for instance. Box 11.1 shows the commands for installing and configuring collected

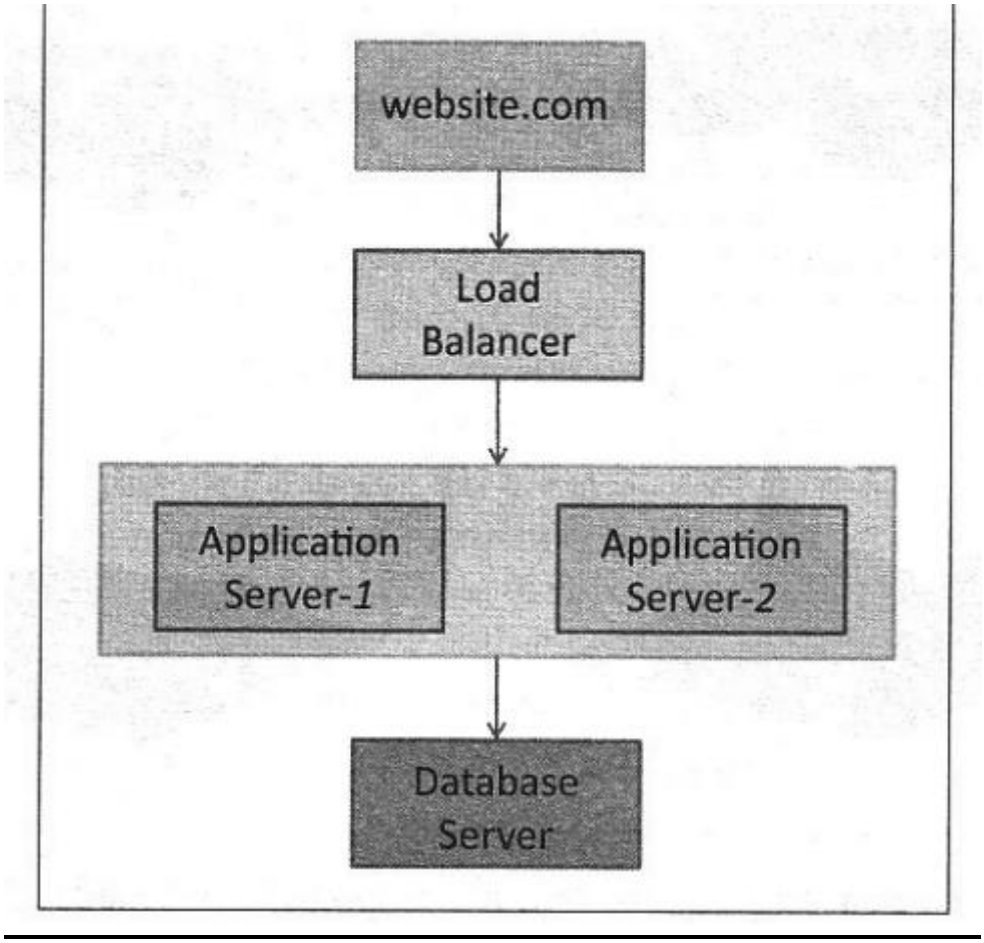


Figure 11.4: Multi-tier deployment architecture for sample application for case study

Box 11.2 shows a python program used for load testing the application. This program uses httperfpy package [66] which is a wrapper over httpef tool. The program in Box 11.2 shows how to perform multiple tests at different demanded request rates. In each step the demanded request rate is increased by 10 and various metrics are recorded such as the observed request rate, response time, for instance.

Figure 11.5 (a) shows the average throughput and response time. The observed throughput increases as demanded request rate increases. As more number of requests are served per second by the application, the response time also increases. The observed throughput saturates beyond a demanded request rate of 50 req/sec. Figure 11.5 (b) shows the CPU usage density of one of the application servers. This plot shows that the application server CPU is non-saturated resource. Figure 11.5 (c) shows the database server CPU usage density.

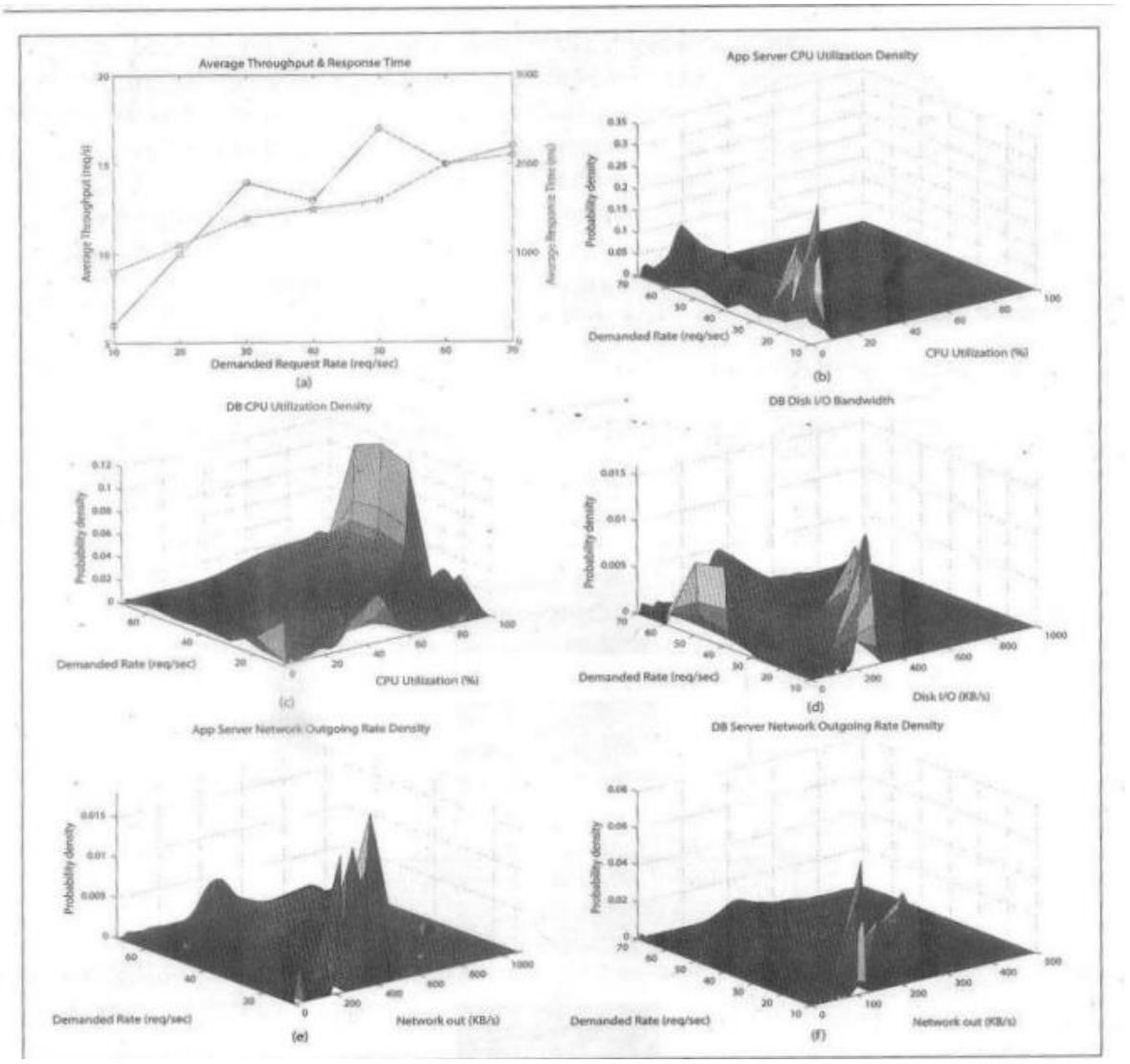


Figure 11.5: (a) Plot of average throughput and response time results obtained from load testing with httperf, (b) Application server-1 CPU utilization density, (c) Database server CPU utilization density, (d) Database server disk I/O bandwidth, (e) Application server-1 network outgoing rate density, (f) Database server network outgoing rate density.

From this density plot we observe taat the database CPU spends a large percentage of time at high utilization levels for demanded request rate more than 40 req/sec. Figure 11.5 (d) shows the density plot of the database disk I/O bandwidth. Figure 11.5 (c) shows the network out rate for one of the application servers and Figure 11.5 (f) shows the density plot of the network out rate for the database server. From this plot we observe a continuous saturation of the network out rate around 200 KB/s.

■ Box 11.2: Python program for load testing a cloud application using httperf

```
#!/usr/bin/env python

from httperfpy import Httperf

#Run in 5 steps and increment request rate by 10 in each step
for i in range(10,60,10):
    print 'Rate='+str(i)
    perf = Httperf(server="www.mywebsite.com",
            port=80,rate=i,num_calls=50, num_conns=400,timeout=60,
        send_buffer=4096,recv_buffer=16384)

    perf.parser = True

    results = perf.run()

    print 'Request rate per sec, Connection rate per sec,
        Avg reply rate, Max reply rate, Response Time'
    print results["request_rate_per_sec"] + ', '+ results["connection_rate_per_sec"] + ', '+
            results["reply_rate_avg"] + ',
        '+ results["reply_rate_max"]+', '+ results["reply_time_response"]
```

Analysing the plots in Figure 11.5 we observe that throughput continuously increases as the demanded request rate increases from 10 to 40 req/sec. Beyond 40 req/sec demanded request rate, we observe that throughput saturates, which is due to the high CPU utilization density of the database server CPU. From the analysis of density plots of various system resources we observe that the database CPU is a system bottleneck.

This case study demonstrates that by running the load tests with httperf several times with monotonically increasing request rates, the observed throughput saturates. Further, by analysing the CPU, disk I/O and network utilizations of the various servers in the application deployment, we were able to detect the bottleneck in the deployment. This bottleneck can be resolved by scaling up (vertical scaling) or scaling out (horizontal scaling) the database tier.

Hadoop bench marking case Study.

In Chapter 4 you have learned about the components of Hadoop, MapReduce job execution flow, Hadoop schedulers, and how to setup a Hadoop cluster. In this section you will learn how to configure different schedulers in Hadoop and benchmark the performance of a Hadoop cluster. For the experiments in this section, a Hadoop cluster comprising of two nodes (master and slave) is created. Fair Scheduler and Capacity Schedulers are configured for the experiments and their performance is benchmarked.

■ Box 11.3: Configuring Fair Scheduler - mapred-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>
```

```
<property>
<name>mapred.job.tracker</name>
<value>master:54311</value>
</property>

<property>
<name>mapred.jobtracker.taskScheduler</name>
</property>

</configuration>
```

▪ Box 11.4: Configuring Fair Scheduler - fair-scheduler.xml

```
<?xml version="1.0"?>
<allocations>
<pool name="user1">
<minMaps>5</minMaps>
<minReduces>5</minReduces>
<maxMaps>25</maxMaps>
<maxReduces>25</maxReduces>
<minSharePreemptionTimeout>300</minSharePreemptionTimeout>
</pool>
<user name="user1">
<maxRunningJobs>6</maxRunningJobs>
</user>
<userMaxJobsDefault>3</userMaxJobsDefault>
<fairSharePreemptionTimeout>600</fairSharePreemptionTimeout>

<pool name="user3">
<minMaps>5</minMaps>
<minReduces>5</minReduces>
<maxMaps>25</maxMaps>
<maxReduces>25</maxReduces>
<minSharePreemptionTimeout>300</minSharePreemptionTimeout>
</pool>
<user name="user3">
<maxRunningJobs>6</maxRunningJobs>
</user>
</allocations>
```

Box 11.4 shows the fair-scheduler.xml file that is used to specify the allocations for each pool, as well as the per-pool and per-user limits on the number of running jobs. The allocations shown in Box 11.4 consist of three pools - user1, user2 and user3, each having a guarantee of 5 map slots and 5 reduce slots. The maximum number of map and reduce slots for each pool are fixed at 25. The maximum number of running jobs per user is fixed to 6 for all users.

The allocations for the pool and the user and pool running job limits can be modified at runtime by editing the allocation configuration file. This file is reloaded by the scheduler 10-15 seconds after the file is modified. With the Fair Scheduler configured, the Hadoop cluster is launched. The Fair Scheduler administration page can be viewed at the URL:

http://<JobTracker-URL>:50030/scheduler

The administration page of the Fair Scheduler shows the pools and their allocations.

The status of the MapReduce jobs submitted to different pools can also be viewed on this page. For each pool, the minimum and maximum share for map and reduce slots, number of running map and reduce tasks, and the fair share for map and reduce tasks can be viewed. The fair share denotes the average number of task slots that a job should have at any given time according to fair sharing. Now let us move on to the Capacity Scheduler. Box 11.5 shows the mapred-site.xml configuration file used for configuring the Capacity Scheduler. Box 11.6 shows the capacity-scheduler.xml configuration file. In this file multiple queues are defined to which the users can submit jobs with the Capacity Scheduler. Properties for each queue are specified in this file. For example, to specify a property for a queue, the property name is used as - mapred.capacity-scheduler.queue.<queue-name>.<property-name>.

■ **Box 11.5: Configuring Capacity Scheduler - mapred-site.xml**

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>
<property>
<name>mapred.job.tracker</name>
<value>master:54311</value>
</property>

<property>
<name>mapred.jobtracker.taskScheduler</name>
<value>org.apache.hadoop.mapred.CapacityTaskScheduler</value>
</property>

<property>
<name>mapred.queue.names</name>
<value>userA,userB,userC</value>
</property>

</configuration>
```

■ **Box 11.6: Configuring Capacity Scheduler - capacity-scheduler.xml**

```
<?xml version="1.0"?>
<configuration>

<property>
<name>mapred.capacity-scheduler.maximum-system-jobs</name>
<value>3000</value>
<description>Maximum number of jobs in the system which can be initialized,
concurrently, by the CapacityScheduler.
</description>
</property>
```

```
<property>
<name>mapred.capacity-scheduler.queue.default.capacity</name>
<value>10</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.default.supports-priority</name>

    <value>false</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.default.minimum-user-limit-percent
</name>
<value>10</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.default.user-limit-factor</name>

    <value>10</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.default.maximum-initialized-active-tas ks
</name>
<value>200000</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.default.maximum-initialized-active-tas ks-
per-user</name>
<value>100000</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.default.init-accept-jobs-factor
</nam e>
<value>100</value>
</property>

<property>
<name>mapred.capacity-scheduler.queue.userA.capacity </name>
<value>30</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.userA.supports-priority </name>
<value>false</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.userA.minimum-user-limit-percent
</na me>
```

```
<value>20</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.userA.user-limit-factor </name>
<value>10</value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.userA.maximum-initialized-active-tasks
</name>
<value>200000 </value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.userA.maximum-initialized-active-tasks
-per-user</name>
<value>100000 </value>
</property>
<property>

    <name>mapred.capacity-scheduler.queue.userA.init-accept-jobs-factor
</name>
<value>100</value>
</property>

:
</configuration>
```

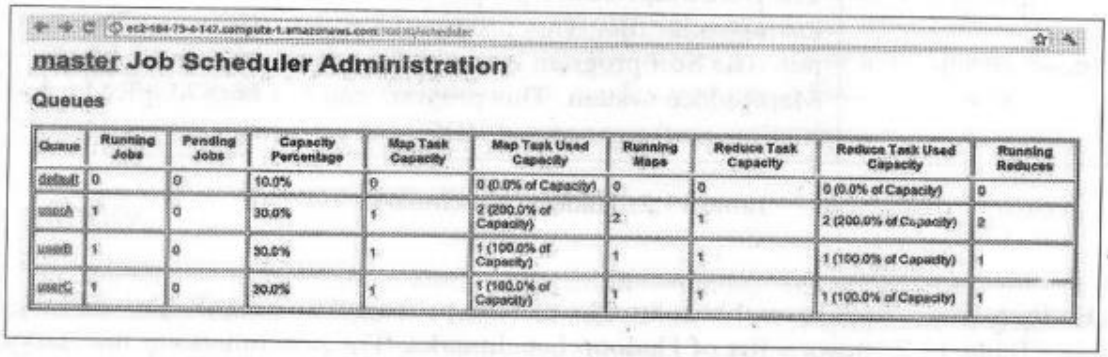


Figure 11.6: Hadoop Capacity Scheduler administration page

With the Capacity Scheduler configured, the Hadoop cluster is launched. The Capacity Scheduler administration page can be viewed at the URL:

http://<JobTracker-URL>:50030/scheduler, as shown in Figure 11.6. The administration page of the Capacity Scheduler shows the queues and their allocated capacities, map and reduce task capacities used and number of running map and reduce tasks.

For monitoring the performance of the nodes in the Hadoop cluster you can use perfor-mance monitoring tools such as collected, nmon, for instance. For the experiments shown in this section, the collected tool is used.

Benchmark	Description
TestDFSIO	TestDFSIO tests the I/O performance of HDFS. This benchmark is useful for stress testing HDFS. TestDFSIO uses a MapReduce job to read or write files in parallel. Each file is read or written in a separate map task, and the output of the map is used for collecting statistics relating to the file just processed. The reduce phase accumulates the statistics to produce a summary.
MRBench	MRBench runs a small job a number of times. MRBench is a complimentary benchmark to the Sort benchmark. MRBench checks whether small job runs are responsive and running efficiently on the cluster.
NNBench	NNBench is useful for load testing the NameNode hardware and configuration. This benchmark generates a lot of HDFS-related requests with normally very small payloads for the sole purpose of putting a high HDFS management stress on the NameNode. The benchmark can simulate requests for creating, reading, renaming and deleting files on HDFS.
Gridmix	GridMix is a suite of benchmarks that model that characteristics of realistic workloads for submitting a mix of synthetic jobs. GridMix requires a MapReduce job trace describing the job mix for a given cluster, and input data from which the synthetic jobs will read bytes.
Sort	The Sort MapReduce program is included in the hadoop-*-examples.jar file. This program does a partial sort of its input. The Sort program is useful for benchmarking the whole MapReduce system. This program can test both MapReduce runtime performance and HDFS I/O performance.

Hadoop comes with several benchmarks that are packaged in the JAR file - hadoop-*-test.jar. Table 11.2 shows a list of Hadoop benchmarks. For benchmarking the Hadoop cluster, the Sort MapReduce program is used in this section. The Sort program does a partial sort of its input. This is useful for benchmarking the whole MapReduce system. This program can test both MapReduce runtime performance and HDFS I/O performance.

Box 11.7 lists the commands used for running the sort benchmark. The first step in benchmarking with sort is to generate input data for the sort program. The RandomWriter MapReduce program is used for generating random data. This program runs a MapReduce job with 10 maps per node by default, and each map generates approximately 10 GB of random binary data, with key and values of various sizes. The number of maps per node and the bytes per map can be changed by setting the properties test.randomwriter.maps_per_host and test.randomwrite.bytes_per_map.

■ Box 11.7: Running Sort Benchmark

```
# Submitting job from a particular user with Fair Scheduler configured:

$sudo -u userA bin/hadoop jar hadoop-examples-1.0.4.jar randomwriter -D
test.randomwrite.bytes_per_map=1000000 random-data

$ sudo -u userA bin/hadoop jar hadoop-examples-1.0.4.jar sort random-data1
sorted-data

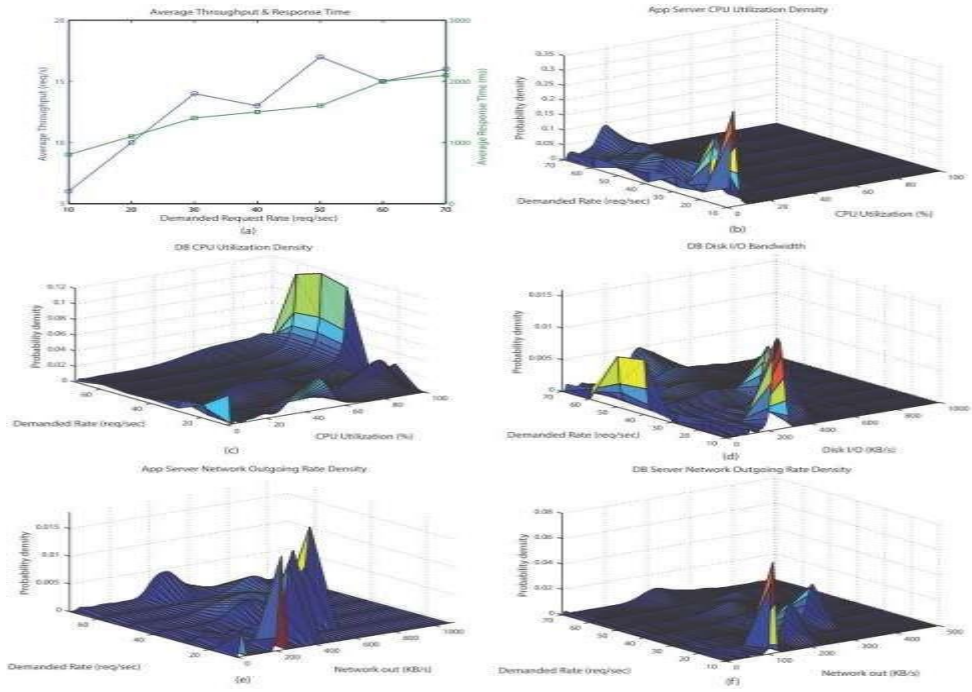
#Submitting job to a particular queue of Capacity Scheduler:

$bin/hadoop jar hadoop-examples-1.0.4.jar randomwriter -D
test.randomwrite.bytes_per_map=1000000 -D mapred.job.queue.name=userA
random-data

$bin/hadoop jar hadoop-examples-1.0.4.jar sort -D mapred.job.queue.name=userA
random-data sorted-data
```

After running the benchmark, the statistics of the Hadoop nodes collected by the collected performance monitoring tool are analysed. Figures 9.7 (a) and (b) show the idle CPU for the master and slave nodes for a benchmark run. From these plots it is observed that for both nodes, high CPU (d) show the disk I/O for the master and slave nodes. Figure 9.8 (a) and (b) show the network I/O for the master and slave nodes. Figure 9.8 (c) and (d) show the memory used for the master and slave nodes. Analysis of statistics collected after a benchmark run can help locate the bottlenecks. The performance can then be tuned by changing properties such as the allocations for pools in the case of the Fair Scheduler and capacities of the queues in case of the Capacity Scheduler. Table 11.3 lists the properties of Hadoop that can be configured for tuning the performance of a Hadoop cluster

Benchmarking Case Study



• Fig (a) shows the average throughput and response time. The observed throughput increases as demanded request rate increases. As more number of requests are served per second by the application, the response time also increases. The observed throughput saturates beyond a demanded request rate of 50 req/sec.

• Fig (b) shows the CPU usage density of one of the application servers. This plot shows that the application server CPU is non-saturated resource.

• Fig (c) shows the database server CPU usage density. From this density plot we observe that the database CPU spends a large percentage of time at high utilization levels for demanded request rate more than 40 req/sec.

• Fig (d) shows the density plot of the database disk I/O bandwidth.

• Fig (e) shows the network out rate for one of the application servers

• Fig (f) shows the density plot of the network out rate for the database server. From this plot we observe a continuous saturation of the network out rate around 200 KB/s.

• Analysis

• Throughput continuously increases as the demanded request rate increases from 10 to 40 req/sec. Beyond 40 req/sec demanded request rate, we observe that throughput saturates, which is due to the high CPU utilization density of the database server CPU. From the analysis of density plots of various system resources we observe that the database CPU is a system bottleneck.

UNIT-5

Cloud Security:
Introduction, CSA Cloud Security Architecture, Authentication, Authorization, Identity& Access Management, Data Security, Key Management, Auditing.
Cloud for Industry, Healthcare& Education:
Cloud Computing for Healthcare, Cloud computing for Energy Systems, Cloud Computing for Transportation Systems, Cloud Computing for Manufacturing Industry, Cloud computing for Education.
Migrating into a Cloud: Introduction, Broad Approaches to migrating in to the cloud, the seven–step model of migration into a cloud.

Introduction:

More, and more organizations are moving their applications and associated data to cloud to reduce costs and reduce the operational and. maintenance overheads, and one of the important considerations is that of security of the daita.in the cloud. Most cloud service providers implement advanced security features similar to those that exist in in-house IT environments.

However, due the out sourced nature .of' the cloud, resource pooling and multi-tenanted; architectures, security remains an important .concern in adoption of cloud computing. :In addition to the traditional vulnerabilities that exist for web app1icatons, the

cloud. applications have additional vulnerabilities because of the shared usage of resources and virtualized resources. Key Security challenges for. cloud applications include:

1)Authentication
2)Authorization
3)Security of Data at Rest
4)Security of Data at Motion
5)Data Integrity
6)Auditing

1) Authentication:-

Authentication refers to digitally confirming the identity of the entity requesting access to some protected information.

In a traditional in-house IT Environment authentication polices are under the control of the organization. However, in cloud computing environments, where applications and data are accessed over the internet, the complexity of digital authentication mechanisms increases rapidly.

2) Authorization:-

Authorization refers to digitally specifying the access rights to the protected resources using access policies. In a traditiona1 in-house IT environment, the access policies are controlled by the organization and can.be altered at their convenience.

An organization, for example, can provide different access policies for different departments.

Authorization in a cloud computing environment requires the use of the cloud service providers services for specifying the access policies.

3) Security of Data at Rest:-

Due to the multi-tenant environments used in the cloud, the application and database servers of different applications belonging to different organizations can be provisioned side-by-side increasing the complexity of securing the. data.

Appropriate separation mechanisms are required to ensure the isolation between applications and. data from different organizations.

4) Security of Data at Motion:-

In traditional in-house IT environments, all the data exchanged between the applications and users remains within the organization's control and geographical boundaries. Organiza
believe that they have complete visibility of all the data exchanged and control the IT infrastructure. With the adoption of tire cloud model, the applications and the data are moved out of the in-house IT infrastructure to the cloud provider. In such a scenario, organizations have to access their applications with the data moving in and out of the cloud over the internet.
Therefore, appropriate security mechanisms are required to ensure the security of data in, and while in, motion.

5) Data Integrity: -

Data integrity ensures that the data is not altered in an unauthorized manner after it is created, transmitted or stored.

Due to the, outsourcing of data storage, in cloud computing environments, ensuring integrity of data is important. Appropriate mechanisms are required för detecting accidental and/or intentional changes in the data.

6) Auditing

Auditing is very important for applications deployed in cloud computing environments. In traditional in-house IT environments, organizations have complete visibility of their applications and accesses to the protected information.

For cloud applications appropriate auditing mechanisms are required to get visibility into the application, data accesses and actions performed by the app1ïcation users, including mobile users and devices such as. wireless laptops and smartphones.

CSA Cloud Security Architecture

Introduction: -

The Cloud Security Alliance (CSA) provides a Trusted Cloud Initiative (TCI) Reference Architecture which is. a methodology and a set of tools that enab1e cloud application developers and security architects to assess where their internal IT and their cloud providers are in terms of security capabilities, and to plan a roadmap to meet the security needs of their business.

The Security and Risk Management (SRM) domain within the TCI Reference Architecture provides the core components of an organization’s information security program to safeguard assets and detect, assess, and monitor risks inherent in operating activities.

The sub-domains of SRM include:

1) Governance, Risk Management and Compliance:-

This sub-domain deals with the identification and implementation of the appropriate organizational structures, processes, and controls to maintain effective information security governance, risk management and compliance.

2) Information Security Management:-

This sub-domain deals with the implementation of appropriate measurements (such as capability maturity models, capability mapping models, security architectures roadmaps and risk portfolios) in order to minimize or eliminate the impact that security related threats and vulnerabilities might have on an organization

3) Privilege Management Infrastructure:-

The objective of this sub-domain is to ensure that users have access and privileges required to execute their duties and responsibilities with Identity and Access Management (IAM) unctions such as identity management, authentication services, authorization services, and privilege usage management.

4) Threat and Vulnerability Management:-

This sub-domain deals with core security such. as vulnerability management, threat management, compliance testing, and penetration testing.

5) Infrastructure Protection Service:-

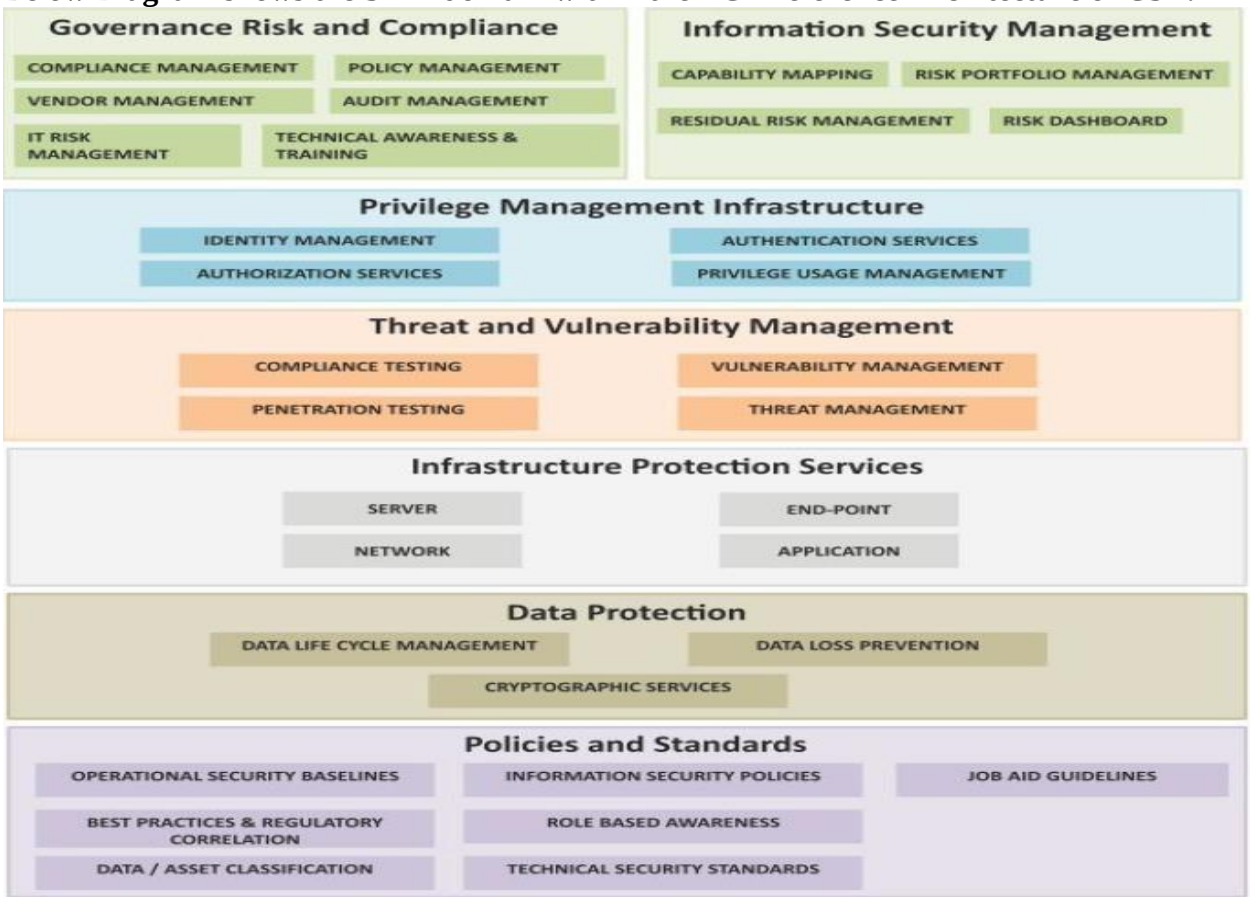
This objective of this sub-domain is to secure Server, End-Point, Network and Application layers.

6) Data Protection

This sub-domain deals with data lifecycle management, data leakage prevention, intellectual property protection with digital rights management, and cryptographic services such as key management and PKI/symmetric encryption.

7) Polices and Standards: - Security policies and standards are derived from risk-based business requirements and exist at a number of different levels including Information Security policy, Physical Security Policy, Business Continuity Policy, Infrastructure Security Po1icies,, Application Security Policies as well as the over-arching Business Operational Risk Management Policy.

Below Diagram shows the SRM domain within the TCI Reference Architecture of CSÄ.



Authenticaiton

Introduction: -

- Authentication refers to confirming the digital identity of the entity requesting access to some protected information.
- The process of authentication involves, but is not limited to, validating at least one factor of identification of the entity to be authenticated.A factor can be something the entity or the user knows (password or pin), something the user has (such as a smart card), or something that can uniquely identify the user (such as fingerprints).
- In multifactor authentication more than one of these factors are used for authentication.
- There are various mechanisms for authentication including:
  -SSO
  -OTP

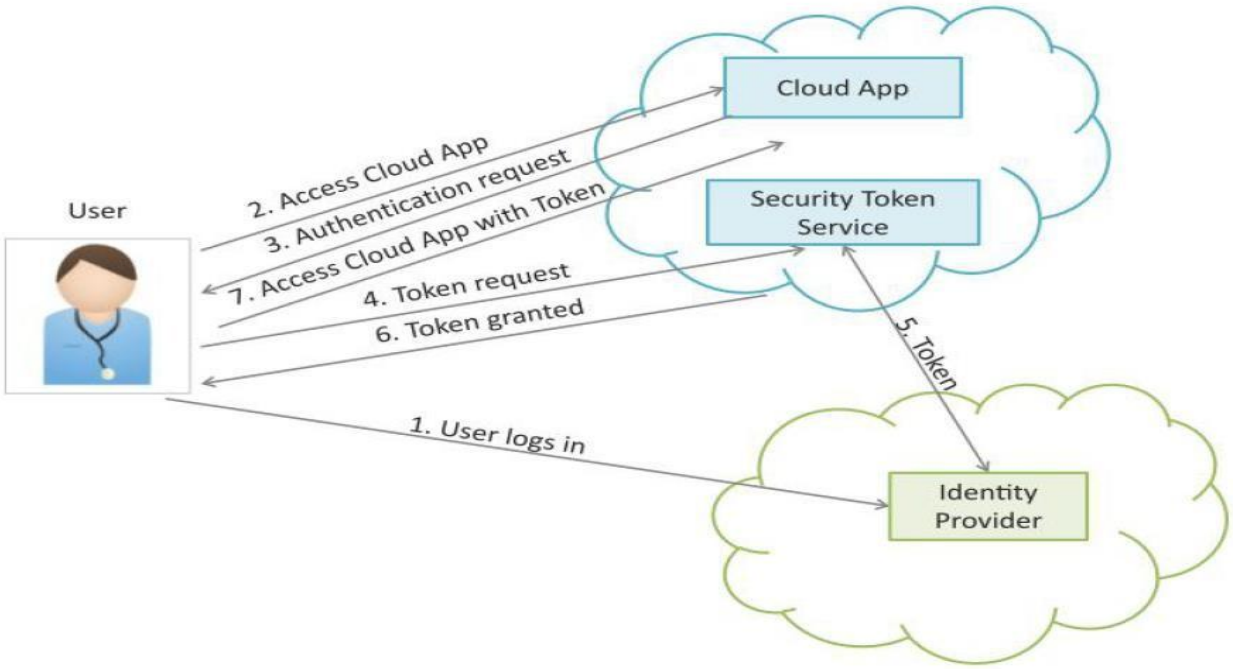
1)Single Sign On (SSO):-

- Single Sign-on (SSO) enables users to access multiple systems or applications after signing in only once, for the first time.
- When a user signs in, the user identity is recognized and there is no need to sign in again and again to access related systems or applications.
- Since different systems or applications may be internally using different authentication mechanisms, SSO upon receiving initial credential translates to different credentials for different systems or applications.
- The benefit of using SSO is that it reduces human error and saves time spent in authenticating with different systems or applications for the same identity.
- There are different implementation mechanisms:
  - SAML-Token
  - Kerberos

a) SAML Token: -

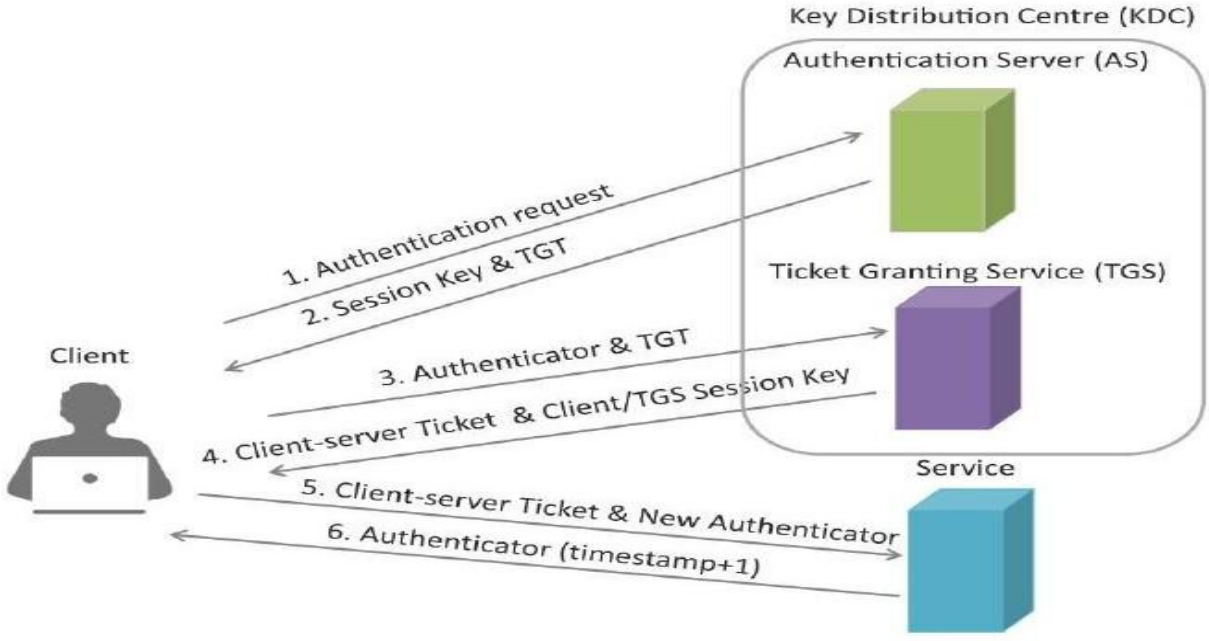
- Security Assertion Markup Language (SAML) is an XML-based open standard data format for exchanging security information (authentication and authorization data) between an identity provider and a service provider.
- SAML-token based SSO authentication
  - When a user tries to access the cloud application, a SAML request is generated and the user is redirected to the identity provider.
  - The identity provider parses the SAML request and authenticates the user. A SAML token is returned to the user, who then accesses the cloud application with the token.
  - SAML prevents man-in-the-middle and replay attacks by requiring the use of SSL encryption when transmitting assertions and messages.
  - SAML also provides a digital signature mechanism that enables the assertion to have a validity time range to prevent replay attacks.

The below diagram shows the Authentication flow for a Cloud Application using SAML SSO



b) Kerberos: -

- Kerberos is an open authentication protocol that was developed At MIT.
- Kerberos uses tickets for authenticating client to a service that communicate over an un-secure network.
- Kerberos provides mutual authentication, i.e. both the client and the server authenticate with each other.
- Below diagram shown Kerberos Authentication Flow:



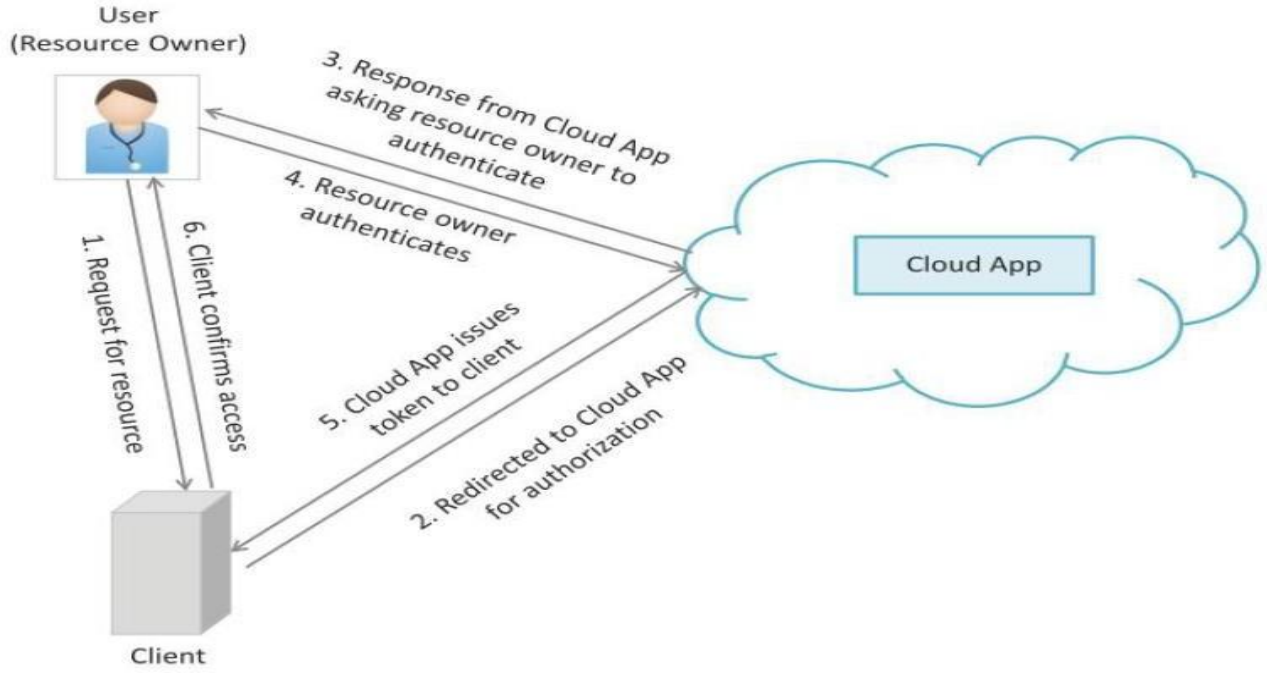
2) One Time Password (OTP) :-

- One time password is another authentication mechanism that uses passwords which are valid for single use only for a single transaction or session.
- Authentication mechanism based on OTP tokens are more secure because they are not vulnerable to replay attacks.
- Text messaging (SMS) is the most common delivery mode for OTP tokens.
- The most common approach for generating OTP tokens is time synchronization.
- Time-based OTP algorithm (TOTP) is a popular time synchronization based algorithm for generating OTPs.

Authorization

Introduction: -

- Authorization refers to specifying the access rights to the protected resources using access policies.
- OAuth:
  - OAuth is an open standard for authorization that allows resource owners to share their private resources stored on one site with another site without handing out the credentials.
  - In the OAuth model, an application (which is not the resource owner) requests access to resources controlled by the resource owner (but hosted by the server).
  - The resource owner grants permission to access the resources in the form of a token and matching shared-secret.
  - Tokens make it unnecessary for the resource owner to share its credentials with the application.
  - Tokens can be issued with a restricted scope and limited lifetime, and revoked independently.

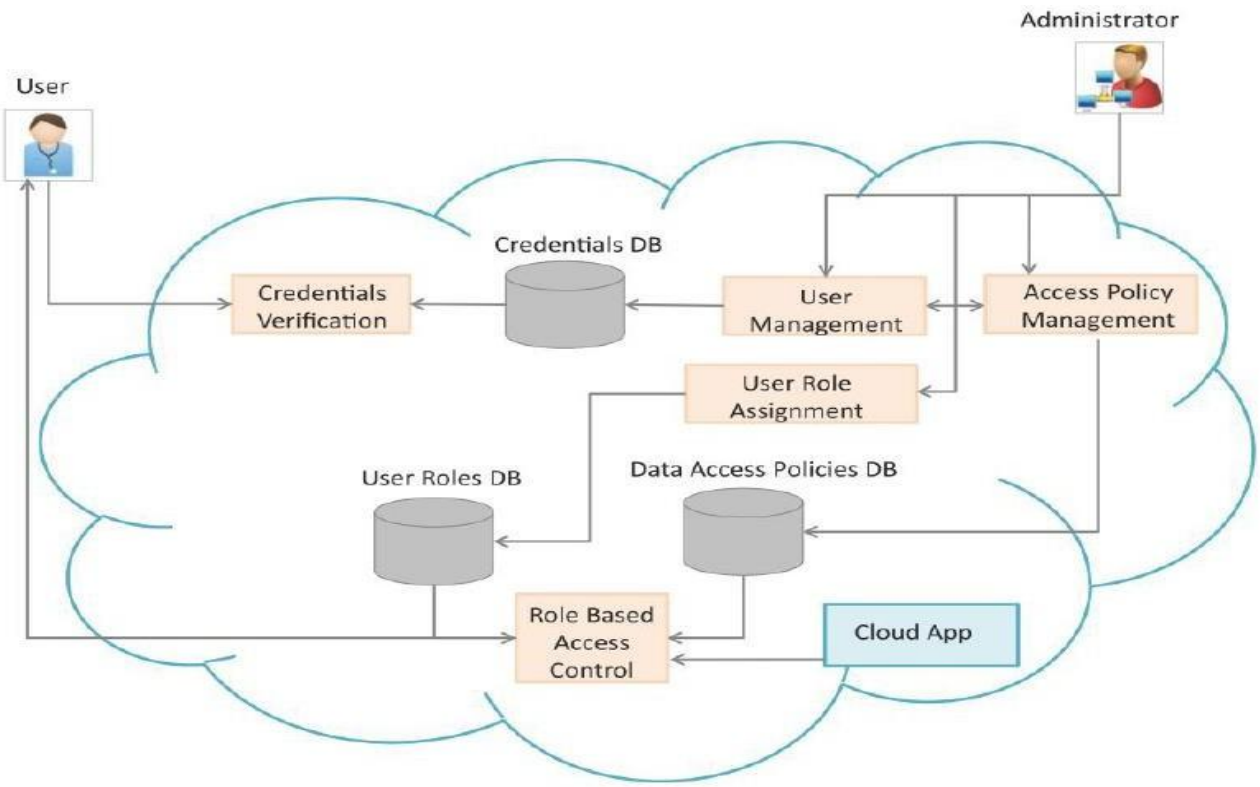


Identity and Access Management

Introduction: -

- Identity management provides consistent methods for digitally identifying persons and maintaining associated identity attributes for the users across multiple organizations.
- Access management deals with user privileges.
- Identity and access management deal with user identities, their authentication, authorization and access policies.
- **Federated Identity Management**
  - Federated identity management allows users of one domain to securely access data or systems of another domain seamlessly without the need for maintaining identity information separately for multiple domains.
  - Federation is enabled through the use single sign-on mechanisms such as SAML token and Kerberos.
- **Role-based access control**
  - Used for restricting access to confidential information to authorized users.
  - These access control policies allow defining different roles for different users.

Below Diagram shows an example of the Role Based Access Control Framework in the cloud.



- Securing data in the cloud is critical for cloud applications as the data flows from applications to storage and vice versa. Cloud applications deal with both data at test and data in motion.

- There are various types of threats that can exist for data in the cloud such as denial of service, replay attacks, man-in the -middle attacks, unauthorized access/modification, etc.

1) Securing Data at Rest: -

- Data at rest is the data that is stored in database in the form of tables/records, files on a file server or raw data on a distributed storage or storage area network (SAN).

- Data at rest is secured by encryption.

- Encryption is the process of converting data from its original form (i.e.,plaintext) to a scrambled form (ciphertext) that is unintelligible. Decryption converts data from ciphertext to plaintext.

  - **Encryption can be of two types:**
    - Symmetric Encryption (symmetric-key algorithms)
    - Asymmetric Encryption (public-key algorithms)

a) Symmetric Encryption:

- Symmetric encryption uses the same secret key for both encryption and decryption.

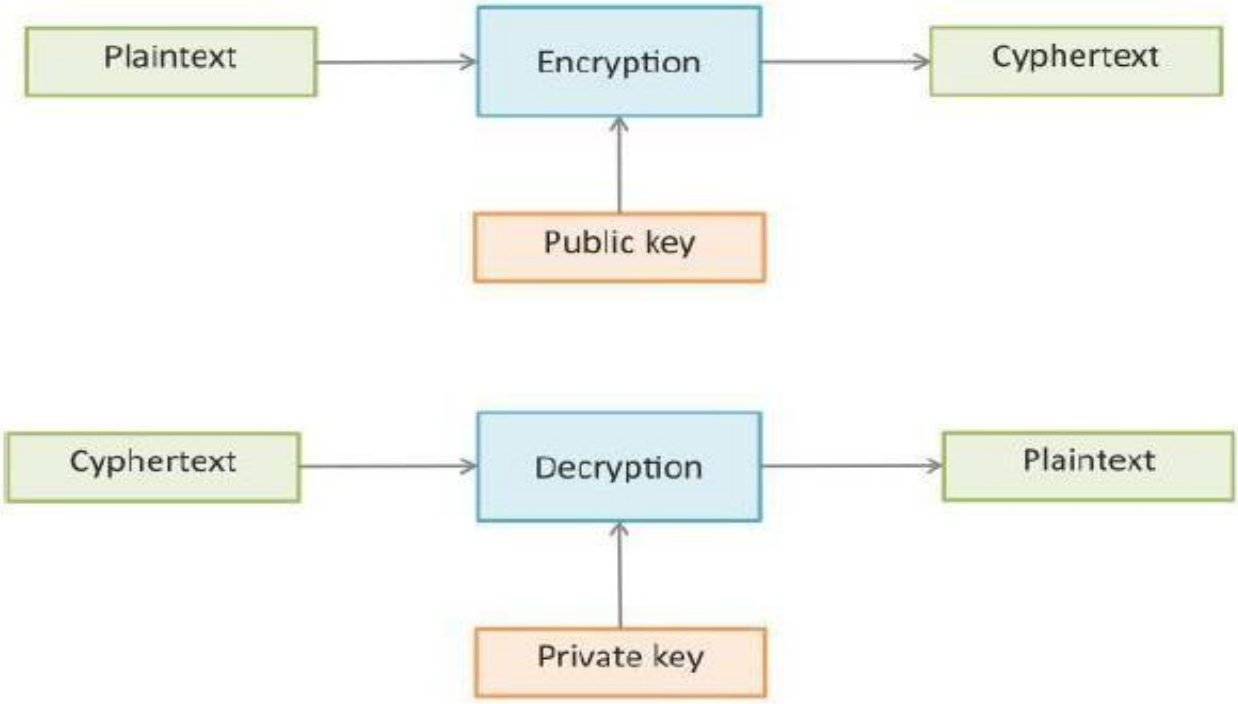
- The secret key is shared between the sender and the receiver.

- Symmetric encryption is best suited for securing data at rest since the data is accessed by known entities from known locations.

- Popular symmetric encryption algorithms include:
  - Advanced Encryption Standard (AES)
  - Twofish
  - Blowfish
  - Triple Data Encryption Standard (3DES)
  - Serpent
  - RC6
  - MARS

b) Asymmetric Encryption:

- Asymmetric encryption uses two keys, one for encryption (public key) and other for decryption (private key).



- The two keys are linked to each other such that one key encrypts plaintext to ciphertext and other decrypts ciphertext back to plaintext.
- Public key can be shared or published while the private key is known only to the user.
- Asymmetric encryption is best suited for securing data that is exchanged between two parties where symmetric encryption can be unsafe because the secret key has to be exchanged between the parties and anyone who manages to obtain the secret key can decrypt the data.
- In asymmetric encryption a separate key is used for decryption which is kept private.

c) Encryption Levels:

Encryption can be performed at various levels described as follows:

- Application
- Host
- Network
- Device

Application:

- Application-level encryption involves encrypting application data right at the point where it originates i.e. within the application.
- Application-level encryption provides security at the level of both the operating system and from other applications
- An application encrypts all data generated in the application before it flows to the lower levels and presents decrypted data to the user.

Host:

- In host-level encryption, encryption is performed at the file-level for all applications running on the host.
- Host level encryption can be done in software in which case additional computational resource is required for encryption or it can be performed with specialized hardware such as a cryptographic accelerator card.

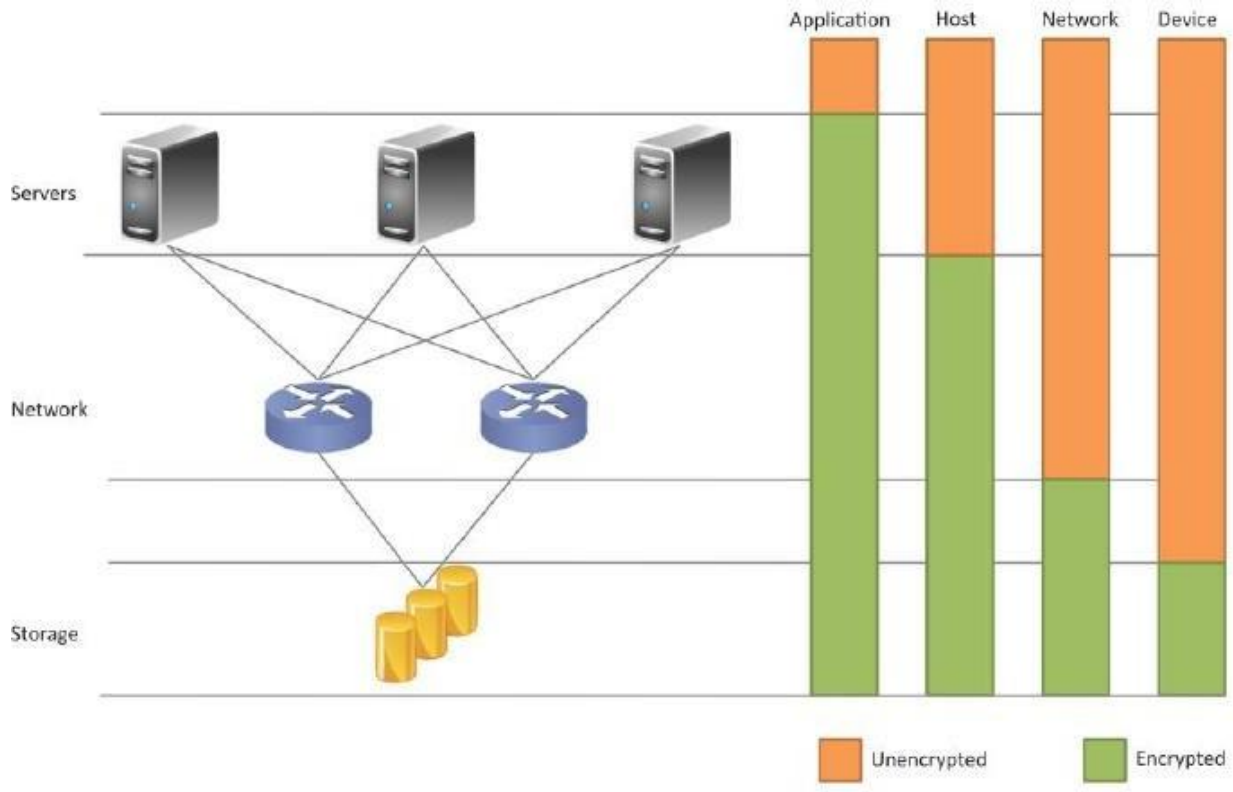
Network:

- Network-level encryption is best suited for cases where the threats to data are at the network or storage level and not at the application or host level.
- Network-level encryption is performed when moving the data form a creation point to its destination using a specialized hardware that encrypts all incoming data in real-time.

Device:

- Device-level encryption is performed on a disk controller or a storage server
- Device level encryption is easy to implement and is best suited for cases where the primary concern about data security is to protect data residing on storage media

Below Diagram shows various Encryption levels:



2)Securing Data in Motion: -

- Securing data in motion, i.e., when the data flows between a client and a server over a potentially insecure network, is important to ensure data confidentiality and integrity.

Data confidentiality:

Data Confidentiality means limiting the access to data so that only authorized recipients can
access it.

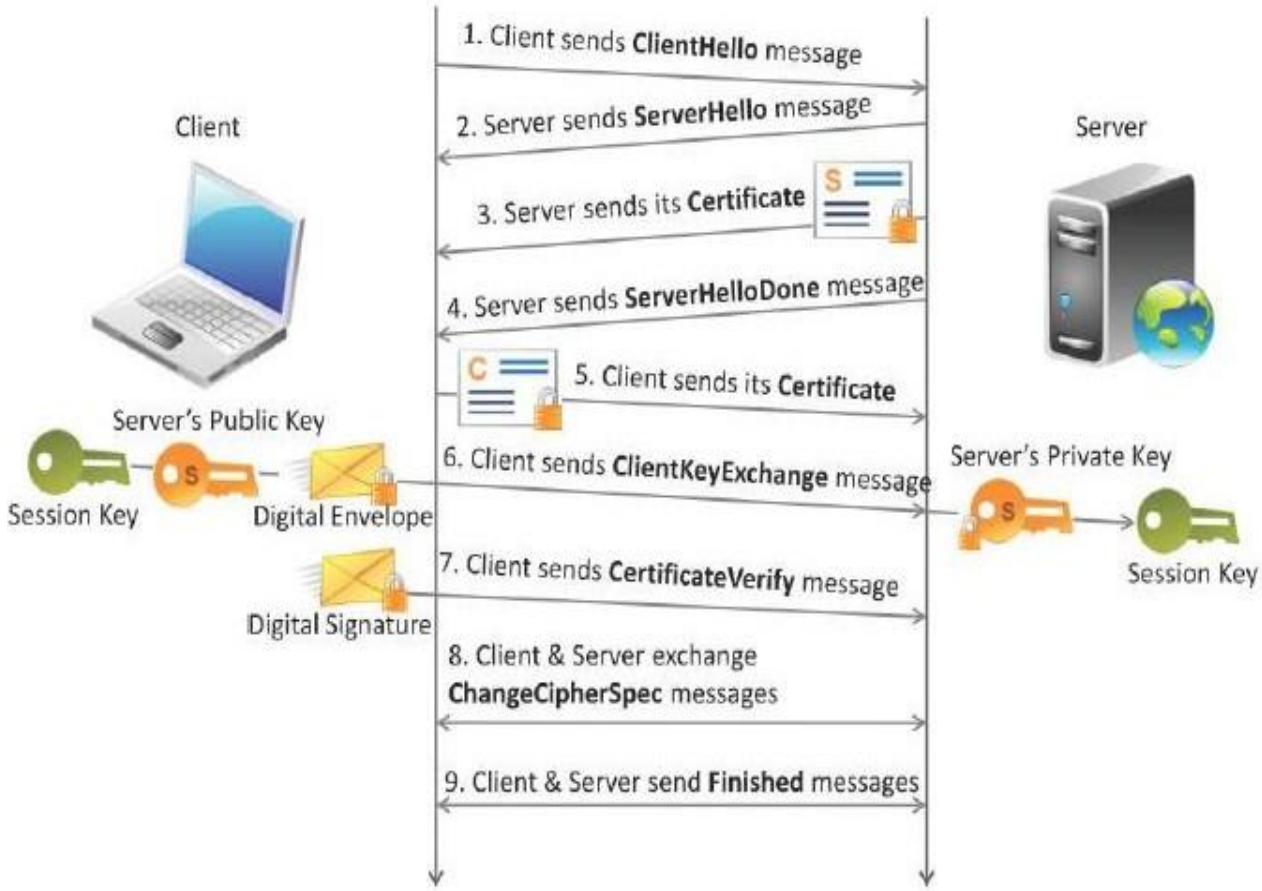
Data integrity:

Data integrity means that the data remains unchanged when moving from sender to receiver.

Data integrity ensures that the data is not altered in an unauthorized manner after it is created, transmitted or stored.

Transport Layer Security (TLS) and Secure Socket Layer (SSL) are the mechanisms used for securing data in motion.

Below diagram shows the TLS Handshake protocol:



TLS and SSL are used to encrypt web traffic using Hypertext Transfer Protocol (HTTP).

TLS and SSL use asymmetric cryptography for authentication of key exchange, symmetric encryption for confidentiality and message authentication codes for message integrity.

Key Management

Introduction: -

Management of encryption keys is critical to ensure security of encrypted data. The key management lifecycle involves different phases including:

Creation: Creation of keys is the first step in the key management lifecycle. Keys must be created in a secure environment and must have adequate strength. It is recommended to encrypt the keys themselves. with a separate master key.

Backup: Backup of keys must be made before putting them into production because in the event of loss of keys, all encrypted data can become useless.

Deployment: In this phase the new key is deployed for encrypting the data. Deployment of a new key involves re-keying existing data.

Monitoring: After a key has been deployed, monitoring the performance of the encryption environment is done to ensure that the key has been deployed correctly.

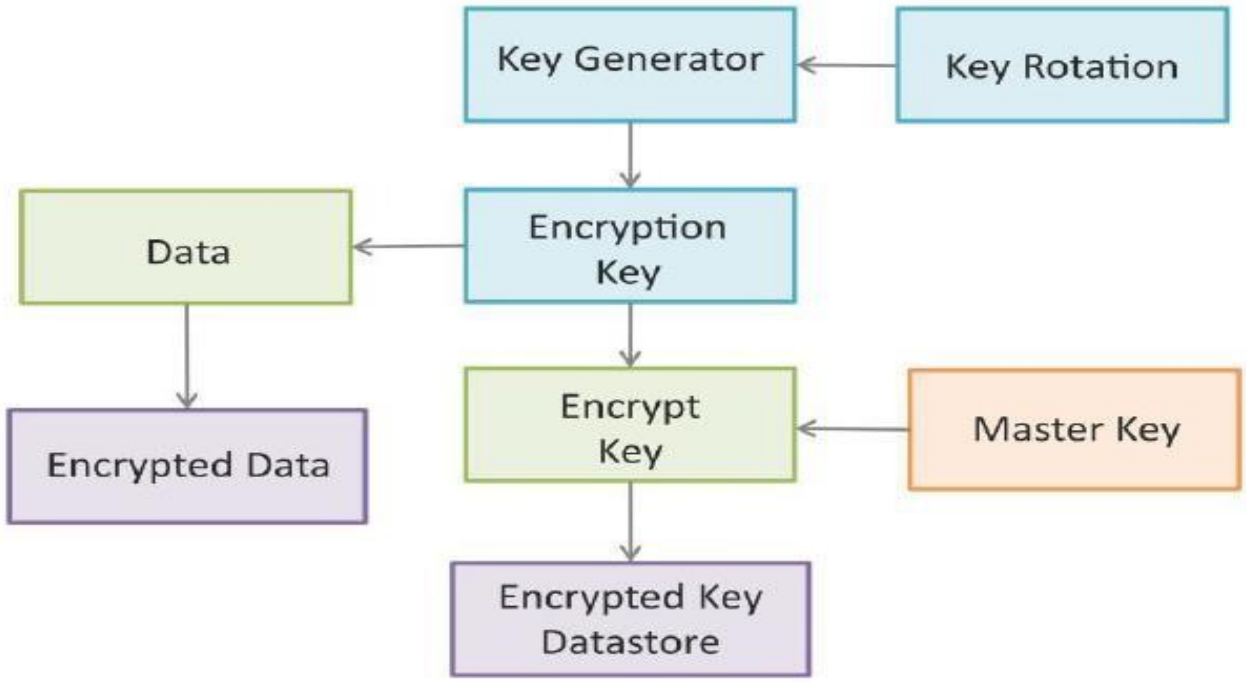
Rotation: Key rotation involves creating a new key and re-encrypting all data with the new key.

Expiration: Key expiration phase begins after the key rotation is complete. It is recommended to complete the key rotation process before the expiry of the existing key.

Archival: Archival is the phase before the key is finally destroyed. It is recommended to archive old keys for some period of time tt› account for scenarios where there is still some data in the system that is encrypted with the old key.

Destruction: Expired keys are finally destroyed after ensuring that there is no data encrypted with the expired keys.

Below diagram shows an example of the key Management approach:



Auditing

Introduction: -

- Auditing is mandated by most data security regulations.
- Auditing requires that all read and write accesses to data be logged.
- Logs can include the user involved, type of access, timestamp, actions performed and records accessed.
- The main purpose of auditing is to find security breaches, so that necessary changes can be made in the application and deployment to prevent a further security breach.

Objectives:

The objectives of auditing include:

- Verify efficiency and compliance of identity and access management controls as per established access policies.
- Verifying that authorized users are granted access to data and services based on their roles.
- Verify whether access policies are updated in a timely manner upon change in the roles of the users.
- Verify whether the data protection policies are sufficient.
- Assessment of support activities such as problem management

Cloud Computing for Education.

Introduction: -

Cloud computing is bringing a transformative impact in the. field of education by improving the reach of quality education to students through the use of online learning platforms and collaboration tools.

In the recent years the concept of Massively Online Open Courses (MOOCs) appears to be gaining popularity worldwide with large numbers students enrolling for online courses.

Some of the Education programs running on the Cloud platforms are listed below:

MOOCs

- MOOCs are aimed for large audiences and use cloud technologies for providing audio/video content, readings, assignment and exams.
- Cloud-based auto-grading applications are used for grading exams and assignments. Cloud-based applications for peer grading of exams and assignments are also used in some MOOCs

Online Programs

- Many universities across the world are using cloud platforms for providing online degree programs.
- Lectures are delivered through live/recorded video using cloud-based content delivery networks to students across the world.

Online Proctoring

- Online proctoring for distance learning programs is also becoming popular through the use of cloud-based live video streaming technologies where online proctors observe test takers remotely through video.

Virtual Labs

- Access to virtual labs is provided to distance learning students through the cloud. Virtual labs provide remote access to the same software and applications that are used by students on campus.

Course Management Platforms

- Cloud-based course management platforms are used to for sharing reading materials, providing assignments and releasing grades, for instance.
- Cloud-based collaboration applications such as online forums, can help student discuss common problems and seek guidance from experts.

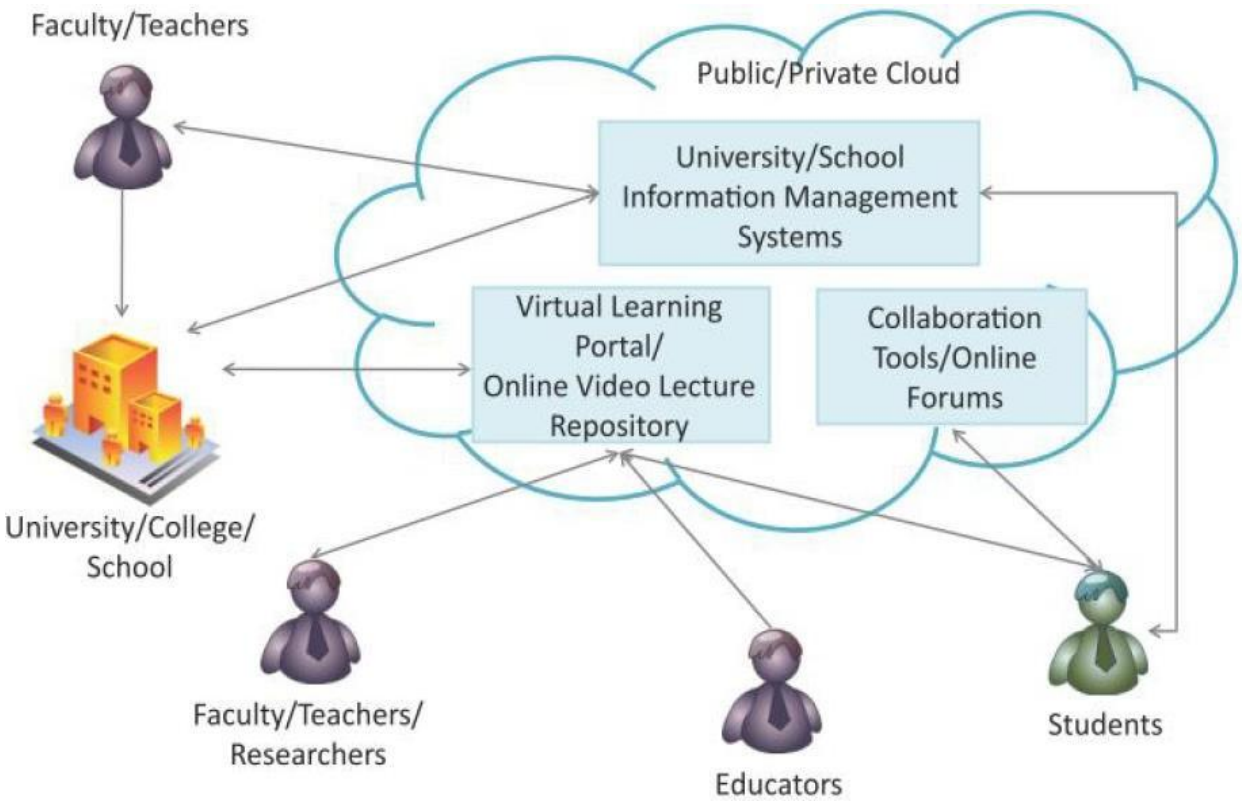
Information Management

- Universities, colleges and schools can use cloud-based information management systems to improve administrative efficiency, offer online and distance education programs, online exams, track progress of students, collect feedback from students, for instance.

Reduce Cost of Education

- Cloud computing thus has the potential of helping in bringing down the cost of education by increasing the student-teacher ratio through the use of online learning platforms and new evaluation approaches without sacrificing quality.

Below diagram shows the generic use of the Cloud for Education:



Cloud Computing for Healthcare: Cloud computing in healthcare helps securely store patient records and medical data, allowing healthcare providers to access it anytime, anywhere. It also supports telemedicine, where patients can consult with doctors online, improving access to care and efficiency in treatment.

Cloud Computing for Energy Systems: Cloud computing in energy helps monitor and manage energy usage in real-time. It allows energy companies to predict when equipment may fail, schedule maintenance, and improve how energy is distributed, making energy use more efficient and reliable.

Cloud Computing for Transportation Systems: In transportation, cloud computing enables real-time tracking of vehicles, route optimization, and monitoring traffic conditions. This helps improve logistics, makes public transport more efficient, and ensures goods and people reach their destinations more quickly.

Cloud Computing for Manufacturing Industry: Cloud computing supports manufacturing by connecting machines and systems via the Internet of Things (IoT). This allows for real-time data analysis, predicting when machines need maintenance, and optimizing the supply chain, leading to cost savings and smoother operations.

Migrating to the Cloud: A Comprehensive Overview

Introduction: Migrating to the cloud involves the process of moving an organization's data, applications, and IT services from on-premises infrastructure to a cloud environment. Cloud migration offers benefits such as improved scalability, flexibility, cost-efficiency, and access to innovative technologies. It enables businesses to scale resources as needed, reduce overhead costs associated with maintaining physical infrastructure, and improve disaster recovery and data redundancy.

Broad Approaches to Migrating to the Cloud:

1. **Lift and Shift (Rehosting):**
    - This approach involves moving applications and data to the cloud with minimal or no modification. It is the quickest way to migrate, but it doesn't take full advantage of cloud-native features.
    - **Example:** Migrating virtual machines from on-premises to the cloud without changing the application architecture.
2. **Replatforming:**
    - In this approach, applications are moved to the cloud with some optimizations or changes made to leverage cloud features such as auto-scaling or managed databases. This method requires more effort than lift-and-shift but helps optimize performance in the cloud.
    - **Example:** Moving a database from on-premises to a fully managed cloud database service like Amazon RDS or Azure SQL.
3. **Refactoring (Re-architecting):**
    - This is a more advanced approach where applications are redesigned to fully leverage cloud-native features such as microservices, containerization, and serverless computing. Refactoring maximizes cloud benefits but requires substantial development resources and time.
    - **Example:** Rebuilding an on-premises application to operate as a cloud-native app using Kubernetes and serverless functions.
4. **Hybrid Cloud:**
    - In this approach, some applications remain on-premises while others are migrated to the cloud. This allows organizations to maintain control over critical legacy systems while benefiting from the cloud for newer, scalable applications.
    - **Example:** Running sensitive workloads on-premises while leveraging cloud resources for less critical applications.
5. **Multicloud:**
    - The multicloud approach involves using multiple cloud providers (e.g., AWS, Azure, Google Cloud) to avoid vendor lock-in and improve redundancy and resilience. This allows businesses to pick the best cloud platform for different workloads.
    - **Example:** Using AWS for storage, Azure for AI services, and Google Cloud for data analytics.

The Seven-Step Model of Cloud Migration:

1. **Planning:**
    - **Objective:** Establish the migration goals, timeline, and budget.
    - **Actions:** Assess the current IT infrastructure, identify which workloads will move to the cloud, and determine the type of cloud (public, private, or hybrid).
    - **Outcome:** Clear objectives, scope, and resource requirements.
2. **Assessment:**
    - **Objective:** Evaluate the readiness of the organization for cloud migration.
    - **Actions:** Perform a detailed assessment of current workloads, performance requirements, security needs, compliance regulations, and costs.
    - **Outcome:** A thorough understanding of which workloads are suitable for the cloud and what changes may be needed.
3. **Design:**
    - **Objective:** Design the cloud architecture that meets the organization's needs.
    - **Actions:** Choose cloud services and models (e.g., IaaS, PaaS, SaaS), determine network architecture, storage needs, and ensure security and compliance.
    - **Outcome:** A tailored cloud architecture blueprint.
4. **Proof of Concept (PoC):**
    - **Objective:** Test the migration process on a smaller scale to validate the approach.
    - **Actions:** Select a small set of non-critical applications or data to migrate first. Assess performance, security, and operational aspects.
    - **Outcome:** Confirmed migration strategy, addressing any potential issues.
5. **Migration Execution:**
    - **Objective:** Begin the full migration of data, applications, and workloads to the cloud.
    - **Actions:** Move the selected applications and data to the cloud using the chosen migration approach (e.g., lift-and-shift, replatforming). Monitor for disruptions and optimize performance.
    - **Outcome:** Successful migration of workloads to the cloud.
6. **Optimization:**
    - **Objective:** Ensure the cloud environment is running efficiently and cost-effectively.
    - **Actions:** Monitor cloud resources, optimize for cost savings (e.g., scale up or down, utilize reserved instances), and enhance performance. Leverage cloud-native features like auto-scaling and load balancing.
    - **Outcome:** An optimized cloud environment that meets performance and budget goals.

7. **Ongoing Management:**
    - **Objective:** Maintain and improve the cloud environment continuously.
    - **Actions:** Regularly monitor performance, security, and compliance. Implement patches, updates, and new features as needed. Prepare for future scaling and evolving business requirements.
    - **Outcome:** A stable and resilient cloud infrastructure that supports ongoing business growth and innovation.

Key Takeaways:

- **Cloud migration** offers substantial benefits like scalability, flexibility, and reduced costs, but requires careful planning and execution.
- **Approaches** vary based on organizational needs: from simple rehosting to full re-architecture for cloud-native optimization.
- A **seven-step migration process** ensures a structured and smooth transition, starting from assessment and planning to ongoing management after the migration is complete.

Migrating to the cloud can transform an organization's IT strategy, improve operational efficiency, and support innovation. By following these steps and choosing the right migration approach, businesses can make a seamless transition to the cloud while maximizing benefits.

******ALL THE BEST******